

Quantum Reservoir Computing for Renewable Energy and Climate Prediction:

A Research Review and Implementation Handbook

Prepared for the QuEST @EEE research programme

3 July 2026; revised 11 July 2026 (Parts IX–XI added)

Abstract

Quantum reservoir computing (QRC) trains only a linear read-out on the measured observables of a fixed, input-driven quantum system, sidestepping the trainability problems of variational quantum circuits while inheriting the temporal-processing strengths of classical reservoir computing. Renewable-energy and climate time series — solar irradiance, wind power, electric load, and coupled ocean–atmosphere indices — are a natural application domain: they are abundantly available, exhibit memory and nonlinearity on many time scales, and support an economically important forecasting industry against which any quantum claim can be honestly benchmarked. This document is both a formal research review and an implementation handbook. It develops classical reservoir computing and its guarantees (echo-state property, fading memory, universality); formulates QRC in the language of quantum channels and derives where nonlinearity, memory, and measurement enter; surveys the principal reservoir architectures and hardware platforms; provides complete, line-by-line-explained implementations in Cirq, Qiskit, and PennyLane, including transpilation to native gate sets, realistic noise, finite-shot estimation, and mid-circuit measurement with reset; catalogues and analyses public renewable-energy and climate datasets with full exploratory data analysis of three of them; assembles end-to-end forecasting pipelines that share one mathematical structure across frameworks; and closes with design principles and ten concrete research directions. All numerical results in this document were produced by the code shipped alongside it, every experiment reports size-matched classical baselines and structureless quantum controls, and no claim of quantum advantage is made where none was observed.

How to read this document

Parts I and II build the theory in a linear order and are prerequisites for everything else. Part III (architectures) and Part V (hardware) are reference material: they can be read selectively. Part IV is the software handbook and is best read at a keyboard, alongside the `code/` directory that ships with this document. Part VI introduces the application domain and its data; Part VII assembles the pieces into complete pipelines; Part VIII distils design judgement and proposes research programmes. A reader who already knows classical reservoir computing can start at Part II; a reader who wants to forecast something by tonight can start at Part VII and backtrack as needed. Parts IX–XI, added in the July 2026 revision, are forward-looking: Part IX designs the staged repository that consolidates this document’s code with the qubit-reuse compiler, Part X gives the theory and pre-registered implementation plan for the recurrence-free reservoir with virtual-node readout, and Part XI wraps the forecast core in a full product pipeline with a survey of classical ML/DL and agentic-AI post-processing.

Epistemic conventions

Three kinds of statement appear in this document and are kept explicitly distinct. *Theory* means a mathematical statement with stated assumptions, attributed to the literature or derived in the text. *Simulation* means a numerical result produced by the exact density-matrix or statevector code in `code/`, on either synthetic surrogate data (always labelled *surrogate*) or real recorded data (always labelled *real*). *Hardware* means a result reported in the literature from a physical quantum device; this document runs no hardware jobs itself, but Part IV’s Qiskit pipeline is written exactly as a hardware submission would be, including transpilation, native basis gates, coupling constraints, realistic noise, and finite shots. Throughout, reported errors are normalised mean-square errors (NMSE), for which predicting the training-set mean scores 1.0; any method worth attention must sit well below that line, and every figure that shows an NMSE also shows it.

Contents

Notation	7
Part I. Foundations of Classical Reservoir Computing	8
1 The reservoir computing paradigm	8
2 Echo state networks	8
2.1 State update and read-out	8
2.2 Ridge regression: derivation and practice	8
3 What makes a good reservoir: the formal properties	9
3.1 Echo state property	9
3.2 Fading memory	9
3.3 Separation and the memory–nonlinearity trade-off	10
3.4 Linear memory capacity	10
3.5 Universality	10
4 Operating point: the edge of chaos	11
5 Reservoir computing for chaotic and geophysical systems	11
6 Baseline discipline	11
Part II. Quantum Foundations and the Quantum Reservoir Computing Formulation	12
7 States, observables, and measurement	12
8 Dynamics: unitaries, channels, and the Lindblad equation	12
8.1 Closed systems	12
8.2 Quantum channels	12
8.3 Continuous-time open dynamics	13
9 The quantum reservoir computing formulation	13
9.1 State update and read-out	13
9.2 Where the nonlinearity enters	13
9.3 Quantum echo state property and fading memory	14
9.4 Universality	14
10 Extracting features: multiplexing and measurement back-action	15
10.1 Temporal and spatial multiplexing	15
10.2 Measurement back-action: the four protocols	15
10.3 Shot noise	15
10.4 Exponential concentration	15
Part III. Architectures for Quantum Reservoirs	17
11 Ising / disordered-Hamiltonian reservoirs	17
12 Spin-chain reservoirs with local structure	17
13 Random-circuit reservoirs (restart protocol)	18
14 Measurement-based and feedback reservoirs	18
15 Dissipative and noise-driven reservoirs	18

16 Continuous-variable and photonic reservoirs	19
17 Rydberg-atom analog reservoirs	19
18 Superconducting-circuit reservoirs	19
19 Trapped-ion reservoirs	20
20 Hybrid quantum–classical reservoirs	20
21 Comparison	20
Part IV. Implementation Handbook: Cirq, Qiskit, and PennyLane	22
22 The code that ships with this document	22
23 An exact reference implementation	22
23.1 The validation suite	24
24 Cirq: transparent circuits and exact expectations	24
25 Qiskit: everything between algorithm and device	27
25.1 Transpilation: coupling maps and basis gates	30
25.2 Noise models and finite shots	30
25.3 Read-out, honestly regularised, and what the numbers say	30
25.4 Mid-circuit measurement and reset	31
26 PennyLane: batching and the differentiable boundary	31
27 Shot noise in practice	33
28 Choosing a framework	34
Part V. Hardware Considerations	36
29 IBM superconducting processors (primary target)	36
30 Quantinuum trapped-ion systems	36
31 IonQ trapped-ion systems	37
32 Rigetti superconducting processors	37
33 Neutral-atom analog platforms (QuEra, Pasqal)	37
34 Photonic platforms (Xanadu-class CV)	37
35 Cross-platform summary	37
Part VI. Renewable-Energy and Climate Data: Catalogue and Analysis	39
36 Why this application domain	39
37 Dataset catalogue	39
38 Three datasets in depth	39
38.1 Solar irradiance (surrogate, NSRDB-like)	39
38.2 System load (surrogate, GEFCom-like)	41
38.3 ENSO sea-surface temperature (real data)	41

Part VII. End-to-End Forecasting Pipelines	43
39 One pipeline, three tasks	43
40 Reference experiments	43
40.1 Choosing the operating point: the temporal edge	43
40.2 Memory capacity	43
40.3 Task demonstrations	44
40.4 The shot budget	44
41 Task walkthroughs	45
41.1 Solar: 30-minute clear-sky index	45
41.2 Load: day-ahead hourly demand (specified design)	46
41.3 ENSO: three-month anomaly	46
42 Uncertainty and evaluation beyond point NMSE	46
Part VIII. Design Principles and Research Directions	48
43 Design principles	48
44 Detecting real quantum utility	48
45 Research directions	49
46 Closing statement	50
Part IX. A Staged Repository: From NumPy Reservoir to Product	51
47 Design rules the repository inherits	51
48 The stage ladder	51
48.1 Stages 0–5: consolidating what exists	52
48.2 Stages 6–8: the new work	53
49 Promotion protocol	53
Part X. Recurrence-Free QRC with Virtual-Node Readout: Theory and Implementation Plan	54
50 The architectural improvement, precisely stated	54
51 Theory: why this should work, and where it can fail	54
51.1 The function class and its honest limitation	54
51.2 Echo-state property by construction	55
51.3 Capacity and the virtual-node lever	55
51.4 Nonlinearity, re-uploading, and the encoding-wrap trap	55
51.5 Operating point: the two edges	55
51.6 Shot noise	55
51.7 Failure modes kept in view	55
51.8 Fit to the qubit-reuse stack	56
52 Evidence base	56
53 Integration with the existing codebase	56
54 Phased plan with acceptance criteria	56

55 Pre-registered success criteria and claim discipline	58
56 Compute and risks	58
Part XI. From Pipeline to Product: Extreme-Climate Forecasting for Users	59
57 Product definition	59
57.1 Personas and their contracts	59
58 End-to-end architecture	59
59 Implementation Phases 8–12	61
60 Economics and operational honesty	61
61 Survey: classical ML/DL and agentic AI at the post-processing end	62
61.1 Statistical post-processing	62
61.2 Conformal prediction: guarantees instead of accuracy claims	62
61.3 Machine-learned stackers over hybrid features	62
61.4 Deep heads and foundation models: two distinct roles	62
61.5 Regime-aware routing	62
61.6 Agentic AI at the user boundary	62
61.7 Adoption design	63
62 Risks specific to the product layer	63
63 Deliverables	63
References	65

Notation

Symbol	Meaning
$u_t \in \mathbb{R}^{d_{\text{in}}}$	input at discrete time t (scalar unless stated)
$y_t, \hat{y}_t$	target and predicted output
$x_t \in \mathbb{R}^{N_x}$	classical reservoir state / extracted feature vector
$W_{\text{res}}, W_{\text{in}}, b$	fixed reservoir, input, and bias weights
W_{out}	trained linear read-out
$f(\cdot)$	elementwise activation (typically tanh)
α	leak rate of a leaky-integrator reservoir
ρ_t	reservoir density matrix at time t
$\mathcal{H}, d = 2^n$	Hilbert space and its dimension for n qubits
$\mathcal{E}_{\theta, u}$	input-dependent CPTP map (quantum reservoir update)
K_j	Kraus operators, $\sum_j K_j^\dagger K_j = \mathbb{I}$
$\mathcal{L}$	Lindblad generator
$O_k, x_t^{(k)} = \text{Tr}(O_k \rho_t)$	measured observable and feature
X_i, Y_i, Z_i	Pauli operators on qubit i
H, J_{ij}, h_i	Hamiltonian, couplings, fields
Δt	input injection interval (evolution time per step)
V	number of virtual nodes (temporal multiplexing)
S	number of measurement shots
L (or W)	input-window length in restart protocols
λ	ridge-regression penalty
MC, MC_d	total and delay- d linear memory capacity
NMSE	mean-square error normalised by target variance
$\langle r \rangle$	mean level-spacing ratio (chaos diagnostic)
T_1, T_2	energy relaxation and dephasing times

Part I

Foundations of Classical Reservoir Computing

1 The reservoir computing paradigm

Reservoir computing (RC) is a way of using a fixed dynamical system as a computational resource. A time-dependent input drives the system; the system’s internal state, by virtue of its own dynamics, becomes a rich nonlinear function of the input history; and a simple *linear* map from the state to the output is the only component that is ever trained. The paradigm emerged twice independently around 2001–2002: as *echo state networks* (ESNs) in the machine-learning community [1, 2] and as *liquid state machines* in computational neuroscience [3]; the two were later unified under the reservoir computing name [4, 5].

The appeal of the paradigm is threefold. First, training is a linear least-squares problem — convex, closed-form, and cheap — so the pathologies of training recurrent networks by gradient descent (vanishing and exploding gradients through time) never arise. Second, because the dynamical core is fixed, *any* sufficiently rich dynamical system can serve as the reservoir; this is what opens the door to physical and, in this document, quantum reservoirs. Third, despite the simplicity of training, reservoir computers are provably universal approximators for an important class of input–output maps (Section 3.5) and are competitive in practice on chaotic time-series prediction, a task family directly relevant to weather and climate [6–8].

Figure 1 fixes the three-layer anatomy used throughout this document: an *encoding* (input layer), a fixed *reservoir*, and a trained linear *read-out*. Every architecture in Part III, quantum or classical, is an instance of this anatomy; design work happens almost entirely in the first two layers, and honesty in evaluation happens in the third.

2 Echo state networks

2.1 State update and read-out

The standard ESN maintains a state $x_t \in \mathbb{R}^{N_x}$ updated as

$$x_{t+1} = (1 - \alpha) x_t + \alpha f(W_{\text{res}} x_t + W_{\text{in}} u_{t+1} + b), \quad \alpha \in (0, 1], \quad (1)$$

whose leak rate α sets the intrinsic time scale of the reservoir and is the single most important hyperparameter to match to the autocorrelation time of the data (a point that returns, in quantum form, as the injection interval Δt in Parts II and VIII). The output is linear in the (possibly augmented) state $\tilde{x}_t = (x_t^\top, 1)^\top$:

$$\hat{y}_t = W_{\text{out}} \tilde{x}_t. \quad (2)$$

Three conventions from Eq. (??) onward hold throughout this document. First, a *washout*: the first T_w states, which still remember the arbitrary initial condition, are discarded before training. Second, *chronological splits*: training data strictly precede validation data, which strictly precede test data; shuffled splits leak information through autocorrelation and are one of the most common sources of inflated results in time-series work. Third, all errors are reported as the normalised mean-square error

$$\text{NMSE} = \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}, \quad (3)$$

for which the constant predictor $\hat{y}_t = \bar{y}$ scores exactly 1.

2.2 Ridge regression: derivation and practice

Collect the post-washout training states row-wise into $X \in \mathbb{R}^{T \times (N_x + 1)}$ and the targets into $y \in \mathbb{R}^T$. Ordinary least squares minimises $\|Xw - y\|_2^2$, but reservoir states are strongly collinear (neighbouring neurons see overlapping input histories), so the normal-equations matrix $X^\top X$ is ill-conditioned and the unpenalised solution overfits noise in the state trajectory. Ridge (Tikhonov) regression minimises instead

$$J(w) = \|Xw - y\|_2^2 + \lambda \|w\|_2^2, \quad \lambda > 0. \quad (4)$$

Setting $\nabla_w J = 2X^\top(Xw - y) + 2\lambda w = 0$ gives the closed form

$$w^* = \left(X^\top X + \lambda \mathbb{I}\right)^{-1} X^\top y. \quad (5)$$

Since $X^\top X \succeq 0$, the regularised matrix $X^\top X + \lambda \mathbb{I} \succ 0$ is always invertible, and the objective (4) is strictly convex, so w^* is the unique global

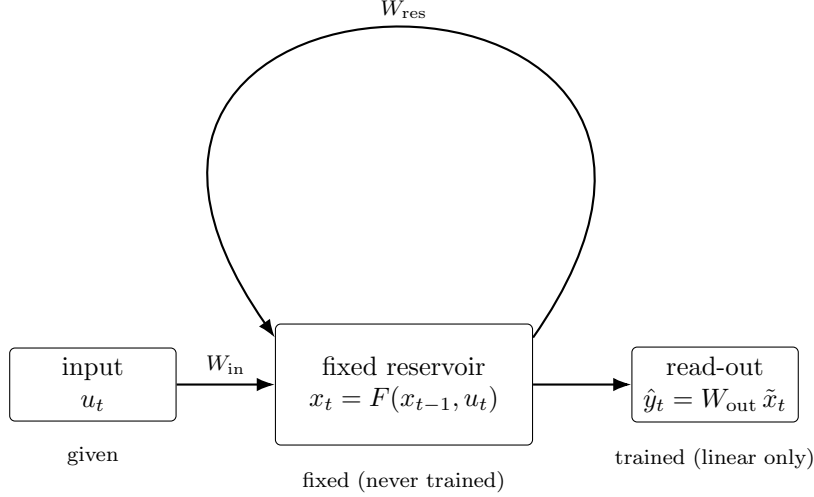


Figure 1: The three-layer anatomy of reservoir computing. Only the read-out is trained; the recurrent core is fixed at initialisation. In quantum reservoir computing (Part II) the middle block becomes an input-driven quantum channel and the read-out acts on measured expectation values.

minimiser. In the singular-value decomposition $X = U\Sigma V^\top$ with singular values s_i , Eq. (5) becomes

$$w^* = \sum_i \frac{s_i}{s_i^2 + \lambda} \left(u_i^\top y \right) v_i, \quad (6)$$

which exhibits ridge as a *spectral filter*: directions of the state space with $s_i^2 \gg \lambda$ pass through essentially untouched, while directions with $s_i^2 \ll \lambda$ — precisely the poorly excited, noise-dominated ones — are shrunk toward zero. This picture matters doubly in the quantum setting, where each feature carries shot noise of known magnitude (Part IV): λ is then not a nuisance parameter but the dial that trades the reservoir’s expressivity against its measurement noise. The penalty is selected on held-out data — either a chronological validation block or, when that block is too short to be representative, by generalised cross-validation on the training span; it is never tuned on the test set.

3 What makes a good reservoir: the formal properties

Three properties, introduced for ESNs but stated here in a form that transfers verbatim to quantum reservoirs, separate systems that can compute from systems that merely fluctuate.

3.1 Echo state property

Definition 3.1 (Echo state property). *A driven system with update $x_{t+1} = F(x_t, u_{t+1})$ on a compact state set has the echo state property (ESP)*

if for every input sequence $\{u_t\}$ and every pair of initial conditions x_0, x'_0 , the resulting trajectories converge: $\|x_t - x'_t\| \rightarrow 0$ as $t \rightarrow \infty$. Equivalently, after a transient the state is a function of the input history alone, $x_t = \phi(\dots, u_{t-1}, u_t)$, independent of initialisation [1].

The ESP is what makes the washout finite and the learned read-out reproducible: without it, the “features” depend on an arbitrary initial condition and the training problem is ill-posed. For the tanh ESN, a sufficient condition is that the largest singular value of W_{res} be below 1; the everyday design rule of setting the *spectral radius* $\rho_{\text{sr}}(W_{\text{res}})$ slightly below 1 is a necessary condition (for zero input) and works well in practice, though it is not sufficient in general [1, 5]. Extensions of the ESP to non-stationary systems and to subsystems — needed when only part of a larger system is observed, as is typical for quantum reservoirs — are developed in Kobayashi et al. [10].

3.2 Fading memory

Definition 3.2 (Fading memory). *A causal, time-invariant operator M mapping input sequences to outputs has fading memory if inputs that agree on a recent window and differ only in the remote past produce outputs that are close: for every $\varepsilon > 0$ there exist $\delta > 0$ and a decreasing weight sequence $w_k \downarrow 0$ such that $\sup_k w_k |u_{t-k} - u'_{t-k}| < \delta$ implies $|M[u]_t - M[u']_t| < \varepsilon$ [11].*

Fading memory formalises the intuition that a forecaster should care about the recent past more

Algorithm 1 Echo state network training (batch, ridge read-out)

- 1: draw $W_{\text{in}}, W_{\text{res}}, b$ at random; rescale W_{res} to spectral radius $\rho_{\text{sr}} < 1$
 - 2: $x_0 \leftarrow 0$
 - 3: **for** $t = 1, \dots, T_{\text{total}}$ **do**
 - 4: $x_t \leftarrow (1 - \alpha)x_{t-1} + \alpha f(W_{\text{res}}x_{t-1} + W_{\text{in}}u_t + b)$
 - 5: **end for**
 - 6: discard $t \leq T_w$ (washout); stack remaining training states into X , targets into y
 - 7: $W_{\text{out}} \leftarrow (X^\top X + \lambda \mathbb{K})^{-1} X^\top y$ $\triangleright \lambda$ chosen on validation data
 - 8: **return** predictions $\hat{y}_t = W_{\text{out}}\tilde{x}_t$ on the untouched test span
-

than the distant past, with influence decaying continuously. Boyd and Chua [11] proved that operators with fading memory are exactly those approximable by Volterra series — the classical result that underlies all reservoir universality theorems. In a reservoir with the ESP, fading memory is automatic: the same contraction that erases the initial condition also erases the remote input past. The tension at the heart of reservoir design is that *too much* contraction erases the recent past as well; the useful regime lies near, but on the stable side of, the boundary (Section 4).

3.3 Separation and the memory–nonlinearity trade-off

The read-out is linear, so any two input histories that must be mapped to different outputs must first be mapped, by the reservoir, to linearly distinguishable states. This *separation property* [3] is the reservoir’s expressivity requirement, complementary to the ESP’s stability requirement. A quantitative unification is the information-processing capacity of Dambre et al. [12]: for a reservoir with N_x linearly independent state variables driven by i.i.d. input, the total capacity to reconstruct *any* orthonormal set of functions of the input history (linear or nonlinear) is bounded by N_x , and saturates it for fading-memory systems. Capacity spent on nonlinear functions is capacity not available for linear memory — the *memory–nonlinearity trade-off* that every reservoir, classical or quantum, must budget for the task at hand.

3.4 Linear memory capacity

The standard operational probe of memory is Jaeger’s *memory capacity* [1]. Drive the reservoir with i.i.d. scalar input u_t , train a separate linear read-out to reconstruct each delayed input

$y_t = u_{t-d}$, and define

$$\begin{aligned} \text{MC}_d &= \max_w \frac{\text{cov}^2(u_{t-d}, w^\top \tilde{x}_t)}{\text{var}(u_{t-d}) \text{var}(w^\top \tilde{x}_t)}, \\ \text{MC} &= \sum_{d=1}^{\infty} \text{MC}_d. \end{aligned} \tag{7}$$

Each $\text{MC}_d \in [0, 1]$ is the fraction of the delay- d input variance linearly recoverable from the state; $\text{MC} \leq N_x$ with equality approached by linear reservoirs [1, 12]. In this document MC is estimated with delays up to $d_{\text{max}} = 25$ and reported alongside every reservoir comparison (Part VII), because a forecasting model’s usable history is bounded by it.

3.5 Universality

The formal justification for training only a linear read-out is a family of universality theorems. For ESNs, Grigoryeva and Ortega [13] proved that echo state networks with linear read-outs are universal: any causal, time-invariant fading-memory filter on uniformly bounded inputs can be approximated to arbitrary accuracy by an ESN of sufficient size. The proof route — show the family of reservoir functionals forms a polynomial algebra that separates points, then apply Stone–Weierstrass — is worth internalising because it transfers: universality proofs for quantum reservoirs (Part II, Section 9.4) follow the same architecture, with the algebra generated by measured observables of the driven quantum state [14–16]. Two caveats temper the theorem’s practical force. Universality is an existence statement about *some* sufficiently large reservoir, not a statement that a random one of fixed size is good; and it says nothing about sample efficiency. Both gaps are exactly where empirical reservoir quality — and any prospective quantum edge — lives.

4 Operating point: the edge of chaos

For a fixed task, reservoir performance as a function of the dynamical regime is strikingly non-monotonic. Deep in the ordered (strongly contracting) regime the state barely responds to input and memory is short; deep in the chaotic regime the ESP fails and the state amplifies microscopic differences until the input signal is scrambled. Performance typically peaks near the order–chaos boundary — the *edge of chaos* — where correlations are long but reproducibility survives [17]. For ESNs the practical dials are the spectral radius ρ_{sr} , the input scaling, and the leak rate α ; systematic constraints on their joint choice are mapped in Storm et al. [18]. Two further classical lessons recur in the quantum setting. First, reservoirs whose dynamics have a symmetry the task must break (for the tanh ESN, the odd symmetry $f(-z) = -f(z)$) fail on symmetric tasks unless the symmetry is explicitly broken, e.g. by biases [19]; the quantum analogue — symmetries of the reservoir Hamiltonian polluting chaos diagnostics and constraining which observables carry information — appears in Part VIII. Second, the edge is a property of the *driven* system, not the autonomous one: input strength shifts it, so the operating point must be tuned with the data in the loop. In quantum many-body reservoirs the edge acquires a second, temporal dimension (the injection interval relative to the Thouless time), identified by Kobayashi and Motome [20] and reproduced numerically in Part VII of this document.

5 Reservoir computing for chaotic and geophysical systems

The application benchmark that made modern RC famous is model-free prediction of chaos. Jaeger and Haas [2] demonstrated orders-of-magnitude improvements in Mackey–Glass [21] prediction; Pathak et al. [6] showed that a parallelised ESN can forecast the Kuramoto–Sivashinsky equation for multiple Lyapunov times and reproduce its climate (attractor statistics); hybrids of reservoirs with imperfect knowledge-based models outperform either alone [22]; and Arcomano et al. [7] scaled the approach to a global atmospheric model, establishing RC as a serious tool for the weather–climate problem class this document targets. Careful comparisons find well-tuned reser-

voirs competitive with, and far cheaper to train than, backpropagated recurrent networks on these tasks [8]. Chaotic benchmarks (Lorenz-63 [23], Mackey–Glass) therefore serve in this document as controlled stand-ins for the harder, noisier renewable and climate series of Part VI, and the NARMA family serves as the standard memory-plus-nonlinearity stress test.

6 Baseline discipline

Because Parts VI–VIII evaluate quantum reservoirs against economically meaningful forecasting tasks, the classical baselines are fixed here once and used everywhere. (i) *Mean predictor*: $\text{NMSE} = 1$ by construction; drawn as a horizontal line on every error axis. (ii) *Persistence*: $\hat{y}_{t+h} = y_t$; embarrassingly strong on slowly varying series such as solar irradiance at short horizons, and the baseline most often missing from overclaiming studies. (iii) *Linear auto-regression on p lags*: the cheapest model with memory; any nonlinear reservoir must beat it to justify its existence. (iv) *Size-matched ESN*: Eqs. (1) and (5) with the number of read-out features equal to the quantum feature count, so comparisons are at matched read-out capacity. A quantum reservoir result in this document is reported as interesting only relative to all four, and Part VIII adds a fifth, quantum-internal control: a structureless (Haar-random) quantum reservoir of the same size.

Part II

Quantum Foundations and the Quantum Reservoir Computing Formulation

7 States, observables, and measurement

This part fixes the quantum-mechanical machinery in exactly the depth the rest of the document uses; standard references are Nielsen and Chuang [24] for closed-system and circuit material and Breuer and Petruccione [25] for open systems.

A system of n qubits lives on the Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$ of dimension $d = 2^n$. Pure states are unit vectors $|\psi\rangle \in \mathcal{H}$; the general state is a *density matrix* ρ , a positive semidefinite operator with $\text{Tr } \rho = 1$, encompassing pure states ($\rho = |\psi\rangle\langle\psi|$, $\text{Tr } \rho^2 = 1$) and statistical mixtures ($\text{Tr } \rho^2 < 1$). Two structural facts about ρ drive everything in this document. First, the *exponential dimension*: ρ has $d^2 - 1 = 4^n - 1$ real parameters, so an n -qubit reservoir carries a state space whose dimension no polynomial-size classical reservoir matches parameter-for-parameter. Second, the *partial trace*: for a bipartite system AB , the state of B alone is $\rho_B = \text{Tr}_A \rho_{AB}$, the unique operator reproducing all local measurement statistics on B . The partial trace is the mechanism by which the Fujii–Nakajima reservoir (Section 9.1) forgets its input qubit while retaining correlations in the rest — memory by entanglement.

Observables are Hermitian operators O ; the Born rule gives the expectation value $\langle O \rangle = \text{Tr}(O\rho)$. The n -qubit Pauli operators $\{\mathbb{I}, X, Y, Z\}^{\otimes n}$ form an orthogonal operator basis, so the set of expectation values of all Pauli strings determines ρ completely; a reservoir read-out measures a small, fixed subset of them — in this document the single-qubit $\langle Z_i \rangle$, $\langle X_i \rangle$ and two-qubit $\langle Z_i Z_j \rangle$. The most general measurement is a positive operator-valued measure (POVM): a set $\{E_m\}$ with $E_m \succeq 0$ and $\sum_m E_m = \mathbb{I}$, outcome m occurring with probability $p_m = \text{Tr}(E_m \rho)$. Projective measurement of a qubit in the computational basis is the special case $E_0 = |0\rangle\langle 0|$, $E_1 = |1\rangle\langle 1|$, and is the only primitive current digital hardware exposes; every “expectation value” in an experiment is a finite-sample average of projective outcomes, a fact whose statistical consequences (Section 10.3) shape reservoir design as strongly as any Hamiltonian choice.

8 Dynamics: unitaries, channels, and the Lindblad equation

8.1 Closed systems

A closed system evolves unitarily, $\rho \mapsto U\rho U^\dagger$ with $U = e^{-iHt}$ generated by a Hamiltonian H (units with $\hbar = 1$ throughout). Unitary evolution is an isometry of state space: it can never contract two distinct initial states toward each other. An immediate and frequently overlooked consequence for reservoir computing is that *a strictly closed quantum system cannot have the echo state property*: initial-condition differences persist forever, so fading memory (Definition 3.2) fails. Every working quantum reservoir therefore contains an explicitly non-unitary element — an input-injection step that discards a subsystem, engineered dissipation, or measurement — and identifying where that contraction enters is the first question to ask of any proposed architecture [14, 26, 27].

8.2 Quantum channels

The general physical evolution of a possibly open system is a *quantum channel*: a completely positive, trace-preserving (CPTP) linear map $\mathcal{E}$ on density matrices. Every channel admits a Kraus representation

$$\mathcal{E}(\rho) = \sum_j K_j \rho K_j^\dagger, \quad \sum_j K_j^\dagger K_j = \mathbb{I}, \quad (8)$$

and, equivalently (Stinespring), can be realised as a unitary on the system plus an environment followed by a partial trace over the environment. Complete positivity (positivity of $\mathcal{E} \otimes \mathbb{I}$ on any enlarged system) is what guarantees the map remains physical when the system is entangled with a bystander; trace preservation is probability conservation. Channels are exactly the update maps a quantum reservoir may use, and their key analytic property for this document is *contractivity*: every channel is non-expansive in trace distance, $\frac{1}{2} \|\mathcal{E}(\rho) - \mathcal{E}(\sigma)\|_1 \leq \frac{1}{2} \|\rho - \sigma\|_1$, with strict contraction (for generic state pairs) precisely when the channel has a non-unitary part. Strict contractivity is the quantum form of the echo state property (Section 9.3).

A convenient linear-algebraic handle used repeatedly in Parts VII–VIII is the *Pauli transfer matrix* (PTM): expanding $\rho = \frac{1}{d} \sum_{\alpha} r_{\alpha} P_{\alpha}$ over Pauli strings P_{α} turns a channel into a real matrix T acting on the coefficient vector r , with the unital block’s spectral radius governing how fast initial-condition information decays. Kobayashi et al. [27] sharpen this picture for reservoirs: fading memory requires the input-independent part of the PTM to be strictly contractive on the traceless sector, while useful input sensitivity requires a *coherence influx* — an input-dependent affine part that keeps re-exciting the contracting state. Channels that contract without influx (for example, uniform depolarisation toward the maximally mixed state) satisfy the ESP but forget the input as fast as the initial condition, and score near NMSE = 1; the distinction is demonstrated numerically in Part IV’s noise studies.

8.3 Continuous-time open dynamics

When the environment coupling is weak and memoryless (Born–Markov), the state obeys the Gorini–Kossakowski–Sudarshan–Lindblad master equation [28, 29]

$$\begin{aligned} \dot{\rho} = \mathcal{L}(\rho) = & -i[H, \rho] \\ & + \sum_k \gamma_k \left(L_k \rho L_k^{\dagger} - \frac{1}{2} \{L_k^{\dagger} L_k, \rho\} \right), \end{aligned} \quad (9)$$

whose solution $\rho(t) = e^{\mathcal{L}t} \rho(0)$ is a CPTP semigroup. The jump operators L_k model the dissipation channels of real hardware: amplitude damping ($L = \sigma^{-} = |0\rangle\langle 1|$, rate $1/T_1$) relaxes excitation toward $|0\rangle$; pure dephasing ($L = Z$, rate contribution to $1/T_2$) destroys phase coherence without moving populations. The distinction is not cosmetic for reservoirs. Amplitude damping drives the system toward a *polarised* fixed point ($|0\rangle^{\otimes n}$) from which fresh input rotations produce large, input-dependent signals, so moderate T_1 decay can supply exactly the contraction the ESP needs while preserving responsiveness [26, 30]. Strong dephasing, by contrast, drives states toward the diagonal and, combined with depolarising elements, toward the maximally mixed state, where all bounded observables concentrate and the signal is erased — contraction without coherence influx [27, 31]. “Noise” is therefore neither friend nor enemy of QRC in general; *which* noise, at what rate, relative to the injection interval, is a design axis (Part VIII).

9 The quantum reservoir computing formulation

9.1 State update and read-out

A quantum reservoir computer replaces the classical update Eq. (??) by an input-dependent channel acting on the reservoir’s density matrix,

$$\begin{aligned} \rho_{t+1} = U_{\Delta t} \left(|\psi_{u_{t+1}}\rangle\langle\psi_{u_{t+1}}| \otimes \text{Tr}_1 \rho_t \right) U_{\Delta t}^{\dagger}, \\ |\psi_u\rangle = \sqrt{1-u} |0\rangle + \sqrt{u} |1\rangle, \end{aligned} \quad (10)$$

with $U_{\Delta t} = e^{-iH\Delta t}$ and

$$\begin{aligned} H = \sum_{i < j} J_{ij} X_i X_j + \frac{\hbar}{2} \sum_i Z_i, \\ J_{ij} \sim \mathcal{U}[-J/2, J/2]. \end{aligned} \quad (11)$$

Equation (10) is a legitimate CPTP map (a partial trace, a tensor product with a fresh state, and a unitary), and each ingredient plays a distinct role: the partial trace Tr_1 is the contraction that grants the ESP and fading memory; the re-preparation $|\psi_u\rangle\langle\psi_u|$ is the coherence influx that keeps the reservoir responsive; and the entangling unitary $U_{\Delta t}$ spreads the fresh input into the retained qubits, where it interferes with the stored history. Figure 2 shows the full pipeline including measurement and the classical read-out.

9.2 Where the nonlinearity enters

Equation (??) is *linear in ρ* , and Eq. (??) is linear too, so a natural worry is that the whole machine is linear and the read-out learns nothing an autoregression could not. The worry dissolves once one asks: *nonlinear in what?* Three distinct mechanisms make the features nonlinear in the *input history*, which is the variable that matters.

(i) *Nonlinear encoding.* The map $u \mapsto |\psi_u\rangle\langle\psi_u|$ in Eq. (10) is quadratic in the amplitudes ($\sqrt{1-u}, \sqrt{u}$), and rotation encodings $R_Y(\gamma u)$ used in Parts III–IV make expectation values trigonometric polynomials in u . For parameterised re-uploading circuits this is a theorem: the output is a truncated Fourier series in the encoded variables, with the accessible frequency spectrum set by the encoding generators and their repetitions [33]. The encoding scale γ therefore controls how much of the read-out’s approximation power sits in high harmonics — the practical saturation effect quantified in Part IV.

(ii) *Products through the update composition.* Because the input enters the channel multiplicatively (the new state is a *product* $|\psi_u\rangle\langle\psi_u| \otimes \text{Tr}_1 \rho_t$,

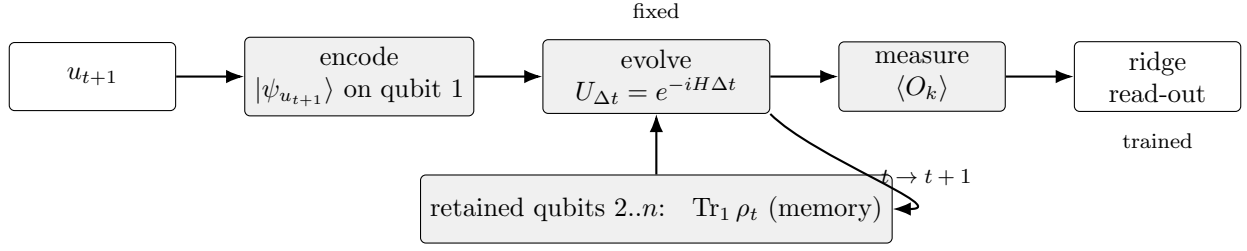


Figure 2: One step of the Fujii–Nakajima quantum reservoir, Eq. (10). The input qubit is re-prepared each step (coherence influx); the remaining qubits carry the fading memory forward; measurement of a fixed observable set feeds the only trained component, a linear read-out.

then conjugated), composing T steps produces expectation values containing products of functions of $u_t, u_{t-1}, \dots$ — exactly the cross terms a NARMA-type target needs. In the PTM picture: the affine input drive is state-dependent through the tensor-product structure, so the composed map is polynomial, not affine, in the input sequence.

(iii) *Measurement-derived nonlinearity.* Any protocol that feeds measured outcomes back into the drive (Part III’s measurement-based and feedback architectures [34, 35]), or that computes nonlinear functions of estimated expectations classically (e.g. including products $\langle Z_i \rangle \langle Z_j \rangle$ as features), imports an additional, explicitly nonlinear stage. The second-moment observables $\langle Z_i Z_j \rangle$ used throughout this document are linear in ρ but carry information about correlations that no set of single-qubit means determines, which is why adding them raised the effective feature rank so sharply in the Part IV experiments.

What the linearity of Eq. (??) *does* imply is an exact finite-dimensional bookkeeping: the reservoir state is a vector in $\mathbb{R}^{4^n - 1}$ evolving affinely for each fixed input value, so QRC never escapes the information-processing capacity bound of Dambre et al. [12] — at most $4^n - 1$ independent functionals, and in practice as many as the *measured, well-conditioned* directions support. The realistic promise of QRC is therefore not unbounded expressivity but a large, physically generated feature bank per physical component [32, 36], whose usable fraction is decided by measurement statistics (Section 10.3).

9.3 Quantum echo state property and fading memory

Definition 9.1 (Quantum ESP). *The reservoir family $\{\mathcal{E}_{\theta,u}\}$ has the echo state property if for every admissible input sequence and every pair of initial states ρ_0, ρ'_0 , $\|\rho_t - \rho'_t\|_1 \rightarrow 0$ as $t \rightarrow \infty$, where $\rho_t = \mathcal{E}_{\theta,u_t} \circ \dots \circ \mathcal{E}_{\theta,u_1}(\rho_0)$.*

Sufficient conditions mirror the classical ones with contraction coefficients replacing spectral radii: if every $\mathcal{E}_{\theta,u}$ is a strict contraction in trace distance with a uniform coefficient $c < 1$, initial-state information decays as c^t and the ESP holds together with fading memory [14, 16, 37]. For the Fujii–Nakajima update the contraction is supplied structurally by the partial trace; for dissipative reservoirs, by the Lindbladian’s spectral gap; for measurement-based schemes, by the state collapse itself. The refinements a practitioner needs are (a) the *nonstationary/subspace ESP* of Kobayashi et al. [10], which handles reservoirs where only a subsystem or a conditioned sector must forget, and (b) the coherence-influx criterion of Kobayashi et al. [27] separating useful contraction (toward an input-sensitive, polarised region) from destructive contraction (toward the maximally mixed state). Both are cheap to check numerically from the PTM and are part of the validation suite shipped with this document’s code.

9.4 Universality

Universality theorems for QRC parallel Section 3.5. Chen and Nurdin [14] established that families of dissipative quantum systems with contractive dynamics and polynomial read-outs realise dense subsets of fading-memory maps; Chen et al. [37] adapted the construction to gate-model devices with a hardware demonstration; Nokkala et al. [15] proved universality for continuous-variable Gaussian reservoirs, notable because Gaussian dynamics are efficiently classically simulable — universality alone therefore confers no quantum advantage; and Martínez-Peña and Ortega [16] gave a finite-dimensional treatment identifying which ingredients (contraction, input-dependence, read-out algebra) are load-bearing. The practical reading is the same as in the classical case: universality licenses the paradigm, and everything interesting — sample efficiency, feature quality per shot, robustness — is decided by the concrete architecture,

which is Part III’s subject.

10 Extracting features: multiplexing and measurement back-action

10.1 Temporal and spatial multiplexing

The measured feature count K , not the Hilbert-space dimension, bounds the read-out. Two standard tricks enlarge K at fixed hardware. *Temporal multiplexing* [32] samples the observables at V intermediate times within each injection interval, $x_t^{(k,v)} = \text{Tr}(O_k e^{-iHv\Delta t/V} \rho_t^{\text{inj}} e^{iHv\Delta t/V})$, multiplying the feature count by V (the *virtual nodes*); the document’s reference implementation uses $V = 4$. *Spatial multiplexing* [38] runs M independent small reservoirs on disjoint qubit sets and concatenates their features, trading entanglement breadth for shot-parallelism and noise isolation — usually the right trade on current hardware, and the reason Part VII’s pipelines shard across small registers rather than maximising register size.

10.2 Measurement back-action: the four protocols

Equation (??) silently assumes access to exact expectation values of an undisturbed state, but projective measurement collapses the state it reads. Four protocols manage the conflict, and choosing among them is a first-class architectural decision [39]:

Restart (rewind). To obtain features at step t , re-run the input sequence from scratch (in practice, from a finite window of length L , valid precisely because fading memory suppresses older inputs) and measure once at the end. Cost per time step grows with $L \times$ shots, but the state before measurement is exact. All gate-model examples in Part IV use this protocol.

Ensemble. Run many copies in parallel and measure each copy once per step — natural for NMR spin ensembles [40] and atomic ensembles, where the “copies” are physical molecules or atoms.

Weak measurement. Couple a meter weakly, extracting a little information per step at the cost of a little disturbance; Mujal et al. [39] quantify the regime where the running estimate is accurate while the reservoir dynamics remain effectively intact, and Franceschetto et al. [35] show the disturbance itself can be harnessed as a computational resource.

Mid-circuit measurement and reset. Measure a designated subset of qubits projectively each step and reset it for the next injection — the hardware-native realisation of Eq. (10)’s partial trace. Its decisive property, established theoretically and on superconducting hardware by Hu et al. [41] (the NISQRC scheme), is that the usable memory time is set by the *engineered* contraction, not by T_1/T_2 : the persistent qubits are refreshed through interaction rather than left to decohere, so temporal processing can run for durations far exceeding coherence times. Part IV demonstrates the primitive in Qiskit with per-step classical registers.

10.3 Shot noise

With S projective shots, the estimator of a Pauli expectation $z = \langle P \rangle$ is a scaled binomial average with standard deviation

$$\sigma(\hat{z}) = \sqrt{\frac{1 - z^2}{S}} \leq \frac{1}{\sqrt{S}}, \quad (12)$$

so halving the feature noise costs $4 \times$ the shots. Shot noise enters the ridge problem as noise on the *design matrix*, not the targets, which biases as well as inflates variance; its interaction with the regulariser is derived and measured in Part IV, and its fundamental consequences — an effective, noise-dependent cap on the number of usable feature directions (*eigentasks*) — are established in Hu et al. [42] and, for training specifically, in Ahmed et al. [43]. Two design rules follow and are enforced throughout this document: always measure the exact-simulation ceiling before any shot study, so that shot effects are not confused with architectural ones; and treat S as a first-class resource reported next to every hardware-relevant number, since a reservoir that needs 10^6 shots per feature to beat a 30-neuron ESN has lost on the only axis that counts.

10.4 Exponential concentration

The scaling threat specific to quantum feature maps is *concentration*: for deep scrambling dynamics, random circuits, or noise-dominated evolution, local observables concentrate exponentially in n around state-independent values, so the input-dependent signal shrinks beneath any fixed shot budget — the reservoir analogue of barren plateaus [44]. For QRC the phenomenon and its symmetry-resolved structure are analysed in Xiong et al. [31], Sannia et al. [45]: scrambling that is useful at small depth becomes signal-destroying at large depth or size, and noise accelerates the

collapse. The operational defences — moderate evolution per step, polarised fixed points, local observables on shallow light-cones, spatial multiplexing instead of monolithic growth, and empirical monitoring of feature variance and effective rank as n grows — recur as design principles in Part VIII. Concentration is the single strongest reason this document’s claims are confined to the small- n regime actually simulated, and why no extrapolation to large registers is offered anywhere.

Part III

Architectures for Quantum Reservoirs

This part surveys the principal quantum reservoir architectures. Each entry follows one template — *model and update; encoding; observables; strengths; weaknesses; hardware suitability; extensions* — so that the comparison table (Table 1) can be read as a summary rather than a substitute. Throughout, “update” refers to the concrete form taken by the channel $\mathcal{E}_{\theta,u}$ of Eq. (??), and epistemic labels follow the frontmatter conventions: entries note explicitly whether the architecture has been demonstrated on hardware or only in simulation.

11 Ising / disordered-Hamiltonian reservoirs

Model and update. The original and still most-studied QRC: n qubits under a fixed disordered Hamiltonian, canonically the transverse-field Ising form of Eq. (11), driven by the injection map Eq. (10) [32]. One time step is: replace the input qubit’s state, evolve unitarily for Δt , read observables (optionally at V intermediate times).

Encoding. State replacement $|\psi_u\rangle = \sqrt{1-u}|0\rangle + \sqrt{u}|1\rangle$ (amplitude encoding of a scalar); multivariate inputs use several input qubits or a rotation encoding $R_Y(\gamma u^{(j)})$ per component.

Observables. $\langle Z_i \rangle$ per virtual node; $\langle Z_i Z_j \rangle$ at the final virtual node; $\langle X_i \rangle$ where transverse coherences are informative.

Strengths. Conceptually clean separation of memory (retained qubits) and influx (re-prepared qubit); strong small-system benchmark record — 5–7 qubits emulating ESNs of hundreds of nodes on NARMA and Mackey–Glass in exact simulation [32, 46]; dynamical regime controllable through $(J\Delta t, h/J, \text{disorder})$ with the performance-optimal region tied to the ergodic-localised boundary [47] and, in time, to the edge of many-body chaos [20].

Weaknesses. Exact density-matrix simulation is $O(4^n)$, capping honest classical study near $n \approx 12$ –14; on gate hardware the continuous evolution must be Trotterised and the injection realised by mid-circuit reset, importing circuit noise; scalar drive on a single input qubit yields features with low effective rank unless multiplexing and two-body observables are used (measured in Part IV).

Hardware suitability. Natural on platforms with always-on couplings (NMR ensembles, where it

was demonstrated experimentally [40]; spin lattices); digital superconducting/trapped-ion realisations via Trotterisation plus mid-circuit reset [37, 41, 48].

Extensions. Higher-order reservoirs coupling multiple registers [49]; artificial memory restriction to tune the memory–nonlinearity balance [50]; engineered dissipation replacing the partial trace [26].

12 Spin-chain reservoirs with local structure

Model and update. As Section 11 but with nearest-neighbour chains or lattices, $H = \sum_i J_i \sigma_i \sigma_{i+1} + \sum_i h_i Z_i$ (Ising, XX, XXZ, or Heisenberg variants), matching hardware connectivity. On-site disorder $h_i \sim \mathcal{U}[-W, W]$ provides a clean chaos dial: sweeping W moves the level-spacing statistics between Wigner–Dyson ($\langle r \rangle \approx 0.53$, ergodic) and Poisson ($\langle r \rangle \approx 0.39$, localised), with reservoir performance peaking near the crossover [20, 47].

Encoding/Observables. As above; local observables are natural because information spreads along the chain at a finite (Lieb–Robinson) velocity, which also localises the causal cone — useful for shallow hardware implementations.

Strengths. Direct map to linear-coupling hardware (superconducting chains, ion chains, Rydberg arrays); physically meaningful dynamical-phase dial; disorder-free operating points also exist (clean lattices work in suitable regimes [51]), which matters for analog platforms where per-site disorder is hard to program.

Weaknesses. Finite information velocity means distant qubits take several steps to feel the input — memory arrives with latency; symmetries of clean chains (e.g. total- Z conservation in XXZ at zero field) restrict which observables respond and polute chaos diagnostics unless resolved by symmetry sector (Part VIII).

Hardware suitability. The best match to fixed-topology digital devices; the reference geometry for Part IV’s transpilation study.

Extensions. Two-dimensional lattices; quasiperiodic fields; programmable-geometry analog arrays (Section 17).

13 Random-circuit reservoirs (restart protocol)

Model and update. A fixed, randomly drawn shallow circuit block W (random single-qubit rotations plus an entangling brickwork) interleaved with input-encoding rotations; the reservoir “state” is implicit in the restart protocol of Section 10.2: for each t , a fresh circuit re-uploads the last L inputs, $|\phi_t\rangle = \prod_{j=1}^L [W R(u_{t-L+j})] |0\rangle^{\otimes n}$, and is measured once [37]. The recurrence-free variant of Ahmed et al. [52] formalises the window as an explicit delay embedding and shows it inherits reservoir guarantees while eliminating back-action bookkeeping entirely.

Encoding. Angle re-uploading $R_Y(\gamma g_i u)$ with fixed random gains g_i ; the output is a truncated Fourier series in the inputs with spectrum set by γ and the number of repetitions [33].

Observables. $\langle Z_i \rangle$, $\langle X_i \rangle$, $\langle Z_i Z_j \rangle$ from terminal measurement; all obtainable from two measurement bases.

Strengths. Trivially portable across gate hardware — no mid-circuit operations required; state before measurement is exact, so shot noise is the only estimator error; window length L is an honest, explicit memory budget; the architecture used by most current hardware demonstrations, including the energy-forecasting studies of Part VI [37, 53].

Weaknesses. Cost per time step scales with L (the window is re-executed each step); memory is hard-truncated at L rather than fading; as a pure feature map it can underperform genuinely recurrent reservoirs on long-memory tasks — quantified in Part IV, where the windowed PennyLane model trails the recurrent simulation on ENSO by a wide margin; deep random blocks concentrate (Section 10.4), so depth must stay moderate.

Hardware suitability. Any gate-model device; the default choice for first hardware experiments.

Extensions. Encoding-scale tuning by gradient descent (Part IV); classical-lag augmentation into the read-out; noise-as-nonlinearity variants (Section 15).

14 Measurement-based and feedback reservoirs

Model and update. Measurement is promoted from read-out to dynamical ingredient. In mid-circuit-reset (NISQRC) form, a designated I/O subset is measured and reset every step while persistent qubits carry memory [41]; in weak-measurement form, a continuous meter record is the feature

stream [35, 39]; in feedback form, measured outcomes condition subsequent gates or drives, $\mathcal{E}_{u,m_t}$ with m_t the outcome, injecting explicit nonlinearity and enabling closed-loop tasks [34].

Encoding/Observables. As the host architecture; the distinctive output is the *record* $\{m_t\}$ or per-step estimated expectations.

Strengths. Memory decoupled from T_1/T_2 (the NISQRC result — hardware-demonstrated on IBM devices [41, 48]); online operation on a single copy rather than restart; feedback adds nonlinearity a linear channel cannot express.

Weaknesses. Conditional logic and fast reset are demanding hardware primitives with their own error budgets; the conditioned dynamics are harder to analyse (the relevant ESP is the subspace/non-stationary form of Kobayashi et al. [10]); weak-measurement gain must be tuned against disturbance.

Hardware suitability. Superconducting platforms with dynamic circuits (demonstrated); trapped ions with mid-circuit detection; photonics with homodyne records (Section 16).

Extensions. Back-action-as-resource protocols [35]; reservoir-assisted quantum-state measurement, inverting the roles of system and meter [54].

15 Dissipative and noise-driven reservoirs

Model and update. The contraction required by Definition 9.1 is supplied by engineered or intrinsic open-system dynamics: evolve under Eq. (9) between injections, or, on gate hardware, let the device’s own noise channels act [14, 26]. In the “natural quantum reservoir” of Suzuki et al. [55], an IBM processor’s intrinsic dissipation supplied both the fading memory and useful nonlinearity, with the demonstration run on hardware.

Encoding. Any of the preceding; dissipation is a property of the between-injection dynamics, not the encoding.

Observables. As host architecture.

Strengths. Turns the platform’s unavoidable imperfection into the stability mechanism; fixed-point structure is designable — amplitude-damping towards a polarised state preserves input sensitivity (Section 8); noise can add task-useful nonlinearity [30, 56].

Weaknesses. The wrong noise erases the signal: dephasing/depolarising fixed points near the maximally mixed state contract without coherence influx [27, 31]; device noise drifts over calibration cycles, so a reservoir defined by it is not stationary

week-to-week — a reproducibility cost quantified only by re-benchmarking.

Hardware suitability. Everywhere, by definition; deliberately engineered dissipation is most controllable in circuit QED and trapped ions.

Extensions. Optimal-dissipation design [26]; dissipative phase transitions as operating points; noise-aware training [43].

16 Continuous-variable and photonic reservoirs

Model and update. Modes of the electromagnetic field replace qubits; states live in an infinite-dimensional Fock space. Gaussian reservoirs — linear optical networks with squeezing, driven by input-dependent displacements or squeezing parameters — are universal for fading-memory maps [15] while remaining efficiently classically simulable, making them the cleanest illustration that universality $\neq$ advantage. Non-Gaussian resources (Kerr nonlinearities, photon counting) lift the simulability barrier.

Encoding. Displacement amplitude, squeezing phase/strength, or pump modulation per time step; time-multiplexed loops make one physical mode serve as many virtual nodes.

Observables. Homodyne quadratures $\langle \hat{x}_\phi \rangle$, photon numbers, correlation functions; measurement is naturally continuous and weak.

Strengths. Room-temperature operation, GHz-class clock rates, telecom-compatible; deterministic Gaussian dynamics with exact covariance book-keeping; experimental memory control demonstrated in multimode optical platforms [57, 58]; large-scale photonic feature maps demonstrated on a Gaussian boson sampler [59].

Weaknesses. Gaussian-only devices offer no complexity-theoretic edge; non-Gaussian elements are lossy and hard to stabilise; loss is the dominant decoherence and directly bounds memory.

Hardware suitability. Integrated photonics and fibre loops (experimental); microwave CV in circuit QED (Section 18).

Extensions. Coherently coupled parametric oscillators [60]; absorption-engineered memory [61]; hybrid photonic–electronic read-outs.

17 Rydberg-atom analog reservoirs

Model and update. Neutral atoms in optical tweezers under the Rydberg Hamiltonian

$$H = \frac{\Omega}{2} \sum_i X_i - \sum_i \Delta_i n_i + \sum_{i < j} \frac{C_6}{r_{ij}^6} n_i n_j, \\ n_i = \frac{1}{2}(\mathbb{I} - Z_i), \quad (13)$$

evolved analogically between injections; inputs enter through the global detuning waveform $\Delta(t)$, per-site detunings Δ_i , or the atom geometry $\{r_{ij}\}$ — three physically distinct encoding knobs [62, 63].

Encoding. Global-detuning encoding is the simplest and, notably, its memory capacity is largely size-independent (the drive is a collective mode); local detunings and geometry unlock size-extensive feature banks [63].

Observables. Site-resolved occupations $\langle n_i \rangle$ and correlators $\langle n_i n_j \rangle$ from fluorescence imaging — destructive, so operation is restart/ensemble style with whole-array re-preparation.

Strengths. The largest QRC hardware demonstration to date — up to 108 qubits on QuEra’s analog platform with competitive accuracy on classification benchmarks [63]; no per-gate compilation error; hundreds of sites already routine [64]; geometry as a reservoir hyperparameter is unique to the platform.

Weaknesses. Destructive read-out makes each time step a full re-preparation (slow effective clock); limited mid-sequence control; $1/r^6$ tails complicate clean nearest-neighbour theory.

Hardware suitability. Purpose-built analog neutral-atom machines (QuEra, Pasqal class).

Extensions. Few-atom cavity variants with optical I/O [65]; digital-analog hybrids (Section 20).

18 Superconducting-circuit reservoirs

Model and update. Two distinct traditions. *Digital:* transmon processors executing Trotterised or random-circuit reservoirs with mid-circuit measurement and reset — the setting of Chen et al. [37], Hu et al. [41], Yasuda et al. [48], Suzuki et al. [55], all hardware demonstrations on IBM-class devices. *Analog circuit QED:* microwave modes and Josephson nonlinearities driven directly by RF input signals, as in the analog quantum reservoir of Senanian et al. [66], which processed microwave signals natively — input in its physical carrier, no digitisation round-trip — and the

coupled-parametric-oscillator reservoir of Dudas et al. [60].

Encoding. Digital: rotation re-uploading or state replacement. Analog: drive amplitude/phase of resonator modes.

Observables. Dispersive read-out of $\langle Z_i \rangle$ and correlators (digital); heterodyne records of mode quadratures (analog).

Strengths. Fast clocks (μ s-scale steps), mature dynamic-circuit toolchains, best-documented noise models — hence the platform Part IV’s hardware-aware pipeline targets; native RF interface is uniquely suited to sensing and signal-processing front-ends [54, 66].

Weaknesses. Short T_1/T_2 make coherence a budget, not a given — exactly the constraint NISQRC’s reset protocol relaxes; two-qubit-gate and read-out errors dominate; fixed coupling maps force SWAP overhead (quantified in Part IV’s transpilation study).

Hardware suitability. IBM/Rigetti-class digital processors; laboratory circuit-QED analog platforms.

Extensions. Feedback reservoirs via real-time classical control [34]; error-mitigated feature estimation (Part V).

19 Trapped-ion reservoirs

Model and update. Ion-chain processors with all-to-all entangling gates (Mølmer-Sørensen) executing the same digital reservoir templates as Section 18, or analog spin models generated by laser-mediated interactions with tunable range $J_{ij} \propto |i - j|^{-\alpha}$.

Encoding. Rotation re-uploading (digital); drive parameters of the spin-model generator (analog).

Observables. High-fidelity state-resolved fluorescence of every ion; mid-circuit measurement with ion shuttling is production capability on race-track architectures [67].

Strengths. Highest gate and read-out fidelities among gate platforms; effective all-to-all connectivity eliminates SWAP overhead — the disordered fully connected Hamiltonian of Eq. (11) is native; long coherence times relax (though NISQRC makes less essential) the memory budget.

Weaknesses. Clock rates orders of magnitude below superconducting (gate times of microseconds to milliseconds), so time-series throughput and shot budgets are the binding constraint; register sizes currently tens of qubits.

Hardware suitability. Quantinuum/IonQ-class systems; no published QRC-specific hardware benchmark of comparable scale to the platforms

above as of this writing — an open opportunity noted in Part VIII.

Extensions. Long-range-interaction reservoirs interpolating chain and fully connected regimes; QCCD-based mid-circuit protocols.

20 Hybrid quantum–classical reservoirs

Model and update. Deliberate composition of a quantum feature bank with classical components: classical lags or an ESN concatenated with quantum features at the read-out [68, 69]; quantum features feeding a classical recurrent memory [70]; or knowledge-based models corrected by a (quantum) reservoir in the style of Pathak et al. [22]. The digital-analog paradigm [71] is the hardware-level analogue: analog entangling blocks stitched by digital single-qubit control.

Encoding/Observables. Inherited from the quantum member; the defining feature is the composition, not the physics.

Strengths. Ablation-friendly by construction — the classical-only column is always available, so the quantum contribution is measured, not asserted; in this document’s own Qiskit experiment the hybrid read-out recovered most of the classical baseline that noisy quantum features alone missed (Part IV); memory-augmentation demonstrably lifts windowed quantum maps on long-memory tasks [68].

Weaknesses. Easy to misreport: a hybrid beating persistence proves nothing about its quantum part unless the classical-only ablation is shown; engineering complexity of two stacks.

Hardware suitability. Universal — and the recommended default posture for applications (Part VIII).

Extensions. Learned fusion weights; physics-informed classical members for the energy pipelines of Part VII.

21 Comparison

Three cross-cutting reading of Table 1 guide the rest of the document. First, every hardware-demonstrated architecture obtains its ESP from an *explicitly engineered* contraction — reset, re-preparation, loss, or truncation — never from hoping intrinsic decoherence lands at a useful rate; designs that name their contraction are the ones that replicate. Second, the encoding knob is where architectures genuinely differ; the reservoir body

Table 1: Architecture comparison. “Memory mechanism” names the contraction that yields the ESP; “HW status” distinguishes hardware demonstrations (**H**) from simulation-only studies (**S**) as of mid-2026, with a representative citation. Clock and size entries are order-of-magnitude characterisations of current practice, not limits.

Architecture	Memory mechanism	Encoding knobs	Observables	HW status	Best-fit regime
Ising / disordered H (§11)	input-qubit partial trace	state replacement; Δt	$\langle Z_i \rangle$, $\langle Z_i Z_j \rangle$, virtual nodes	H (NMR) [40]	theory reference; small- n studies
Spin chain (§12)	as above; locality	disorder W ; fields	local Paulis	S/H [47]	fixed-topology digital HW
Random circuit, restart (§13)	window truncation	re-upload scale γ ; depth	terminal Paulis	H [37]	first HW experiments; portability
Measurement / feedback (§14)	reset / collapse	host’s; feedback law	per-step records	H [41]	long sequences beyond T_1
Dissipative (§15)	Lindblad gap	host’s; noise rates	host’s	H [55]	noise-dominated devices
CV / photonic (§16)	loss; loop decay	displacement, squeezing	quadratures, $\langle n \rangle$	H [57, 59]	high-rate signals; room temp.
Rydberg analog (§17)	re-preparation per step	$\Delta(t)$, Δ_i , geometry	$\langle n_i \rangle$, $\langle n_i n_j \rangle$	H [63]	largest registers today
Superconducting (§18)	reset (digital); loss (analog)	re-upload; RF drive	dispersive $\langle Z \rangle$; heterodyne	H [48, 66]	fast clocks; dynamic circuits
Trapped ion (§19)	reset via shuttling	re-upload; $J_{ij}(\alpha)$	fluorescence, all qubits	S (QRC-specific)	fidelity-critical studies
Hybrid (§20)	member-dependent	member-dependent	concatenated	H [68, 69]	applications; honest ablations

is interchangeable to a first approximation, which is why Part IV teaches one body and three encodings rather than the reverse. Third, no entry in the table earns a quantum-advantage claim: the honest current statement, defended quantitatively in Parts IV and VIII, is *parameter-parity* — small quantum registers matching much larger classical read-outs — plus platform-specific engineering benefits (native RF processing, sensing proximity), with genuine separations an open research target rather than an established fact [36, 72, 73].

Part IV

Implementation Handbook: Cirq, Qiskit, and PennyLane

22 The code that ships with this document

Every number and figure in this document is produced by the programs in the `code/` directory, summarised in Table 2. The environment is Python 3.12 with NumPy 2.4, Cirq 1.7, Qiskit 2.5 with Qiskit-Aer 0.17, and PennyLane 0.45; each script runs standalone in seconds to minutes on one CPU core. The pedagogical order of this part is deliberate: first an *exact* density-matrix implementation that defines ground truth and hosts the validation suite; then three framework implementations, each chosen to teach the thing that framework does best — Cirq for transparent circuit construction and exact expectation values, Qiskit for everything between an algorithm and a physical device (transpilation, noise, shots, dynamic circuits), and PennyLane for batched evaluation and differentiability at the reservoir/variational boundary.

23 An exact reference implementation

The reference implementation represents ρ explicitly as a $2^n \times 2^n$ complex matrix and applies Eq. (10) literally. Its purpose is threefold: it defines the analytic ceiling against which every shot-noise and hardware-noise result is measured; it is the only implementation in which conceptual errors cannot hide behind framework abstractions; and it hosts the validation suite that every later change must pass. The injection map is eleven lines (Listing 1); note that the input qubit is fixed as the *leftmost* tensor factor, so re-insertion by a Kronecker product lands in the correct slot without any index permutation — the single most common bug in hand-rolled QRC code is silently rebuilding the state with the qubit order scrambled.

```

1 def input_state(u):
2     """rho_in(u) = |psi_u><psi_u|, |psi_u> = sqrt(1-u)|0> + sqrt(u)|1>, u in [0,1]."""
3     a, b = np.sqrt(1.0 - u), np.sqrt(u)
4     psi = np.array([a, b], dtype=complex)
5     return np.outer(psi, psi.conj())
6
7
8 def trace_out_first(rho, n):
9     """Partial trace over qubit 0 of an n-qubit density matrix."""
10    d_rest = 2 ** (n - 1)
11    r = rho.reshape(2, d_rest, 2, d_rest)
12    return np.einsum("ajak->jk", r)
13
14
15 def inject(rho, u, n):
16     """rho -> rho_in(u) (x) Tr_0(rho).
17
18     The input qubit is site 0 (the leftmost tensor factor), so re-insertion
19     lands at the correct slot with no index permutation required.
20     """
21    return np.kron(input_state(u), trace_out_first(rho, n))

```

Listing 1: The Fujii–Nakajima injection, Eq. (10), in `qrc.core.py`. `trace_out_first` implements Tr_1 by reshaping to a rank-4 tensor and contracting the input indices.

W

```

1 class QuantumReservoir:

```

Table 2: Programs shipped with this document. All paths relative to `code/`.

File	Role	Contents
<code>qrc_core.py</code>	reference	exact Fujii–Nakajima reservoir, ESN baseline, ridge, NARMA, memory capacity, shot-noise model, validation suite
<code>datasets.py</code>	data	solar and load surrogates (labelled synthetic), real ENSO series
<code>eda.py</code>	analysis	all Part VI exploratory figures and statistics
<code>experiments.py</code>	results	temporal-edge sweep, memory capacity, prediction demos, shot-noise study (Part VII figures)
<code>cirq_qrc_minimal.py</code>	framework	restart-protocol reservoir in Cirq (Listing 3)
<code>qiskit_qrc_hw.py</code>	framework	hardware-aware pipeline and mid-circuit reset demo (Listing 4)
<code>pennylane_qrc_hybrid.py</code>	framework	broadcast feature map and differentiable encoding tuning (Listing 5)
<code>figstyle.py</code>	support	publication figure style

```

2  """Exact density-matrix simulation of the Fujii--Nakajima protocol."""
3
4  def __init__(self, n_qubits=5, J=1.0, h=1.0, dt=2.0, virtual_nodes=4,
5              use_zz=True, seed=7):
6      self.n = n_qubits
7      self.V = virtual_nodes
8      self.use_zz = use_zz
9      self.H = ising_hamiltonian(n_qubits, J=J, h=h, rng=seed)
10     self.U_sub = propagator(self.H, dt / virtual_nodes)
11     # Observables: local Z at each site (read at each virtual node) and,
12     # optionally, all two-point ZZ correlators (read at the last node).
13     self.Z_ops = [local_op(Z, q, self.n) for q in range(self.n)]
14     self.ZZ_ops = ([two_site_op(Z, a, Z, b, self.n)
15                    for a in range(self.n) for b in range(a + 1, self.n)]
16                   if use_zz else [])
17     self.n_features = self.n * self.V + len(self.ZZ_ops)
18
19     def initial_state(self):
20         dim = 2 ** self.n
21         return np.eye(dim, dtype=complex) / dim # maximally mixed start
22
23     def run(self, inputs):
24         """Drive the reservoir with 'inputs' (values in [0, 1]).
25
26         Returns the feature matrix of shape (T, n_features): exact
27         expectation values <Z_i> at each of the V virtual nodes, plus
28         <Z_i Z_j> at the final node of each interval.
29         """
30         rho = self.initial_state()
31         feats = np.empty((len(inputs), self.n_features))
32         for k, u in enumerate(inputs):
33             rho = inject(rho, float(u), self.n)
34             row = []
35             for _ in range(self.V):
36                 rho = self.U_sub @ rho @ self.U_sub.conj().T
37                 row.extend(np.real(np.trace(op @ rho)) for op in self.Z_ops)
38             row.extend(np.real(np.trace(op @ rho)) for op in self.ZZ_ops)
39             feats[k] = row
40         return feats

```

Listing 2: The reservoir driver in `qrc_core.py`: exact propagation with V virtual nodes and optional two-point correlators.

23.1 The validation suite

`qrc_core.py` ends with `run_validation_suite()`, executed before any experiment in this document was trusted. The suite checks anchors whose expected values are known independently of the code, so that a silent bug cannot masquerade as a physics result: (i) *injection rebuild* — applying `inject` and tracing back out returns the injected state and the retained marginal exactly; (ii) *channel sanity* — the propagator is unitary to machine precision and the evolved ρ stays Hermitian, unit-trace, and positive; (iii) *ridge recovery* — on synthetic data generated by a known linear map with negligible noise, Eq. (5) recovers the map to 10^{-10} ; (iv) *NMSE anchors* — the mean predictor scores 1.000 and the exact-target “predictor” scores 0.000; (v) *shot-noise scaling* — the empirical standard deviation of the sampled features follows Eq. (12), halving when S quadruples. Anchors (i)–(v) all

pass in the shipped code; the discipline of *building the anchors before the experiment* is restated as a design principle in Part VIII because, in the authors’ experience, it is the single highest-yield habit in quantum machine-learning practice.

24 Cirq: transparent circuits and exact expectations

Cirq (Google) exposes circuits as plain Python data structures — moments of gates on line or grid qubits — with first-class parameter resolution through `sympy` symbols. That transparency makes it the right teaching vehicle for the *restart-protocol* reservoir of Section 13: one template circuit is built once with symbolic input angles, and each time step resolves the symbols with the current input window. Listing 3 is the complete program.

```
1  """cirq_qrc_minimal.py -- A minimal quantum reservoir computer in Cirq.
2
3  Protocol: the *restart (rewind) protocol*. For every time step k we build a
4  fresh circuit that re-uploads the most recent L inputs u_{k-L+1}, ..., u_k,
5  interleaved with a fixed random entangling block W. Because the reservoir
6  has fading memory, a finite window L approximates the full input history,
7  and no mid-circuit measurement is needed -- every feature is read from a
8  terminal measurement, which suits any gate-model backend.
9
10 Two design choices matter and are worth reading closely: the encoding
11 scale (pi/2 here; a pi-scale drives the features into a high-frequency,
12 poorly-generalising regime) and the observable set (local <Z_i> alone is
13 nearly rank-deficient; adding <X_i> and <Z_i Z_j> triples the usable
14 feature directions).
15
16 Task: one-step-ahead prediction of the NARMA-2 benchmark sequence.
17 Run:  python3 cirq_qrc_minimal.py          (finishes in a few seconds)
18 """
19 import cirq
20 import numpy as np
21 import sympy
22
23 # ----- config
24 N_QUBITS = 5          # reservoir size
25 WINDOW = 6           # input window L re-uploaded per circuit (fading memory)
26 T = 400              # length of the driving sequence
27 SHOTS = 0            # 0 -> exact expectation values; >0 -> sampled estimates
28 SCALE = np.pi / 2    # encoding scale (see module docstring)
29 NOISE_P = 0.0         # per-moment depolarising probability (try 0.002)
30 SEED = 7
31
32 rng = np.random.default_rng(SEED)
33 qubits = cirq.LineQubit.range(N_QUBITS)
34
35 # ----- fixed entangling block W
36 # One layer = random single-qubit Y-rotations (drawn once, then frozen)
37 # followed by a brickwork of CZ gates. W is the "reservoir dynamics": it is
38 # never trained, exactly as W_res is frozen in a classical echo-state network.
39 fixed_angles = rng.uniform(0, 2 * np.pi, size=(2, N_QUBITS))
```

```

40
41
42 def entangling_block() -> cirq.Circuit:
43     ops = []
44     for layer in range(2):
45         ops.append(cirq.Moment(
46             cirq.ry(fixed_angles[layer, i]).on(q) for i, q in enumerate(qubits)))
47         pairs = [(i, i + 1) for i in range(layer % 2, N_QUBITS - 1, 2)]
48         ops.append(cirq.Moment(cirq.CZ(qubits[a], qubits[b]) for a, b in pairs))
49     return cirq.Circuit(ops)
50
51
52 W_BLOCK = entangling_block()
53
54 # ----- parameterised input layer
55 # The window of inputs enters through symbolic RY angles; sympy symbols keep
56 # one template circuit whose parameters are bound per time step via a
57 # ParamResolver -- much faster than rebuilding gates from floats.
58 theta = [sympy.Symbol(f"th{j}") for j in range(WINDOW)]
59
60
61 def template() -> cirq.Circuit:
62     c = cirq.Circuit()
63     for j in range(WINDOW):
64         # Each past input is written on every qubit (global re-uploading),
65         # scaled per qubit by a fixed random gain to break symmetry.
66         gains = input_gains[j]
67         c.append(cirq.Moment(
68             cirq.ry(gains[i] * theta[j]).on(q) for i, q in enumerate(qubits)))
69         c += W_BLOCK
70     if SHOTS:
71         c.append(cirq.measure(*qubits, key="z"))
72     return c
73
74
75 input_gains = rng.uniform(0.4, 1.0, size=(WINDOW, N_QUBITS))
76 TEMPLATE = template()
77
78 # ----- backend
79 if NOISE_P > 0:
80     sim = cirq.DensityMatrixSimulator(
81         noise=cirq.ConstantQubitNoiseModel(cirq.depolarize(NOISE_P)))
82 else:
83     sim = cirq.Simulator(seed=SEED)
84 observables = ([cirq.Z(q) for q in qubits] + [cirq.X(q) for q in qubits]
85               + [cirq.Z(qubits[a]) * cirq.Z(qubits[b])
86                  for a in range(N_QUBITS) for b in range(a + 1, N_QUBITS)])
87
88
89 def features_for_window(u_win: np.ndarray) -> np.ndarray:
90     """Map the last L inputs (in [0,1]) to the 20 features <Z_i>, <X_i>, <Z_iZ_j>."""
91     resolver = cirq.ParamResolver({theta[j]: SCALE * u_win[j]
92                                     for j in range(WINDOW)})
93     if SHOTS:
94         result = sim.run(TEMPLATE, resolver, repetitions=SHOTS)
95         bits = 1.0 - 2.0 * result.measurements["z"] # 0 -> +1, 1 -> -1
96         z = bits.mean(axis=0)
97         zz = [np.mean(bits[:, a] * bits[:, b])
98               for a in range(N_QUBITS) for b in range(a + 1, N_QUBITS)]
99         return np.concatenate([z, np.zeros(N_QUBITS), zz]) # (X needs basis change)
100     vals = sim.simulate_expectation_values(TEMPLATE, observables, resolver)
101     return np.real(np.asarray(vals))
102
103
104 # ----- benchmark
105 def narma2(T, seed=0):
106     g = np.random.default_rng(seed)

```

```

107     u = g.uniform(0, 0.5, size=T)
108     y = np.zeros(T)
109     for k in range(1, T - 1):
110         y[k + 1] = 0.4 * y[k] + 0.4 * y[k] * y[k - 1] + 0.6 * u[k] ** 3 + 0.1
111     return u, y
112
113
114 u, y = narma2(T, seed=1)
115 u_scaled = u / 0.5 # -> [0, 1]
116
117 X = np.zeros((T, len(observables)))
118 for k in range(T):
119     win = np.zeros(WINDOW)
120     lo = max(0, k - WINDOW + 1)
121     win[WINDOW - (k - lo + 1):] = u_scaled[lo:k + 1]
122     X[k] = features_for_window(win)
123
124 # Ridge read-out (the only trained component)
125 washout, split = 20, int(0.7 * T)
126 Xd = np.hstack([X, np.ones((T, 1))])
127 A = Xd[washout:split]
128 w = np.linalg.solve(A.T @ A + 1e-6 * np.eye(A.shape[1]), A.T @ y[washout:split])
129 pred = Xd[split:] @ w
130 nmse = np.mean((y[split:] - pred) ** 2) / np.var(y[split:])
131 print(f"Cirq restart-protocol QRC | qubits={N_QUBITS} window={WINDOW} "
132       f"shots={'exact' if SHOTS == 0 else SHOTS} | test NMSE = {nmse:.4f}")

```

Listing 3: `cirq_qrc_minimal.py`: a complete restart-protocol quantum reservoir in Cirq. Test NMSE on NARMA-2: 0.277 (exact expectations, 5 qubits, window 6).

Walk-through. The configuration block fixes 5 line qubits, a window of $L = 6$ re-uploaded inputs, and — decisively, as shown below — an encoding scale of $\pi/2$. The template builds, per window slot j : one moment of $R_Y(g_i \theta_j)$ input rotations with fixed random per-qubit gains g_i (so different qubits see different projections of the same input), then a frozen two-layer entangling block of random R_Y rotations and brickwork CZ s — the reservoir body, drawn once at seed time and never trained. `features_for_window` resolves the six symbols against the current window and calls `simulate_expectation_values`, which returns *exact* statevector expectations of the requested observables — Cirq’s cleanest feature for reservoir research, since it separates architecture quality from sampling noise by construction. The observable set is $\langle Z_i \rangle$, $\langle X_i \rangle$, and all $\langle Z_i Z_j \rangle$: twenty features. The optional `SHOTS > 0` branch shows the sampled path (`cirq.measure` with key “z”, then $\langle Z \rangle = 1 - 2P(1)$ and pairwise products from the bit table); the X expectations would need a basis change before measurement, which the exact path sidesteps. Ridge and NMSE close the file; the washout here discards early steps whose windows are zero-padded.

The two design lessons the file encodes. Both were found the honest way — by the sweep in Table 3 — and generalise well beyond Cirq. First, *ob-*

servable richness: with local $\langle Z_i \rangle$ alone the twenty-dimensional dynamics collapse onto five nearly collinear features and the model barely beats the mean predictor; adding $\langle X_i \rangle$ and $\langle Z_i Z_j \rangle$ triples the usable directions and cuts the error by a factor of three at the good scale. Second, *encoding scale:* at $\gamma = \pi$ the re-uploaded rotations drive the features into high harmonics of the input (Section 9.2) that generalise poorly; halving to $\pi/2$ is worth more than any other single change. Neither lesson is visible without the sweep — which is the third lesson.

Table 3: Encoding-scale and observable sweep for Listing 3 (NARMA-2 test NMSE; exact expectations; simulation). The shipped configuration is bold.

Window L	Scale γ	Observables (count)	NMSE
6	π	Z (5)	0.948
6	π	Z, X, ZZ (20)	0.690
6	$\pi/2$	Z (5)	0.848
6	$\pi/2$	Z, X, ZZ (20)	0.277
8	π	Z, X, ZZ (20)	0.830
8	$\pi/2$	Z, X, ZZ (20)	0.402

25 Qiskit: everything between algorithm and device

Qiskit (IBM) is the most hardware-aware of the three frameworks, and this section uses it for exactly that layer: what changes when the reservoir of Section 24 must run on a machine with

a fixed qubit topology, a native gate alphabet, decoherence, read-out error, and a finite shot budget. Listing 4 is the complete program; it is written so that replacing `AerSimulator` by a `QiskitRuntimeService` backend — and nothing else — submits the same experiment to an IBM device.

```

1  """qiskit_qrc_hw.py -- A hardware-aware quantum reservoir computer in Qiskit.
2
3  Two parts:
4
5      PART A   Windowed (restart-protocol) reservoir executed the way an IBM job
6                would run: transpiled to a linear coupling map and a native basis
7                (ecr, id, rz, sx, x), under a depolarising + readout noise model,
8                with features estimated from a finite number of shots.
9                Task: one-step-ahead prediction of the solar clear-sky index
10               (synthetic surrogate from datasets.py).
11
12      PART B   A mid-circuit measurement + reset demonstration: the dynamic-
13                circuit primitive that realises the Fujii--Nakajima partial-trace
14                injection on hardware and gives memory unbounded by T1/T2.
15
16  Swap 'AerSimulator' for a 'QiskitRuntimeService' backend (and keep the same
17  'transpile' call against 'backend.target') to run PART A on an IBM device.
18  Run:  python3 qiskit_qrc_hw.py                      (about a minute on one core)
19  """
20  import warnings
21  warnings.filterwarnings("ignore")
22
23  import numpy as np
24  from qiskit import QuantumCircuit, ClassicalRegister, QuantumRegister, transpile
25  from qiskit.transpiler import CouplingMap
26  from qiskit_aer import AerSimulator
27  from qiskit_aer.noise import NoiseModel, depolarizing_error, ReadoutError
28
29  from datasets import solar_surrogate
30
31  # ----- config
32  N, WINDOW, T, SHOTS, SEED = 5, 8, 400, 2048, 7
33  rng = np.random.default_rng(SEED)
34
35  BASIS = ["ecr", "id", "rz", "sx", "x"] # IBM Heron native set
36  COUPLING = CouplingMap([(i, i + 1) for i in range(N - 1)]) # linear chain
37
38
39  def noise_model() -> NoiseModel:
40      nm = NoiseModel(basis_gates=BASIS)
41      nm.add_all_qubit_quantum_error(depolarizing_error(4e-4, 1), ["sx", "x"])
42      nm.add_all_qubit_quantum_error(depolarizing_error(6e-3, 2), ["ecr"])
43      ro = ReadoutError([[0.99, 0.01], [0.02, 0.98]])
44      for q in range(N):
45          nm.add_readout_error(ro, [q])
46      return nm
47
48
49  BACKEND = AerSimulator(noise_model=noise_model(), seed_simulator=SEED)
50
51  # ----- PART A: windowed reservoir
52  fixed_angles = rng.uniform(0, 2 * np.pi, size=(2, N))
53  input_gains = rng.uniform(0.4, 1.0, size=(WINDOW, N))
54
55

```

```

56 def reservoir_circuit(u_win: np.ndarray) -> QuantumCircuit:
57     """Fresh circuit re-uploading the last WINDOW inputs (restart protocol)."""
58     qc = QuantumCircuit(N, N)
59     for j in range(WINDOW):
60         for q in range(N):
61             qc.ry(float(np.pi * input_gains[j, q] * u_win[j]), q)
62         for layer in range(2):
63             for q in range(N):
64                 qc.ry(float(fixed_angles[layer, q]), q)
65             for a in range(layer % 2, N - 1, 2):
66                 qc.cz(a, a + 1)
67     qc.measure(range(N), range(N))
68     return qc
69
70
71 def counts_to_features(counts: dict) -> np.ndarray:
72     """<Z_i> and <Z_i Z_j> from a counts dictionary (little-endian keys)."""
73     shots = sum(counts.values())
74     z = np.zeros(N)
75     zz = np.zeros((N, N))
76     for key, c in counts.items():
77         bits = np.array([1 - 2 * int(b) for b in key[::-1]]) # 0->+1, 1->-1
78         z += c * bits
79         zz += c * np.outer(bits, bits)
80     z /= shots
81     zz /= shots
82     pairs = [zz[a, b] for a in range(N) for b in range(a + 1, N)]
83     return np.concatenate([z, pairs]) # 5 + 10 = 15 features
84
85
86 # Data: daytime clear-sky index, one-step-ahead target
87 sol = solar_surrogate()
88 day = sol[(sol.elevation_deg > 5) & sol.ghi.notna()]
89 kt = np.clip(day.kt.to_numpy()[:T + 1], 0, 1.05)
90 u_series, target = kt[:-1] / 1.05, kt[1:]
91
92 # Build, transpile, and run ALL step circuits as one batched job -- exactly
93 # how a hardware submission is structured (one job, T circuits, S shots).
94 circuits = []
95 for k in range(T):
96     win = np.zeros(WINDOW)
97     lo = max(0, k - WINDOW + 1)
98     win[WINDOW - (k - lo + 1):] = u_series[lo:k + 1]
99     circuits.append(reservoir_circuit(win))
100 tcircs = transpile(circuits, backend=BACKEND, coupling_map=COUPLING,
101                    basis_gates=BASIS, optimization_level=1, seed_transpiler=SEED)
102 print(f"transpiled depth (first circuit): {tcircs[0].depth()}, "
103      f"2q gates: {tcircs[0].count_ops().get('ecr', 0)}")
104 job = BACKEND.run(tcircs, shots=SHOTS)
105 Xq = np.array([counts_to_features(c) for c in job.result().get_counts()])
106
107 # Hybrid augmentation: append cheap classical lags to the quantum features.
108 # In principle the read-out can fall back on the lags alone; in practice,
109 # with ~250 training points, fifteen shot-noisy quantum columns inject
110 # enough variance that even a tuned ridge cannot fully ignore them, and the
111 # hybrid lands slightly BELOW the classical-only baseline. Reproducing this
112 # effect -- and being suspicious of any benchmark that hides it -- is one of
113 # the more valuable exercises in this file.
114 lags = np.stack([np.roll(u_series, i) for i in range(3)], axis=1)
115 X = np.hstack([Xq, lags])
116
117 # Ridge read-out with the penalty chosen by generalised cross-validation
118 # (GCV) on the training span only -- deterministic and split-free, which
119 # matters here because a short chronological validation tail can land in an
120 # unrepresentative cloud regime and mis-select the penalty.
121 washout, split = 20, int(0.7 * T)
122

```

```

123
124 def fit_eval(Xf):
125     Xd = np.hstack([Xf, np.ones((T, 1))])
126     A, ytr = Xd[washout:split], target[washout:split]
127     U, s, Vt = np.linalg.svd(A, full_matrices=False)
128     Uty, n = U.T @ ytr, A.shape[0]
129     best = (np.inf, None)
130     for lam in 10.0 ** np.arange(-6, 2):
131         f = s ** 2 / (s ** 2 + lam) # shrinkage factors
132         resid = ytr - U @ (f * Uty)
133         gcv = n * (resid @ resid) / (n - f.sum()) ** 2
134         if gcv < best[0]:
135             best = (gcv, lam)
136     lam = best[1]
137     w = Vt.T @ ((s / (s ** 2 + lam)) * Uty)
138     p = Xd[split:] @ w
139     return np.mean((target[split:] - p) ** 2) / np.var(target[split:])
140
141
142 nmse = fit_eval(X)
143 nmse_qonly = fit_eval(Xq)
144 nmse_conly = fit_eval(lags)
145 pers = np.mean((target[split:] - u_series[split:] * 1.05) ** 2) /
146     np.var(target[split:])
147 print(f"PART A | noisy, {SHOTS} shots, native basis")
148 print(f" quantum + classical lags : NMSE = {nmse:.4f}")
149 print(f" quantum features only : NMSE = {nmse_qonly:.4f}")
150 print(f" classical lags only : NMSE = {nmse_conly:.4f}")
151 print(f" persistence : NMSE = {pers:.4f}")
152
153 # ----- PART B: mid-circuit measurement + reset (leaky)
154 # One LONG circuit processes STEPS inputs sequentially. Each step: encode
155 # u_k on the I/O qubit, entangle, measure the I/O qubit into its own
156 # classical register, then RESET it. The unmeasured qubits carry memory
157 # forward; the reset realises the partial-trace injection on hardware and
158 # decouples usable memory from T1/T2 (the NISQRC mechanism).
159 STEPS = 12
160 qr, io = QuantumRegister(N, "q"), 0
161 regs = [ClassicalRegister(1, f"s{k}") for k in range(STEPS)]
162 qc = QuantumCircuit(qr, *regs)
163 u_demo = u_series[:STEPS]
164 for k in range(STEPS):
165     if k:
166         qc.reset(io)
167         qc.ry(float(np.pi * u_demo[k]), io)
168         for a in range(N - 1):
169             qc.cz(a, a + 1)
170         for q in range(N):
171             qc.ry(float(fixed_angles[0, q]), q)
172         qc.measure(io, regs[k])
173 tqc = transpile(qc, backend=BACKEND, coupling_map=COUPLING,
174                 basis_gates=BASIS, optimization_level=1, seed_transpiler=SEED)
175 res = BACKEND.run(tqc, shots=4096).result().get_counts()
176 traj = np.zeros(STEPS)
177 for key, c in res.items(): # key = "sK ... s1 s0" space-joined
178     bits = key.split()[::-1] # -> chronological order
179     for k in range(STEPS):
180         traj[k] += c * (1 - 2 * int(bits[k]))
181 traj /= 4096
182 print("PART B | per-step <Z_io> trajectory from mid-circuit measurement:")
183 print(" ", np.array2string(traj, precision=3))

```

Listing 4: qiskit_qrc_hw.py: a hardware-aware windowed reservoir (Part A) and a mid-circuit measurement-and-reset demonstration (Part B). Simulation with a realistic noise model.

25.1 Transpilation: coupling maps and basis gates

A circuit as written uses idealised gates between arbitrary qubit pairs; a device executes a fixed native alphabet between physically coupled pairs. `transpile` bridges the two: it *routes* (inserting SWAPs where the logical circuit touches uncoupled qubits), *translates* into the basis alphabet — here IBM’s current `ecr`, `rz`, `sx`, `x`, `id`, where `rz` is a software frame change and effectively free — and *optimises* at the requested level. The listing pins a linear five-qubit `CouplingMap` to make routing costs visible and `seed_transpiler` for reproducibility (routing is stochastic). Measured outcome for one window circuit: depth 78 with 32 two-qubit `ecr` gates — the number that, multiplied by the two-qubit error rate, dominates the noise budget below. Against a real backend one passes `backend.target` instead of the manual map and lets the reported calibration data drive layout selection; the manual construction here exists so the reader can see exactly which constraints bind. The practical rule the numbers teach: on linear topologies, reservoir circuits should be written brickwork-local from the start (as this one is), because every nonlocal gate becomes three.

25.2 Noise models and finite shots

The `NoiseModel` attaches depolarising errors of 4×10^{-4} per single-qubit gate and 6×10^{-3} per `ecr` — mid-table values for current superconducting hardware — plus an asymmetric read-out confusion matrix (1%/2%). With 32 two-qubit gates per circuit the accumulated depolarisation is already tens of percent of a fully mixed admixture, which is why the quantum-only column below sits where it does. All $T = 400$ step circuits are transpiled and submitted as *one batched job* of `SHOTS= 2048` each — the shape of a real hardware submission, and the efficient one, since per-job overhead dwarfs per-circuit cost on queued devices. `counts_to_features` converts each counts dictionary to $\langle Z_i \rangle$ and $\langle Z_i Z_j \rangle$; the one genuine trap is bit order — *Qiskit keys are little-endian* (qubit 0 is the rightmost character), handled here by reversing the key once, in exactly one place.

25.3 Read-out, honestly regularised, and what the numbers say

The read-out augments the fifteen quantum features with three classical lags of the input (the hybrid posture of Section 20) and selects the

ridge penalty by generalised cross-validation on the training span — deterministic and split-free, chosen after a short chronological validation tail proved unrepresentative on this cloud-regime-switching series. Table 4 gives the test results on one-step prediction of the (surrogate) solar clear-sky index.

Table 4: Qiskit pipeline test NMSE (solar clear-sky index, one step ahead; noisy simulation, 2048 shots). “Quantum only” is the fifteen measured features; “classical only” is three lags of the input.

Feature set	NMSE
Quantum + classical lags (hybrid)	0.192
Quantum features only	0.606
Classical lags only	0.134
Persistence	0.138

Read the table the way Part VIII will insist all such tables be read. The task is nearly persistent, so persistence and three lags are strong; the noisy quantum features alone are far worse; and the hybrid *recovers most but not all* of the classical baseline — with ~ 250 training points, fifteen shot-noisy columns inject enough variance that even a tuned ridge cannot fully ignore them. There is no quantum benefit here, and the pipeline says so. The value of the exercise is that every mechanism a hardware paper must control — routing overhead, noise accumulation, shot variance, regularisation under feature noise, and the ease of accidentally reporting only the flattering column — is present, measured, and small enough to fit in one file.

25.4 Mid-circuit measurement and reset

Part B of the listing demonstrates the dynamic-circuit primitive that realises Eq. (10) on hardware

and underlies the NISQRC result [41]: one *long* circuit processes twelve inputs sequentially; each step encodes u_k on the I/O qubit, entangles, measures the I/O qubit into *its own single-bit classical register*, and resets it. Per-step registers matter — a single register would overwrite the history — and the space-joined, reversed key parsing at the end reconstructs the chronological record. The printed per-step $\langle Z_{io} \rangle$ trajectory responds to the input sequence with visible lag: the persistent qubits are carrying information forward across measurements, on a circuit whose total duration would exceed any coherence budget if the memory lived in undisturbed phases. On real IBM backends the same construction runs under dynamic-circuit support with `if_else` available for the feedback architectures of Section 14.

26 PennyLane: batching and the differentiable boundary

PennyLane (Xanadu) organises computation around the *QNode* — a quantum function bound to a device that behaves as a differentiable array operation under autograd, JAX, PyTorch, or TensorFlow interfaces. Two capabilities pay rent in reservoir work: *parameter broadcasting*, which evaluates a whole time series of input windows in one batched call, and *differentiability*, which is deliberately used here for one scalar — the encoding gain — while the reservoir body stays frozen. Listing 5 is the complete program; the task is 3-month-ahead prediction of the *real* ENSO sea-surface-temperature anomaly.

```

1  """pennylane_qrc_hybrid.py -- A QRC-inspired hybrid model in PennyLane.
2
3  Design: a windowed quantum feature map in the reservoir spirit (the
4  entangling body is frozen; only a classical ridge read-out is trained),
5  evaluated for the whole time series in ONE batched QNode call via
6  PennyLane's parameter broadcasting. A second stage then shows the one
7  thing differentiability buys a reservoir practitioner cheaply: tuning a
8  single global encoding gain gamma by alternating (i) a classical ridge
9  refit and (ii) a gradient step on gamma with the read-out held fixed.
10 This avoids differentiating through the linear solve and is the standard
11 practical recipe when a model sits on the reservoir/variational boundary.
12
13 Task: 3-month-ahead prediction of the REAL monthly ENSO sea-surface-
14 temperature anomaly (statsmodels 'elnino' dataset, 1950-2010).
15 Run:  python3 pennylane_qrc_hybrid.py          (about a minute)
16 """
17 import numpy as np

```

```

18 import pennylane as qml
19 from pennylane import numpy as pnp
20
21 from datasets import enso_real
22
23 # ----- config
24 N, WINDOW, SEED, HORIZON = 5, 8, 7, 3
25 rng = np.random.default_rng(SEED)
26 dev = qml.device("default.qubit", wires=N) # exact, broadcast-capable
27
28 fixed_weights = rng.uniform(0, 2 * np.pi, size=(2, N, 3)) # frozen reservoir
29 input_gains = rng.uniform(0.4, 1.0, size=(WINDOW, N))
30
31
32 @qml.qnode(dev, interface="autograd")
33 def reservoir(u_win, gamma):
34     """u_win: (batch, WINDOW) windows; gamma: global encoding gain.
35
36     Returns  $2N + N(N-1)/2 = 20$  features per window:  $\langle Z_i \rangle$ ,  $\langle X_i \rangle$ ,  $\langle Z_i Z_j \rangle$ .
37     Broadcasting handles the batch axis; one call evaluates every window.
38     """
39     for j in range(WINDOW):
40         for q in range(N):
41             qml.RY(gamma * np.pi * input_gains[j, q] * u_win[... , j], wires=q)
42             qml.StronglyEntanglingLayers(fixed_weights, wires=range(N))
43     return ([qml.expval(qml.PauliZ(q)) for q in range(N)]
44             + [qml.expval(qml.PauliX(q)) for q in range(N)]
45             + [qml.expval(qml.PauliZ(a) @ qml.PauliZ(b))
46                 for a in range(N) for b in range(a + 1, N)])
47
48
49 def windows(u):
50     Xw = np.zeros((len(u), WINDOW))
51     for k in range(len(u)):
52         lo = max(0, k - WINDOW + 1)
53         Xw[k, WINDOW - (k - lo + 1):] = u[lo:k + 1]
54     return Xw
55
56
57 def ridge(F, y, lam=1e-2):
58     Fd = np.hstack([F, np.ones((len(F), 1))])
59     return np.linalg.solve(Fd.T @ Fd + lam * np.eye(Fd.shape[1]), Fd.T @ y)
60
61
62 def nmse(y, p):
63     return float(np.mean((y - p) ** 2) / np.var(y))
64
65
66 # ----- pipeline
67 en = enso_real()
68 an = en.anomaly_c.to_numpy()
69 u = (an - an.min()) / (an.max() - an.min()) # -> [0, 1]
70 y = np.roll(an, -HORIZON)
71 u, y = u[:-HORIZON], y[:-HORIZON]
72 Xw = windows(u)
73 washout, split = 24, int(0.7 * len(y))
74
75
76 def evaluate(gamma):
77     F = np.array(reservoir(Xw, float(gamma))).T # (T, 20), one call
78     w = ridge(F[washout:split], y[washout:split])
79     Fd = np.hstack([F, np.ones((len(F), 1))])
80     return nmse(y[split:], Fd[split:] @ w)
81
82
83 print(f"fixed reservoir, gamma = 1.00 | ENSO {HORIZON}-mo NMSE = "
84       f"{evaluate(1.0):.4f}")

```

```

85 print(f"persistence baseline | ENSO {HORIZON}-mo NMSE = "
86       f"{nmse(y[split:], an[:-HORIZON][split:]).4f}")
87
88 # ----- alternating tuning of the encoding gain (14 iterations)
89 # Step (i): refit the ridge read-out classically at the current gamma.
90 # Step (ii): one gradient step on gamma with that read-out FROZEN, so the
91 # gradient flows only through the quantum circuit (parameter-shift capable).
92 gamma = pnp.array(1.0, requires_grad=True)
93 opt = qml.GradientDescentOptimizer(stepsize=0.15)
94 tr = slice(washout, split)
95 for it in range(14):
96     F_tr = np.array(reservoir(Xw[tr], float(gamma))).T
97     w_fix = pnp.array(ridge(F_tr, y[tr]), requires_grad=False)
98
99     def loss(g):
100         F = qml.math.stack(reservoir(Xw[tr], g)).T
101         Fd = qml.math.concatenate([F, pnp.ones((F.shape[0], 1))], axis=1)
102         return pnp.mean((Fd @ w_fix - y[tr]) ** 2)
103
104     gamma, c = opt.step_and_cost(loss, gamma)
105     print(f" iter {it}: train MSE = {float(c):.4f} -> "
106           f"gamma = {float(gamma):.3f}")
107
108 print(f"tuned encoding, gamma = {float(gamma):.2f} | ENSO {HORIZON}-mo "
109       f"NMSE = {evaluate(gamma):.4f}")

```

Listing 5: `pennylane_qrc_hybrid.py`: a broadcast windowed quantum feature map with a trained ridge read-out, plus alternating gradient tuning of a single encoding gain. Real ENSO data.

Walk-through. The `QNode` re-uploads a window of 8 inputs through gain-weighted R_Y rotations interleaved with a frozen `StronglyEntanglingLayers` block, and returns twenty expectations (Z , X , ZZ). The ellipsis indexing `u_win[..., j]` is the broadcasting idiom: called with a $(T, 8)$ array, the device vectorises over the leading axis and one call returns all T feature rows. The tuning stage then alternates two steps: refit the ridge read-out classically at the current gain γ ; take one gradient step on γ with that read-out *frozen*, so the gradient flows only through the quantum circuit and no differentiation through the linear solve is attempted — the standard, stable recipe at the reservoir/variational boundary, and vastly cheaper than end-to-end training since the quantum parameter count is one.

What the numbers teach. Fixed $\gamma = 1.0$ scores NMSE 0.976 — barely better than the mean; fourteen alternating iterations walk γ to 0.69 and the error to 0.747; persistence scores 0.695; and the *recurrent* exact reservoir of Section 23 on the same task scores 0.596 (Part VII). The ranking is the lesson. A windowed feature map, however tuned, hard-truncates memory at $L = 8$ months, and ENSO carries predictive structure well beyond that; genuine recurrence — state carried forward indefinitely with fading weights — is not a nicety but the difference between beating persistence and

not. Differentiability recovered what the encoding scale had thrown away (a threefold error reduction, for one parameter and fourteen circuit-pair evaluations) but could not manufacture memory the architecture does not have. When to cross fully into variational training — trainable body, parameter-shift gradients through hundreds of parameters — is a cost question: each shift-rule gradient costs two full batched evaluations *per parameter*, which is precisely the expense the reservoir paradigm exists to avoid [74, 75].

27 Shot noise in practice

The exact simulations above have an analytic ceiling; hardware estimates approach it as $S^{-1/2}$. The shipped shot-noise model perturbs each exact Pauli expectation z by binomially exact noise, $\hat{z} = \text{clip}(z + \xi\sqrt{(1-z^2)/S}, -1, 1)$ with $\xi \sim \mathcal{N}(0, 1)$, and the validation suite confirms the empirical scaling of Eq. (12). Applied to the NARMA-10 reference task (Part VII, Figure 9): the exact-feature ceiling is NMSE 0.283; at $S = 10^2$ the score is 0.90, at $S = 10^3$ it is 0.64, at $S = 10^4$ it is 0.53, and even $S = 10^5$ leaves 0.45 — still 60% above ceiling. Three practices follow, used throughout this document and restated in Part VIII: measure the ceiling first, so architecture and sampling effects are never conflated; report S next to every number,

since shots are the real hardware currency; and let the regulariser know — feature noise of known variance shifts the optimal λ upward, which is why every read-out in this document selects λ on data rather than by convention. The fundamental version of these observations — sampling noise caps the number of resolvable feature directions (*eigen-tasks*), with an information-optimal read-out construction — is developed in Hu et al. [42], and shot-optimal training specifically in Ahmed et al. [43].

28 Choosing a framework

Table 5 condenses the working differences; the recommendation logic beneath it is short. For *architecture research* — new encodings, observables, dynamical regimes — use the exact NumPy path (or Cirq’s exact expectations) so sampling never confounds design, and port later. For *hardware-bound work* on superconducting devices, build in Qiskit from day one: transpilation realism, calibration-driven noise models, and dynamic circuits are where that project’s risk lives. For *hybrid models* that may drift toward trainable components, or that live inside a JAX/PyTorch stack, PennyLane’s interfaces and broadcasting make it the lowest-friction host. The three are not exclusive: this document’s own pipeline prototyped exactly, validated in Cirq, and hardened in Qiskit.

Table 5: Working comparison of the three primary frameworks for QRC (versions as used here: Cirq 1.7, Qiskit 2.5/Aer 0.17, PennyLane 0.45).

Axis	Cirq	Qiskit	PennyLane
Circuit model	moments of gates; explicit, list-like	<code>QuantumCircuit</code> ; register-oriented	quantum functions (<code>QNodes</code>)
Exact expectations	<code>simulate_expectation_values</code> (statevector)	<code>Estimator</code> primitives; Aer	analytic on <code>default.qubit</code>
Density-matrix / noise	<code>DensityMatrixSimulator</code> per-moment noise	Aer <code>NoiseModel</code> : gate errors, read-out, thermal; calibration import	<code>default.mixed</code> ; channel ops
Transpilation to real topologies	basic device gatesets	best in class : routing, basis translation, optimisation levels, targets	delegates to plugins
Mid-circuit measure/reset	supported in simulators	production dynamic circuits ; per-step classical registers (Listing 4)	<code>qml.measure(reset=True)</code> ; allocates an extra wire on <code>default.mixed</code> — prefer compiled-unitary or NumPy paths for density-matrix studies
Batching a time series	loop over resolvers (fast)	batched job of many circuits (hardware-shaped)	parameter broadcasting : one call
Differentiability	external	external	native : autograd/JAX/Torch/TF, parameter shift
Symbolic parameters	<code>sympy</code> , first-class	<code>Parameter</code> objects	function arguments
Hardware access	Google-adjacent; via converters	IBM Quantum; broad vendor reach	vendor plugins (IBM, IonQ, Braket, ...)
Best QRC role here	transparent restart-protocol teaching; exact studies	the hardware-facing pipeline	hybrid/differentiable boundary; fast batched sweeps

Table 6: Secondary tools, one line each, for completeness.

Tool	One-line role for QRC work
QuTiP [76]	the open-systems workhorse: Lindblad solvers for dissipative reservoirs (Section 15)
Qulacs [77]	fastest CPU statevector loops for large sweep campaigns
TensorCircuit [78]	tensor-network backends and JAX-native batching for larger n at low entanglement
CUDA-Q / cuQuantum	GPU statevector and density-matrix kernels when sweeps outgrow CPUs
Strawberry Fields [79]	the CV/photonic reservoirs of Section 16, Gaussian and Fock backends
Amazon Braket	multi-vendor hardware access (IonQ, Rigetti, QuEra) behind one API
Quantinuum toolchain	H-series access; mid-circuit measurement and QCCD primitives
IonQ / Rigetti clouds	direct vendor access for the platforms of Part V
Mitiq [80]	framework-agnostic error mitigation (ZNE, PEC) for feature estimation

Part V

Hardware Considerations

This part reads the platform landscape through one question: *which hardware properties bind for reservoir computing specifically?* Reservoirs do not need deep coherent circuits, error-corrected logic, or trained on-device parameters; they need fast repeated state preparation and measurement, good mid-circuit measurement and reset where recurrence is wanted, enough shots to beat Eq. (12), and noise of a *kind* that contracts without erasing (Section 8). Those priorities rank platforms differently from, say, fault-tolerance roadmaps, and the ranking is task-dependent. Quantitative gate-fidelity and coherence figures below are order-of-magnitude characterisations of current practice; exact values change with every calibration cycle and should be read from the provider’s live calibration data at run time, as the Part IV pipeline does when pointed at a real backend.

29 IBM superconducting processors (primary target)

IBM’s fixed-frequency transmon processors are this document’s primary hardware target because they combine the largest publicly accessible register counts, the most mature dynamic-circuit support, and the toolchain (Part IV) whose abstractions map one-to-one onto the physics. The QRC-relevant profile:

Gate set and topology. Native alphabet `ecr/rz/sx/x` on a heavy-hex lattice: each qubit couples to two or three neighbours. For reservoirs this makes brickwork-local circuits (Listings 3 and 4) essentially free to route, while any nonlocal coupling costs SWAP chains — the depth-78/32-ecr accounting of Section 25.1 is the honest unit of thinking. Single-qubit error rates sit around 10^{-4} , two-qubit around a few 10^{-3} ; the two-qubit count therefore dominates any reservoir step’s error budget.

Coherence and clocks. Transmon T_1, T_2 of order $10^2 \mu\text{s}$ against μs -scale gate layers permit circuit durations of hundreds of layers before decoherence saturates — but the reservoir-specific point is that NISQRC-style operation (Section 25.4) makes usable *memory* independent of this budget, which was demonstrated on exactly this platform [41]. Wall-clock throughput — circuits $\times$ shots per second — is the binding resource for time-series work,

and superconducting platforms lead it by orders of magnitude.

Mid-circuit measurement and reset. Production capability with per-step classical registers and classically conditioned gates (`if_else`); measurement durations of order μs with active reset. This is the primitive that separates genuinely recurrent hardware reservoirs from windowed feature maps, and its error (read-out confusion of order 10^{-2} , plus measurement-induced dephasing of spectators) should be characterised per-qubit before trusting long sequences; the natural-reservoir demonstrations of Yasuda et al. [48], Suzuki et al. [55] and the NISQRC experiment [41] are the platform’s QRC track record, alongside the utility-scale dynamics simulations that certify large-register control [81].

Error mitigation. Estimator-level mitigation (read-out mitigation, zero-noise extrapolation, probabilistic error cancellation) applies directly to feature estimation, at shot-multiplier costs that must be budgeted like any other shots [80]. For reservoirs a cheaper posture often suffices: keep circuits shallow, measure local observables, and let the read-out’s regularisation absorb residual bias — mitigation is worth its multiplier mainly when features must be compared across sessions.

Access economics. Queued cloud access with per-second pricing on premium plans; batched jobs (Part IV’s shape) amortise queue overhead. Session/batch primitives keep calibration approximately fixed within an experiment — important because a noise-defined reservoir drifts between calibrations (Section 15).

30 Quantum trapped-ion systems

The H-series QCCD architecture transports ions between gate zones, giving effective all-to-all connectivity, two-qubit fidelities at the 99.8–99.9% level, and mid-circuit measurement with conditional logic as a routine primitive [67]. For QRC: the fully connected disordered Hamiltonian of Eq. (11) compiles without routing overhead, and measurement/reset quality is the best available — the natural platform for *clean* recurrent-reservoir physics experiments (dynamical-regime studies, ESP characterisation) where per-step fi-

delity matters more than throughput. The cost is clock rate: transport and cooling make shot rates orders of magnitude below superconducting, so long-time-series benchmarks are expensive; restart-protocol studies with short windows, or physics studies with modest T , fit best. No published QRC benchmark at the scale of the superconducting or neutral-atom demonstrations exists as of this writing — flagged as an opportunity in Part VIII.

31 IonQ trapped-ion systems

Photonic-interconnect-oriented ion traps with all-to-all gates within a chain, accessed via native API and Braket. Similar QRC reasoning to Section 30 with smaller registers and, historically, less exposed mid-circuit control; algorithmic-qubit marketing figures should be translated back into raw two-qubit fidelity and shot throughput before budgeting an experiment.

32 Rigetti superconducting processors

Tunable-coupler transmons with faster two-qubit gates but historically lower fidelities than IBM; native parametric access and low-latency classical integration make it a reasonable testbed for feedback reservoirs (Section 14) where round-trip latency to classical logic is the experiment. For the pipelines of Part VII the IBM stack’s maturity currently wins on reproducibility.

33 Neutral-atom analog platforms (QuEra, Pasqal)

Rydberg tweezer arrays (Section 17) invert several trade-offs: hundreds of qubits with programmable *geometry*, analog evolution with no per-gate compilation error, but destructive fluorescence read-out — every time step is a full re-preparation, and there is no mid-circuit measurement in the digital sense. The 108-qubit QRC demonstration of Kornjača et al. [63] is the largest to date and defines the platform’s role: large feature banks via ensemble/restart operation on classification-style or short-window temporal tasks, with geometry and detunings as reservoir hyperparameters, and logical-array progress [64] indicating the control ceiling is still rising. Per-step latency (array reload of order tens of ms) makes long autoregressive forecasting loops the platform’s weak axis.

34 Photonic platforms (Xanadu-class CV)

Time-multiplexed squeezed-light processors [82] and tabletop multimode systems (Section 16) offer room-temperature operation, natural weak/continuous measurement, and clock rates far beyond matter qubits — attractive for streaming-signal reservoirs — with loss as the dominant decoherence and non-Gaussian resources as the scarce ingredient. Experimental optical QRC with controllable memory [57, 58] and large-scale Gaussian-boson-sampler feature maps [59] are the current state of practice; microwave-domain analog circuit QED [60, 66] plays the same role for RF signals.

35 Cross-platform summary

Two decision rules compress the table. If the task is a *long temporal sequence* (forecasting loops, streaming), throughput and MCM dominate: superconducting first, photonic for native-signal front-ends. If the task is a *physics question about reservoirs* (dynamical regimes, ESP, coherence’s role), fidelity and controllability dominate: trapped ions and neutral atoms first, with the analog platforms unique for geometry-as-hyperparameter questions. In every case the noise model is part of the reservoir: re-characterise it per session, and treat cross-session comparisons as different reservoirs unless proven otherwise.

Table 7: Platform comparison on QRC-binding axes (orders of magnitude, mid-2026 practice). “MCM” = mid-circuit measurement and reset quality; “throughput” = circuits×shots per wall-clock unit.

Platform	Connectivity	MCM	Throughput	Noise character	QRC record
IBM superconducting	heavy-hex, local	production	highest	depolarising + read-out; T_1 decay	NISQRC; natural reservoirs [41, 48, 55]
Quantinuum ion	all-to-all	excellent	low	low, well-characterised	(open)
IonQ ion	all-to-all (chain)	limited exposure	low	low	early studies
Rigetti supercond.	lattice, local	available	high	higher gate error	feedback testbed
QuEra/Pasqal neutral	geometry-program.	none (destructive)	medium (batch)	analog drift; re-prep.	108-qubit QRC [63]
Photonic CV	mode-mesh	continuous/weak	very high	loss	optical QRC [57, 59]

Part VI

Renewable-Energy and Climate Data: Catalogue and Analysis

36 Why this application domain

Renewable-energy and climate series are the right proving ground for QRC for reasons that survive scrutiny. The tasks are *temporal* with memory and nonlinearity at many scales — cloud advection in minutes, weather synoptics in days, seasonality in months, ENSO in years — exactly the structure reservoirs exist to exploit. The economic stakes are real and quantified: grid operators price forecast error directly, and the classical forecasting literature is mature enough to supply honest baselines [83–86]. The data are public, long, and well documented (Table 8). And the problem class connects to genuine scientific frontiers — data-driven weather models now rival numerical weather prediction [87, 88], and classical reservoir computing already operates at global-atmosphere scale [6, 7] — so quantum variants inherit a meaningful yardstick rather than a toy one.

The QRC-and-adjacent record on these problems, read with Part VIII’s discipline, is early but concrete. Hybrid quantum–classical reservoirs have modelled thermal convection and two-dimensional Rayleigh–Bénard turbulence — the fluid-dynamical core of weather — in simulation [69, 89]. The recurrence-free QRC of Ahmed et al. [52] targets chaotic dynamics and *extreme-event* prediction, the tail-risk problem that dominates both grid operations and climate adaptation, with shot-optimal training developed in Ahmed et al. [43]. Short-term load forecasting with gate-model QRC on simulated IBM backends reports competitive accuracy at few-qubit scale on real Moroccan-grid-adjacent data [53]; wind-power prediction has been benchmarked with quantum neural networks across feature maps and ansätze [90, 91]. Perspective pieces map where quantum computation could and could not matter for weather and climate [92] and for power systems generally [93, 94]. None of these works establishes quantum advantage; collectively they establish feasibility, define tasks, and — read critically — exhibit most of the benchmarking pitfalls Part VIII catalogues.

37 Dataset catalogue

Table 8 catalogues twelve public datasets spanning the domain. Selection criteria: documented provenance and licence, machine-readable access, length sufficient for chronological train/validation/test splits, and a live classical benchmark literature. The “QRC suitability” column encodes this document’s judgement of where each dataset meets current quantum reservoirs — which, per Parts IV–V, means *low-dimensional inputs* (scalar-to-few channels; high-dimensional fields need aggressive classical reduction first), *thousands of time steps* (enough for honest splits but within shot budgets), and targets where *memory and nonlinearity*, not raw dimensionality, carry the difficulty.

38 Three datasets in depth

Three series carry the document’s experiments: a solar irradiance series and a system-load series — both *surrogates*, generated by `datasets.py` to the statistical template of their public archetypes (NSRDB-like and GEFCom-like respectively) so that this document is self-contained, licence-clean, and exactly reproducible — and the *real* monthly ENSO sea-surface-temperature record. Every statistic and figure below is computed by `eda.py` from the shipped generators; the surrogate label is repeated wherever a number could otherwise be mistaken for a measurement of the physical world.

38.1 Solar irradiance (surrogate, NSRDB-like)

Construction. One year of 30-minute global horizontal irradiance (GHI) at a low-latitude site (1.35°N): a deterministic Haurwitz clear-sky curve modulated by a two-state Markov cloud regime (clear/cloudy) with Beta-distributed clear-sky index within regime, monsoon-season occupancy modulation, and 2% of daytime points removed in contiguous blocks to emulate sensor outages.

The forecasting variable. Solar work forecasts not GHI but the *clear-sky index*

$$k_t = \frac{\text{GHI}}{\text{GHI}_{\text{clear}}}, \quad (14)$$

Table 8: Public datasets for renewable-energy and climate QRC studies. Licences paraphrase the provider’s terms; verify before redistribution. “Resolution” is temporal $\times$ spatial where applicable.

Dataset (ref.)	Domain; licence	Variables; resolution	Size; span	Missing data; benchmark use	Preprocessing; QRC suitability
NSRDB [95]	solar; open (attrib.)	GHI/DNI/DHI, met. fields; 30 min $\times$ 4 km, CONUS+	TB-scale; 1998–	gap-filled product; solar-forecast standard	site extraction; clear-sky index (Eq. 14); high for univariate k_t tasks
SURFRAD [96]	solar (ground truth); open	broadband radiation suite; 1–3 min, 7 US sites	GB; 1995–	instrument gaps flagged; validation standard	QC flags; averaging; high for minute-scale ramp studies
WIND Toolkit [97]	wind; open	wind speed/power at hub heights; 5 min $\times$ 2 km	TB-scale; 2007–2013	modelled (WRF) — no sensor gaps; wind standard	site extraction; power-curve mapping; medium (modelled data caveat)
GEFCom2014 [98]	load/price/solar/wind; research use	hourly load, temperature stations; task-defined	MB-GB; 2005–2011	organiser-cleaned; probabilistic-forecast standard	pinball-loss targets; high for load track (used as template here)
UCI Electricity-LoadDiagrams [99]	load; open (CC-BY)	370 client loads; 15 min	140 MB; 2011–2014	zeros before client onboarding	client selection; resample; high ; standard in deep-forecasting papers
ENTSO-E Transparency [100]	grid; free account	load, generation by fuel, prices; 15–60 min, per zone	GB-scale; 2015–	reporting gaps by TSO	API pulls; zone selection; high for national-load tasks
OPSD [101]	power system; open	curated E+weather hourly	ENTSO-packages; GB; 2015–2020	documented per package	direct CSV; high ; lowest-friction European option
Grid stability [102]	grid dynamics; open (UCI)	4-node DSGC simulation, 11 features + label	10 k rows; static	none (synthetic)	classification target; medium : not a time series, useful for feature-map studies
Battery degradation [103, 104]	storage; open	cycling curves, capacity, impedance	GB; 124 cells / NASA packs	protocol-dependent	cycle summarisation; medium : long-horizon regression, slow “time” axis
ERA5 [105]	weather/climate reanalysis; open (CDS)	full atmospheric state; hourly $\times$ 31 km global	PB-scale; 1940–	complete (reanalysis)	variable/level/site extraction; medium : needs reduction to few channels
Jena weather [106]	weather (site); open	14 met. variables; 10 min	100 MB-scale; 2003–	short sensor gaps	resample; high : the standard small multivariate testbed
CMIP6 [107]	climate projections; open (per-model)	multi-model fields, scenarios; monthly/daily	PB-scale; 1850–2100	complete (model)	index extraction (e.g. Niño 3.4); low-medium : distribution-shift studies

which removes the deterministic solar geometry that would otherwise dominate every error metric [83, 84]; predicting GHI directly and reporting low errors is the domain’s classic self-deception, and the persistence baseline must be applied to k_t , not GHI.

Statistics (daytime, elevation $> 5^\circ$; $n = 8286$). GHI: mean 461, s.d. 292, median 425, 5th/95th percentiles 64/908, maximum 1041 (all W m^{-2}), skewness 0.15. Clear-sky index: mean 0.72, s.d. 0.28, median 0.87, skewness -0.93 — the strong left skew and the mass near $k_t \approx 0.9$ –1.0 are the bi-modal clear/cloudy signature that makes the series regime-switching rather than smoothly stochastic. Missing data: 273 daytime points (1.6%), longest gap 11 steps (5.5 h) — long enough that naive interpolation would fabricate a whole afternoon; the pipeline masks rather than fills.

Temporal structure (Figure 3). The daytime-

only autocorrelation of k_t decays within a few hours — memory is short and regime-bound. The Welch spectrum of GHI shows the harmonic comb of the diurnal cycle (24, 12, 8 h lines) atop a broadband cloud continuum; after the k_t transform the comb collapses, which is the quantitative content of Eq. (14). Implications for reservoir design: short usable memory (small MC requirement), strong conditional heteroscedasticity (regime transitions are where errors concentrate), and near-persistent behaviour at 30-minute horizons — which is why Part IV’s honest table shows persistence so hard to beat, and why value, if any, lives at ramp events and longer horizons.

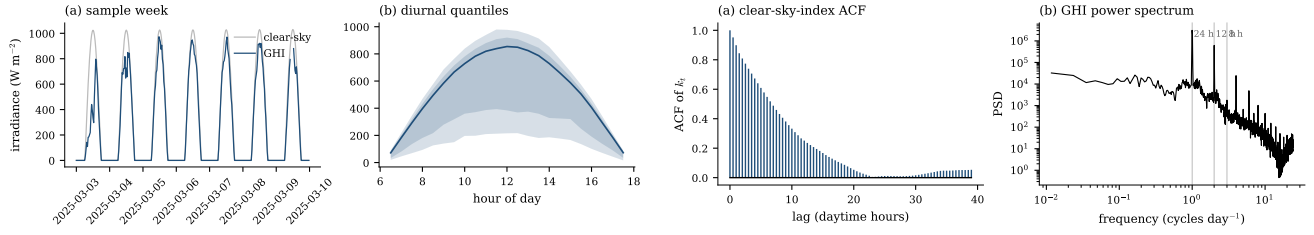


Figure 3: Solar surrogate EDA (from `eda.py`). Left pair: a sample week of GHI against the clear-sky curve, and the diurnal quantile fan. Right pair: daytime autocorrelation of k_t , and the GHI Welch spectrum with diurnal harmonics marked. Simulation on surrogate data.

38.2 System load (surrogate, GEFCom-like)

Construction. Three years of hourly system load: a double-peaked diurnal profile, weekday/week-end distinction (weekend factor 0.86), annual seasonality, a linear cooling-degree-day (CDD) temperature response of $55 \text{ MW}/^\circ\text{C}$ above the comfort threshold, AR(1) residuals, and eleven holiday days per year at factor 0.88.

Statistics. Mean 4280 MW, s.d. 542, range 2647–5618, skewness -0.10 ; correlation with coincident temperature 0.64 and with CDD 0.63 — the canonical load-weather coupling that makes exogenous inputs mandatory for competitive load forecasting [85, 98].

Temporal structure (Figure 4). The autocorrelation shows the double comb at 24h and 168h — daily and weekly cycles — persisting essentially undamped across the eight-day window: load is the opposite of solar, a *long-memory, strongly periodic* series where calendar features do most of the work and the residual after calendar and temperature is the actual forecasting problem. The correlation heat map (load, lags, temperature, CDD, calendar flags) quantifies the redundancy a read-out must negotiate. Implications: multivariate encoding (load lag, temperature, calendar) rather than scalar; memory requirements modest *after* the calendar is handled classically; the correct QRC posture is hybrid — quantum features for the weather-driven residual, classical features for the calendar — exactly the Part VII pipeline.

38.3 ENSO sea-surface temperature (real data)

Provenance. Monthly Niño-region sea-surface temperature, 1950–2010 ($n = 732$), the NOAA record as bundled with `statsmodels`; anomalies are computed against the monthly climatology, removing the seasonal cycle. This is the one fully real series in the experiments, and the one with genuine climate-dynamics content: ENSO is the domi-

nant mode of interannual variability, with delayed-oscillator dynamics — coupled ocean-atmosphere feedback with wave-transit delays — giving it intrinsic memory and weak chaos [108].

Statistics. SST mean 23.09°C , s.d. 2.25, range 18.95–29.24. Anomaly: s.d. 1.08°C , range -2.43 to $+4.60$, skewness $+1.15$ — the positive skew is physical (El Niño warm events run hotter than La Niña events run cold) and any model evaluated only on MSE will under-serve exactly the strong-event tail that matters.

Temporal structure (Figure 5). The anomaly autocorrelation decays over roughly 12–18 months with a weak negative lobe near 24 months — the oscillator signature; the variance-preserving spectrum $fS(f)$ concentrates power in the 2–7 year ENSO band. Implications: memory of a year-plus is *required*, which is precisely why the windowed ($L = 8$ months) feature map of Part IV loses to the recurrent reservoir here and why this series, small as it is, is the document’s sharpest architectural discriminator. The known hard cases — the spring predictability barrier, event-magnitude asymmetry — make ENSO indices a permanently honest benchmark: decades of classical effort define what “good” means.

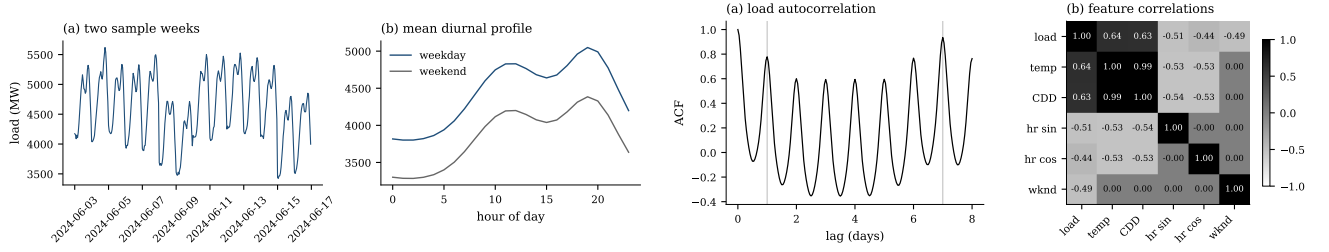


Figure 4: Load surrogate EDA. Left pair: two sample weeks, and weekday versus weekend diurnal profiles. Right pair: autocorrelation over eight days with daily/weekly lags marked, and the feature correlation matrix. Simulation on surrogate data.

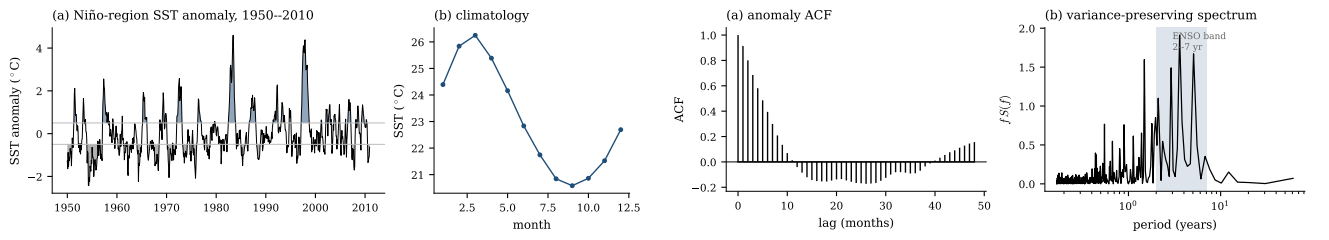


Figure 5: ENSO EDA (real data). Left pair: the anomaly series with $\pm 0.5^\circ\text{C}$ event shading, and the monthly climatology. Right pair: anomaly autocorrelation over 48 months, and the variance-preserving spectrum with the 2–7-year band marked.

Part VII

End-to-End Forecasting Pipelines

39 One pipeline, three tasks

The three application tasks of Part VI differ in physics but not in shape: every competent forecasting study, quantum or classical, traverses the same nine stages, and this part specifies them once so that the task walkthroughs of Section 41 can be read as three instantiations of one object rather than three recipes. The stages, with the discipline attached to each: (1) *acquisition* — source, licence, version pinned (Table 8); real versus surrogate declared in every caption downstream. (2) *quality control* — gaps located and *masked, not filled* unless the fill is itself validated; outages longer than the autocorrelation time split the series rather than bridging it. (3) *target transform* — forecast the variable the physics makes stationary (k_t , not GHI; anomaly, not SST; residual-after-calendar, not raw load), because every baseline and error metric inherits this choice. (4) *encoding* — scale inputs to the encoding’s natural domain ($[0, 1]$ for state replacement, angle range for rotations) using *training-set* statistics only; choose scalar reupload, multi-channel rotations, or state replacement per Part III. (5) *reservoir* — fix architecture, size, and operating point ($J\Delta t$, Section 40.1) before touching test data; record the seed. (6) *read-out* — ridge with λ chosen on a chronological validation block or by GCV (Section 2.2); features standardised by training statistics. (7) *uncertainty* — at minimum, residual-quantile intervals; for operational credibility, pinball-loss quantile read-outs (Section 42). (8) *baselines* — the full Part I battery (mean, persistence, linear lags, size-matched ESN) plus the Haar control; a missing baseline is a missing result. (9) *interpretation* — error decomposed by regime (clear/cloudy, weekday/weekend, event/quiet), not just averaged; claims scoped to what was actually run.

40 Reference experiments

All experiments below run the exact reference reservoir (Section 23) at the shipped configuration — $n = 5$ qubits, $V = 4$ virtual nodes, disordered Ising couplings $J = 1$, field $h = 1$, seed 7, thirty features — with every baseline evaluated at the same feature count. Everything is *simulation*; solar and load data are *surrogates*; ENSO is *real*.

The scripts print every number quoted.

40.1 Choosing the operating point: the temporal edge

The one reservoir hyperparameter with no classical analogue intuition is the injection interval Δt : how much unitary scrambling the register undergoes between inputs. Figure 6 sweeps $J\Delta t$ over 2.5 decades on NARMA-10. The dependence is strongly non-monotonic: at $J\Delta t \leq 0.25$ the propagator is near identity, the virtual-node features are nearly collinear, and the model cannot beat the mean predictor (NMSE ≈ 1.05); performance improves steeply to a minimum of 0.283 at $J\Delta t = 1$; beyond it, over-scrambling degrades the features to a saturation plateau around 0.6–0.75 for $J\Delta t \geq 4$. This is the *temporal* edge of Kobayashi and Motome [20] reproduced in a five-qubit Ising register: their analysis identifies two independent optima — a parametric edge, where the Hamiltonian sits near the boundary between regular and chaotic level statistics, and a temporal one, where information injected per step is scrambled enough to mix into the register but not so much that it thermalises before read-out — and prescribes exactly this sweep as the design procedure. Two practice notes. The optimum is task- and input-scaling-dependent (Section 4): re-sweep when either changes. And on hardware the sweep is cheap relative to everything else, because it reuses the same circuits with a rescaled evolution time — there is no excuse for publishing at an arbitrary Δt .

40.2 Memory capacity

At the chosen operating point, Figure 7 shows the delay-resolved memory function (Eq. (7), delays to 25). Total capacities: QRC 11.8; size-matched ESN 5.7; Haar-random control 4.5. Two honest readings. First, the *structured* reservoir at its edge retains linearly recoverable input over roughly twice the delay range of either control — and the gap to the *Haar* control specifically shows this is a property of the tuned dynamics, not of “being quantum”: a random unitary of the same size, same observables, same read-out remembers less than half as much. Second, MC is a diagnostic, not a score: it is measured on i.i.d. input, weights all

Table 9: Where each pipeline stage lives in the shipped code, per framework path. Stages 1–3 and 6–9 are classical and shared; the framework choice affects only stages 4–5 (and how features are estimated).

Stage	Reference (NumPy)	Cirq	Qiskit	PennyLane
1–3 data	<code>datasets.py</code> (generators and ENSO loader), <code>eda.py</code> (QC statistics)			
4 encoding	<code>input.state</code> , $u \in [0, 1]$	$R_Y(\gamma g_i u)$ re-upload	R_Y re-upload, transpiled	broadcast R_Y window
5 reservoir	<code>QuantumReservoir</code> (exact ρ)	frozen brickwork, exact $\langle O \rangle$	noisy backend, shots, MCM	frozen <code>StronglyEntanglingLayers</code> block
6 read-out	ridge via <code>ridge.fit</code> / GCV variant (Listing 4); identical maths in all paths			
7–9	<code>experiments.py</code> (baselines, regime slicing, figures); conventions of Part I			

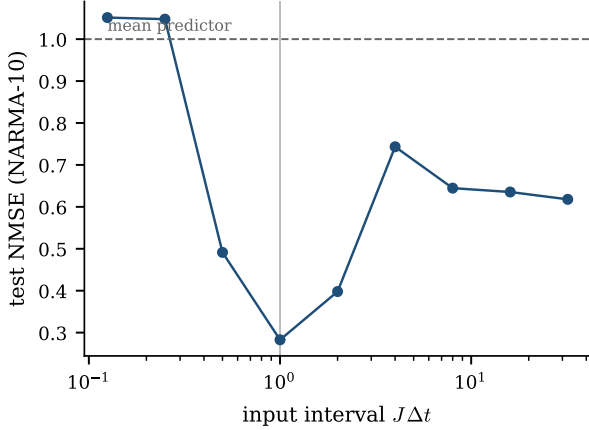


Figure 6: Test NMSE on NARMA-10 versus injection interval $J\Delta t$ (exact simulation; log axis). The minimum at $J\Delta t = 1$ is the temporal edge; the dashed line is the mean predictor.

delays equally, and says nothing about nonlinearity; the NARMA-10 column below, not this figure, is the performance claim.

40.3 Task demonstrations

Figure 8 shows test-span trajectories for the three tasks; Table 10 is the complete score sheet.

Reading the table with Part VIII’s eyes. *NARMA-10*: the QRC beats the size-matched ESN by 32% and the Haar control by $3\times$ — the designed benchmark result, on the synthetic task built to reward memory-plus-nonlinearity, consistent with the small-register literature [32, 46]. *Solar, one step*: four models within $\pm 3\%$ of each other and of persistence — a statistical tie. The honest conclusion is that 30-minute-ahead k_t at this site carries almost no exploitable nonlinear structure beyond its last value, and *no method*, quantum or classical, adds value here; a study reporting only the QRC column would look publishable and would mislead. *ENSO, three months*: every reservoir beats persistence decisively (memory helps), the tuned ESN leads, and the QRC trails

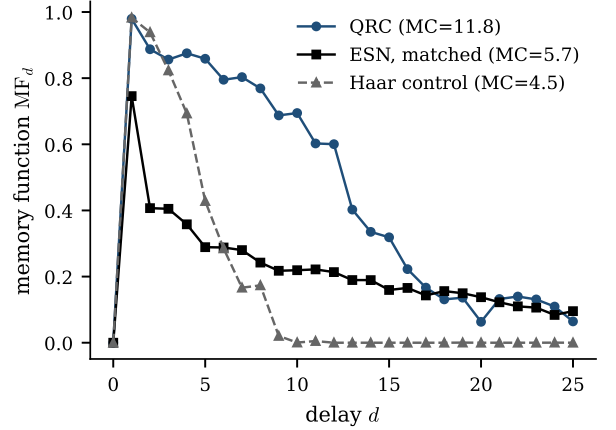


Figure 7: Memory function MF_d versus delay for the QRC at its operating point, the size-matched ESN, and the Haar-random control (exact simulation, i.i.d. input).

it by 7% — a genuinely informative defeat, in two ways. It quantifies the cost of the window-free but small quantum register on a long-memory task, and against Part IV’s windowed quantum map on the identical task (0.747) it isolates *recurrence itself* as worth 20 points of NMSE — the architectural conclusion this document considers its most transferable.

40.4 The shot budget

Figure 9 replays the NARMA-10 task with the exact features replaced by finite-shot estimates (eight noise seeds per point). The analytic ceiling is 0.283; the sampled model reaches 0.90 at $S = 10^2$, 0.64 at 10^3 , 0.53 at 10^4 , and 0.45 at 10^5 — converging as $S^{-1/2}$ but still 60% above ceiling at a hundred thousand shots per feature. The arithmetic this implies for hardware is the useful output: at thirty features and, say, $S = 10^4$, one time step of the restart protocol costs 3×10^5 measured circuits (times the window length), so a two-thousand-step training series is a 10^9 -shot commitment — weeks of queued device time at current

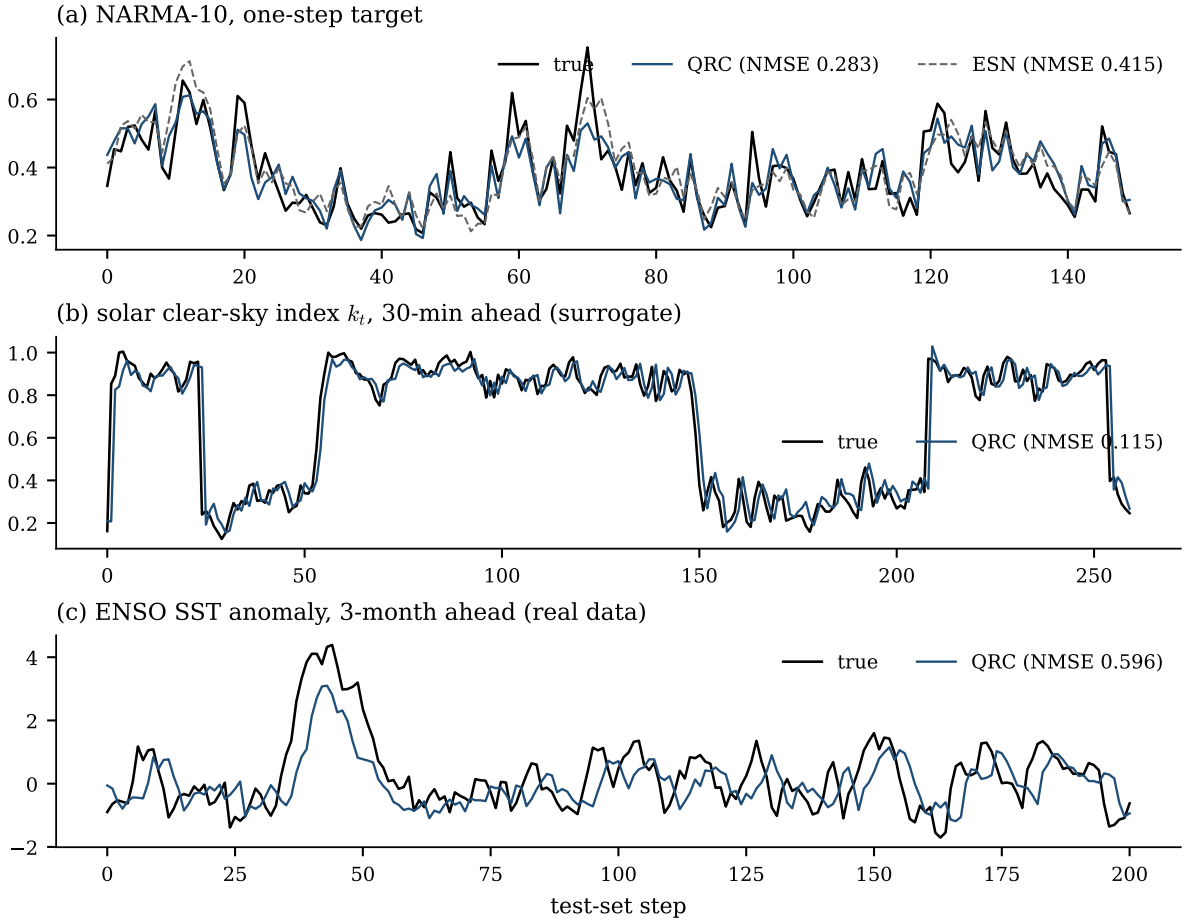


Figure 8: Test-span predictions (exact simulation of the reservoir). (a) NARMA-10 with QRC and size-matched ESN overlaid; (b) solar clear-sky index one step ahead (surrogate data); (c) ENSO SST anomaly three months ahead (real data). NMSE values in the legends match Table 10.

Table 10: Master results table: test NMSE, exact-expectation simulation, thirty features for all feature-based models, chronological splits. Bold marks the best model per task. “—” = not applicable.

Model	NARMA-10	Solar k_t , 1 step (surrogate)	ENSO anomaly, 3 mo (real)
QRC (Ising, edge)	0.283	0.115	0.596
ESN, size-matched	0.415	0.116	0.555
Haar control	0.872	—	—
Linear lags	0.740	0.113	0.567
Persistence	—	0.120	0.690
Mean predictor	1.000	1.000	1.000

throughputs, for a task a laptop solves exactly in seconds. This is not an argument against hardware experiments; it is the argument for *small* T , batched jobs (Part IV), shot-efficient observables, and for treating the eigentask analysis of Hu et al. [42] and shot-optimal training of Ahmed et al. [43] as required reading before any device campaign.

41 Task walkthroughs

41.1 Solar: 30-minute clear-sky index

Stages 1–3. Surrogate NSRDB-like series (real deployment: NSRDB site extraction, or SURFRAD for minute-scale work); gaps masked (1.6%, longest 5.5 h — split, never bridge); target k_t per Eq. (14), daytime only (elevation $> 5^\circ$), so the diurnal cycle never inflates skill. *Stage 4.* Scalar k_t , clipped to $[0, 1.2]$ then affinely mapped to $[0, 1]$ by training-set range, state-replacement encoding. *Stage 5.* Reference reservoir at $J\Delta t = 1$. *Stage*

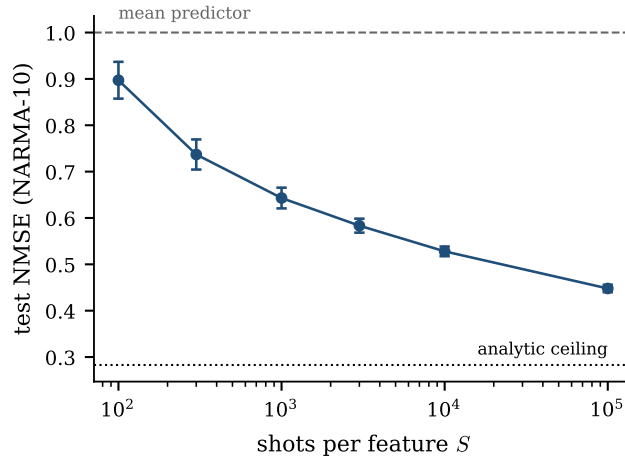


Figure 9: Shot-noise study on NARMA-10: test NMSE versus shots per feature S (mean $\pm$ s.d. over eight sampling seeds), against the exact-feature ceiling (dotted) and the mean predictor (dashed). Simulation.

6. Ridge, λ by validation block within the training year. *Stage 7.* Residual quantiles by cloud regime. *Stage 8–9.* Table 10: statistical tie with persistence — reported as such. Where a quantum study of this task could still earn its keep, per the EDA: not the one-step average, but ramp events (regime transitions) and 2–6 h horizons where persistence collapses; that is a different target (Stage 3) and a harder benchmark (event-based scores, Section 42), and it is listed as research direction R7 in Part VIII.

41.2 Load: day-ahead hourly demand (specified design)

This pipeline is *specified* against the surrogate’s EDA but not run in `experiments.py`; it is included because its structure — the hybrid posture — is the one most applications should copy, and it is the natural first project for a reader (Part VIII, R6). *Stages 1–3.* GEFCom-like hourly load with coincident temperature (real deployment: GEFCom2014 load track, ENTSO-E, or OPSD); target is the residual after a classical calendar model (hour-of-week means, holiday factor), because Figure 4 shows the calendar explains most variance and no quantum resource should be spent relearning a lookup table. *Stage 4.* Multi-channel rotation encoding: residual lag, temperature, CDD — three channels onto five qubits with fixed random gains. *Stage 5.* Reference reservoir; re-sweep $J\Delta t$ (the residual’s autocorrelation differs from NARMA’s). *Stage 6.* Hybrid read-out: quantum features concatenated with the calendar prediction and lags — Part IV’s Qiskit read-out shape, with the classical-only ablation mandatory. *Stage 7.* Pinball-loss quantiles, the GEFCom standard [98].

Stages 8–9. Success criterion declared in advance: beat the calendar-plus-linear model on weather-driven residuals, by regime; anything else is decoration.

41.3 ENSO: three-month anomaly

Stages 1–3. Real NOAA monthly SST (statsmodels copy pinned); anomaly against monthly climatology computed from training years only — computing climatology on the full record is a subtle leak. *Stage 4.* Scalar anomaly min–max mapped to $[0, 1]$ on training range, state replacement. *Stage 5.* Reference reservoir; recurrence is the point (Section 40.3) — windowed encodings need $L \gtrsim 18$ months before they stop losing to the recurrent register. *Stage 6.* Ridge; with $n = 732$ months total, the validation block is thin — GCV is the stable choice. *Stage 7.* Event-conditional errors: skill on $|\text{anomaly}| > 0.5^\circ\text{C}$ months reported separately, because the +1.15 skew means MSE under-weights exactly the warm events that matter. *Stages 8–9.* Table 10: beats persistence, trails the tuned ESN — stated plainly; the residual research question (can coherence-era registers close the recurrent-memory gap at matched read-out?) is R2 in Part VIII.

42 Uncertainty and evaluation beyond point NMSE

Operational forecasting is probabilistic, and the read-out’s linearity makes the upgrade nearly free: replacing the squared loss by the pinball loss at quantile τ , $\ell_\tau(e) = \max(\tau e, (\tau - 1)e)$ with $e = y - \hat{y}$, turns the same feature matrix into a quantile regression, one convex problem per τ —

the evaluation standard of the energy-forecasting community [85, 98]. Two further metrics are attached to the walkthroughs above because averages hide the value: *regime-conditional* NMSE (clear/cloudy; weekday/weekend; event/neutral), and *event-based* scores — hit/false-alarm rates for solar ramps and warm-ENSO exceedances — the frame in which extreme-event reservoir work is already cast [52]. A model that merely matches a baseline on average but improves the event tail would be operationally valuable and invisible to Table 10; none of the models above demonstrated that here, and the claim is left where it belongs, as an open, testable target.

Part VIII

Design Principles and Research Directions

43 Design principles

Ten principles condense Parts I–VII. Each is stated as an imperative with its justification and the place in this document where it was earned.

D1 — Name your contraction. Every working reservoir has an explicit mechanism that erases initial conditions and the remote past: a partial trace, a reset, engineered dissipation, a window truncation. State which one your design uses, verify it numerically (trajectory convergence from different ρ_0 ; PTM spectral radius), and check it contracts toward an *input-sensitive* region rather than the maximally mixed state (coherence influx, Section 9.3). Architectures that inherit stability from unexamined device noise are not reproducible across calibration cycles (Sections 15, 29).

D2 — Sweep the operating point. The injection interval and dynamical regime are not defaults but the reservoir’s most sensitive dials; the temporal-edge sweep of Section 40.1 moved NARMA-10 error by a factor of nearly four at fixed architecture. Re-sweep when the task, input scaling, or noise level changes; on hardware the sweep reuses circuits and is cheap (Figure 6; 20, 47).

D3 — Budget features, not qubits. The read-out sees K measured, well-conditioned directions, not $4^n - 1$ abstract ones. Track effective rank and memory capacity of the actual feature matrix; enrich observables (X , ZZ) and multiplex before adding qubits (Sections 10, 24; Table 3).

D4 — Treat the encoding scale as physics. Rotation encodings place the model’s approximation power on a frequency comb set by the scale γ [33]; the wrong scale wastes the register on harmonics the task lacks. The single largest error reduction in Part IV came from halving γ , and the cheapest gradient in the document was the one tuning it (Sections 24, 26).

D5 — Anchors before experiments. Build the validation suite first: identities with known answers (injection rebuild, unitarity, ridge recovery, NMSE of the mean = 1, shot-noise scaling). Every later anomaly is then attributable in minutes rather than days (Section 23.1).

D6 — Shots are the currency. Measure the exact ceiling before any sampled result; report S next to every number; budget mitigation multipliers explicitly; and read Hu et al. [42] and Ahmed et al. [43] before a device campaign. A method that

needs 10^5 shots per feature to approach its own ceiling has stated its cost (Sections 27, 40.4).

D7 — The baseline battery is part of the model. Mean, persistence, linear lags, size-matched ESN, Haar control; chronological splits; the physically stationary target (k_t , anomaly, residual). Part VII’s solar row exists to show how publishable a meaningless result can look when the battery is absent (Sections 6, 40.3).

D8 — Default to the hybrid posture, with ablations. Concatenate quantum features with the cheap classical ones and report quantum-only and classical-only columns; the difference is the measured quantum contribution. Part IV’s noisy hybrid *underperforming* its classical ablation is exactly the kind of result this posture is designed to surface (Sections 20, 25.3).

D9 — Respect concentration and symmetry at scale. Monitor feature variance and effective rank as n , depth, or noise grow; expect exponential signal collapse in scrambling or noise-dominated regimes and design against it (shallow steps, local observables, spatial multiplexing); resolve symmetry sectors before trusting chaos diagnostics or observable choices (Section 10.4; 31, 44, 45).

D10 — Scope every claim. Theory, simulation, or hardware; surrogate or real; the phrase “quantum advantage” only behind a controlled scaling comparison that includes cost. The defensible current claim for QRC in this domain is parameter-parity plus engineering fit, and this document says no more anywhere (frontmatter; Section 21).

44 Detecting real quantum utility

A benchmark that could in principle certify a quantum benefit has five properties, and their absence diagnoses most overclaims. *Matched read-out capacity:* classical baselines get the same number of trained parameters and comparable tuning effort. *Structure controls:* a Haar-random quantum reservoir of the same size separates “tuned dynamics” from “being quantum” (Figure 7); an efficiently simulable quantum control (Gaussian or Clifford reservoir) separates “quantum hardware” from “quantum formalism” [15, 72, 73]. *Scaling curves, not points:* performance versus size, shots, and data on both sides, since crossings, not gaps at

one configuration, are the object of interest. *Cost on the axis*: wall-clock, shots, and energy; Section 40.4’s arithmetic belongs in every hardware paper. *Pre-registered success criteria*: the target, metric, split, and baselines fixed before the test set is touched — the cheapest and most neglected of the five.

45 Research directions

Ten directions follow, ordered roughly by increasing ambition. Each gives motivation, background, a concrete route, difficulty, data, metrics, and publication potential. All are scoped so that a small group with the shipped code and modest cloud access can start immediately.

R1 — A temporal-edge atlas across architectures

Motivation: D2 is currently a per-study ritual; a systematic map would make it a lookup. *Background*: Kobayashi and Motome [20] establish the two-edge (parametric, temporal) structure for SYK-class reservoirs; Part VII reproduced the temporal edge in a 5-qubit Ising register. *Route*: sweep ($J\Delta t$, disorder, task-memory-length) across the Part III templates (Ising, chain, brickwork circuit) at $n \leq 12$ exact, correlating optima with level-spacing ratio and PTM gap; deliver a design chart and the sweep harness. *Difficulty*: low–medium (compute-bound, conceptually clean). *Data*: NARMA family, solar k_t , ENSO. *Metrics*: NMSE surfaces; optimum-location predictors. *Potential*: strong methods paper (PRApplied/PRResearch class); immediately useful tooling.

R2 — Closing the recurrent-memory gap on ENSO-class tasks

Motivation: Part VII’s sharpest result — recurrence worth 20 NMSE points over windowed maps, tuned ESN still ahead of the small register. *Background*: memory-augmented hybrids [68], feedback reservoirs [34], NISQRC’s coherence-independent memory [41]. *Route*: implement feedback and memory-augmented variants on the exact simulator; identify which mechanism closes the gap to the matched ESN at fixed feature count; then a small hardware run (mid-circuit reset, Part IV Part-B shape) on the real ENSO series. *Difficulty*: medium–high. *Data*: ENSO (real); Niño 3.4 from reanalysis for length. *Metrics*: NMSE and warm-event skill versus matched ESN. *Potential*: first

honest hardware benchmark on a real climate index; high.

R3 — Shot-optimal read-outs for forecasting

Motivation: Section 40.4’s arithmetic is the adoption barrier. *Background*: eigentask construction [42]; shot-optimal training [43]. *Route*: apply eigentask-weighted observable selection to the solar and load pipelines; produce NMSE-versus-total-shot-budget frontiers against naive uniform allocation. *Difficulty*: medium. *Data*: surrogate then NSRDB/GEFCom. *Metrics*: error at fixed budget; budget at fixed error. *Potential*: the paper every hardware group cites in its methods; solid.

R4 — Event skill: ramps and extremes

Motivation: Part VII shows averages are ties; operational value lives in tails. *Background*: extreme-event reservoir prediction [52]; energy community’s event metrics [83, 98]. *Route*: define ramp events on SURFRAD minute data and warm events on ENSO; train quantile read-outs (Section 42); compare event-conditional skill (POD/FAR, tail pinball) across the reservoir battery. *Difficulty*: medium (metrics discipline is the work). *Data*: SURFRAD (real), ENSO (real). *Metrics*: event scores, reliability diagrams. *Potential*: high — the first tail-focused QRC study in the domain would define the evaluation standard.

R5 — Noise as a characterised resource, on hardware

Motivation: D1/D5 versus the reality that device noise drifts. *Background*: natural reservoirs [55]; dissipation-as-resource [26]; coherence-influx criterion [27]. *Route*: on one superconducting backend, characterise the same reservoir circuit across calibration cycles; correlate task performance with measured T_1/T_2 /read-out drift; test the amplitude-damping-helps / dephasing-hurts prediction by Trotter-inserting engineered channels in simulation first. *Difficulty*: medium (patience-bound). *Data*: NARMA + solar surrogate. *Metrics*: performance-drift curves; influx diagnostics. *Potential*: a reproducibility protocol the field lacks; solid.

R6 — The hybrid load benchmark, pre-registered

Motivation: Section 41.2 is specified but unrun; it is the cleanest first real-data project. *Background*:

GEFCom practice [85, 98]; QRC load studies at few-qubit scale [53]. *Route*: execute the pipeline exactly as specified (calendar model, residual target, hybrid read-out, ablations, pinball quantiles) on GEFCom2014 and one ENTSO-E zone; pre-register the success criterion. *Difficulty*: low-medium. *Data*: GEFCom2014, ENTSO-E/OPSD (all real). *Metrics*: pinball loss versus calendar-plus-linear; ablation deltas. *Potential*: modest novelty, high credibility value — the reference others must match methodologically.

R7 — Solar beyond persistence: multi-horizon, regime-aware

Motivation: the one-step tie (Table 10) says move the goalposts to where value exists. *Background*: solar benchmarking practice [83, 84]; hybrid model-plus-reservoir posture [22]. *Route*: 2–6 h horizons on NSRDB/SURFRAD k_t with regime-conditional evaluation; test whether reservoir memory helps most at regime transitions; optionally fuse a cheap clear-sky/NWP prior. *Difficulty*: medium. *Data*: NSRDB, SURFRAD (real). *Metrics*: NMSE by horizon and regime; ramp scores (with R4). *Potential*: directly relevant to the application community; good cross-venue paper.

R8 — Empirical concentration scaling for QRC

Motivation: D9 currently cites theory; design needs measured curves. *Background*: exponential concentration and symmetry analysis [31, 45]; eigentask noise ceiling [42]. *Route*: exact simulation to $n = 12$ –14: feature variance, effective rank, and task NMSE versus n , depth, noise rate, and symmetry sector; fit decay constants; identify the multiplexing crossover where two n -qubit registers beat one $2n$ -qubit register at fixed shots. *Difficulty*: medium-high (careful numerics). *Data*: NARMA suffices. *Metrics*: scaling exponents; crossover maps. *Potential*: the quantitative backbone for honest scaling claims; strong.

R9 — An analog-platform energy demonstration

Motivation: the largest registers are analog (Section 33); the domain lacks any energy-data result on them. *Background*: 108-qubit Rydberg QRC [63]; global-detuning encoding’s size-independent memory. *Route*: map a short-window load or ramp *classification* task (matching the platform’s restart/ensemble operation) onto

global-detuning waveforms in emulation; then a partner-hardware run; report against the full battery including a matched classical kernel. *Difficulty*: high (platform access and encoding design). *Data*: GEFCom/SURFRAD-derived labels. *Metrics*: accuracy at matched trained parameters; shots and wall-clock on the axis. *Potential*: headline-adjacent if honest; must be pre-registered to stay so.

R10 — Clean-physics ESP and edge studies on trapped ions

Motivation: Part V flags the absence of ion-platform QRC benchmarks despite the best per-step fidelities and native all-to-all coupling. *Background*: QCCD mid-circuit primitives [67]; dynamical-phase reservoir physics [20, 47]. *Route*: small- T , high-fidelity implementations of the Eq. (11) reservoir with mid-circuit reset; measure trajectory-convergence rates (ESP directly, not via proxy) and the temporal edge on-device; compare with exact simulation at identical seeds. *Difficulty*: high (access; shot-rate limits set T). *Data*: NARMA-2/10 at small T . *Metrics*: measured contraction rates; edge location on-device versus simulation. *Potential*: the cleanest hardware-theory comparison available to the field; high scientific value independent of any advantage question.

46 Closing statement

This document set out to be the reference its own Part VIII demands: theory stated with its assumptions, implementations that run, data characterised before being forecast, baselines that would embarrass a weak claim, and research directions scoped to be started rather than admired. The honest summary of the field it surveys is neither triumphal nor dismissive. Small quantum registers, operated at their dynamical sweet spot, demonstrably match classical read-outs several times their size on memory-nonlinearity benchmarks, and the engineering paths around their two structural taxes — measurement statistics and the recurrence gap — are active and credible. Whether that matures into practical value for renewable energy and climate forecasting is exactly the kind of question that is settled by pre-registered benchmarks on real data, run by people who would publish either answer. The code, data discipline, and directions above are offered to make that settling faster.

Part IX

A Staged Repository: From NumPy Reservoir to Product

This part designs, from scratch, a single repository that carries the whole programme of this document: a small exact quantum reservoir in NumPy; the same reservoir expressed as hardware-faithful circuits on an exact simulator; noisy simulation of a simple Hamiltonian; a battery of more complex Hamiltonians; qubit-reuse compilation of those Hamiltonians; an independent recurrence-free reservoir (RF-QRC, Part X); the integration of RF-QRC with the existing test architecture and the reuse compiler; and finally the product pipeline of Part XI. The repository is not hypothetical: Stages 1–5 already exist as verified code (the `code/` directory shipped with this document and the companion qubit-reuse package), and the design below states exactly which files move where, so that assembling the repository is a refactor with green tests at every step rather than a rewrite.

47 Design rules the repository inherits

Six rules, all earned earlier in this document or in the companion projects, are promoted from habits to repository law.

R1 — Anchors first. No stage accepts code before its validation anchors exist and fail meaningfully when sabotaged. The anchor suite of Part IV (inject/rebuild identity, propagator unitarity, ρ Hermitian/trace-one/PSD, ridge recovery of a known linear map to 10^{-10} , NMSE(mean) = 1.000, shot-noise $\sigma \propto S^{-1/2}$) seeds `stage0_anchors/`, and every later stage appends its own.

R2 — The exact reference defines the ceiling. The NumPy density-matrix reservoir of Stage 1 is the semantic definition of every architecture in the repository; framework ports (Qiskit, Cirq, PennyLane) are validated *against it at identical seeds*, never against intuition.

R3 — One endianness point. Qiskit count keys are little-endian; `counts_to_features` performs the single `key[::-1]` reversal and nothing else in the repository may reorder bits.

R4 — Printed configuration, fixed seed. Every experiment script prints its full configuration block at runtime; `SEED = 7` project-wide; no result enters a table unless a script printed it.

R5 — Negative controls are first-class. The all-to-all zero-compression rows (SK, SYK₄), the

Haar-random structureless reservoir, and the mean-predictor NMSE = 1.0 line ship in the same harness as the headline configurations and appear in the same tables.

R6 — Promotion gates, not vibes. A stage is *done* when its exit criterion — a script with an exit code — passes; downstream stages import only from stages that are done. The repository’s continuous integration is exactly `run_tests.py` run stage by stage.

48 The stage ladder

Each stage answers one question and owns one directory. Table 11 summarises; the subsections that follow give the contents, the entry and exit criteria, and the mapping to existing assets.

```
1 qrc-stack/
2   README.md
3   run_tests.py
4   stage0_anchors/
5     test_core_anchors.py
6   stage1_numpy_core/
7     qrc_core.py
8     datasets.py
9   stage2_circuits_exact/
10    qiskit_qrc_hw.py
11   stage3_noisy_simple/
12    noise_models.py
13   stage4_hamiltonian_battery/
14    hamiltonians.py
15   stage5_qubit_reuse/
        pyproject.toml
        # anchors-first driver; exit 0 iff all stages green
        configs/
        test_reuse_anchors.py
        test_rfqrc_anchors.py
        baselines.py
        eda.py
        tasks.py
        figstyle.py
        cirq_qrc_minimal.py
        pennylane_qrc_hybrid.py
        exp_tfim1d_noisy.py
        exp_family_scan.py
        # 7 families, <r> dial
```

Table 11: The repository’s stage ladder. “Exists” names verified code that seeds the stage; new files are set in italics.

#	Question answered	Seed assets	Exit criterion
0	do our instruments work?	anchor suites of both code-bases	all anchors pass; sabotage test fails
1	exact small-scale QRC (NumPy)	<code>qrc_core.py</code> , <code>experiments.py</code>	NARMA-10/MC results reproduce Part VII numbers
2	same reservoir as circuits (exact)	<code>qiskit_qrc_hw.py</code> , <code>cirq_qrc_minimal.py</code>	circuit features = NumPy features at seed 7
3	noisy simple Hamiltonian	Part IV noise pipeline	noisy-vs-exact degradation curves printed; gate-local caveat stated
4	complex Hamiltonians	seven-family benchmark (reuse repo)	$\langle r \rangle$ diagnostics hit 0.386/0.531 anchors; family scan runs
5	qubit reuse on those	<code>greuse_*</code> (7 modules)	self-calibrated TVD gate; compression table incl. zero-compression rows
6	independent RF-QRC	<i>rfqrc_reservoir/metrics/baselines.py</i> , <i>mfe_model.py</i>	Part X Phase 0–4 acceptance
7	RF-QRC $\times$ reuse integration	<i>greuse_batch.py</i> + Stage 5+6	batch-and-compress validated; width independent of memory horizon
8	product pipeline	<i>data_plane/</i> <i>serving/</i> <i>decision/</i> <i>surfaces/</i> <i>governance/</i>	Part XI Phase 8–12 acceptance

```

16  greuse_ir.py          greuse_analysis.py    greuse_scheduler.py
17  greuse_compiler.py   greuse_validation.py  qrc_experiment.py
18  exp_reuse_compression.py
19  stage6_rfqrc/
20  rfqrc_reservoir.py   rfqrc_baselines.py    rfqrc_metrics.py
21  mfe_model.py         exp_mfe_extremes.py   exp_lorenz63_check.py
22  stage7_integration/
23  greuse_batch.py      exp_rfqrc_reuse.py    exp_vn_fusion_proto.py
24  solar_ramps_data.py  exp_solar_ramps.py
25  stage8_product/
26  data_plane/         connectors.py qc.py transforms.py feature_store.py replay.py
27  serving/            feature_provider.py heads.py conformal.py shadow_battery.py
28  decision/           stacker.py regime_router.py thresholds.py alerts.py
29  surfaces/           api.py dashboard/ agent/ # tools, guardrails, RAG
30  governance/         monitors.py promotion_gates.py model_cards.py
31  docs/               # this handbook, regenerated figures

```

Listing 6: Repository tree. Stages 1–5 are populated by moving verified files; Stages 6–8 are populated by the plans of Parts X–XI.

48.1 Stages 0–5: consolidating what exists

Stage 0. The two anchor suites merge under one driver. The merged driver keeps the qubit-reuse suite’s self-calibration idea — an equivalence test passes only if $\text{TVD}(\text{direct}, \text{reuse}) \leq 2.5 \times \text{TVD}(\text{direct}, \text{direct-rerun}) + 0.02$ — and Part IV’s exact-simulation anchors. A deliberate-sabotage mode (flip the endianness reversal, break a Kraus normalisation) must turn the suite red; an anchor that cannot fail is decoration.

Stage 1. `qrc_core.py` moves in unchanged: the Fujii–Nakajima density-matrix reservoir [32], the classical baselines, and the task generators. Its NARMA-10, memory-capacity, and ENSO numbers (Part VII) are the stage’s regression targets. The recurrent 5-qubit CZ-ring ENSO reservoir stays here as the long-memory comparator that Stage 6 must face honestly.

Stage 2. The framework ports (Part IV) move in with their validated-against-reference-at-identical-seed tests. This stage owns the transpilation path (native basis, coupling maps) that makes “hard-

ware simulator” more than a phrase: circuits leave this stage ready for a device even though this repository runs none.

Stage 3. Noisy Aer on the simplest Hamiltonian family (TFIM-1D): depolarising and thermal-relaxation models, degradation curves versus noise strength and shots. The stage’s documentation carries the caveat this project measured the hard way: *gate-local noise models cannot see the 3–5× depth inflation that qubit reuse introduces*, so Stage 5 noise claims must never be extrapolated from Stage 3 models alone.

Stage 4. The seven-family Hamiltonian battery (TFIM-1D, mixed-field Ising, XXZ, XY, TFIM-2D, Sherrington–Kirkpatrick, JW-mapped SYK₄) with the disorder dial W and level-spacing diagnostics [20, 47]; the $\langle r \rangle$ anchors 0.386 (Poisson) and 0.531 (GOE) gate the stage.

Stage 5. The reuse compiler’s seven modules move in unchanged with their compression scans. The stage’s headline artefacts are the interaction-graph law (nearest-neighbour chains compress $10 \rightarrow 5$ regardless of gate count; the 2D lattice less; all-to-all not at all) and the equivalence contract: reuse preserves the *classical measurement distribution* only — exactly the contract a QRC read-out needs, and not one a protocol needing the unmeasured state may assume (DeCross *et al.*, Phys. Rev. X **13**, 041057 (2023)).

48.2 Stages 6–8: the new work

Stage 6 implements Part X Phases 0–4: the recurrence-free reservoir with virtual-node read-out, its metric battery (predictability horizon, event F-score versus prediction time, VPT, effective rank), the expanded baseline set (NVAR, QELM ablation, Haar control), and the MFE extreme-event anchor study, with the Lorenz-63 side check where the classical reservoir is expected to win at matched degrees of freedom [43, 52].

Stage 7 implements Part X Phases 5–6: the solar-ramp study on registry data, the batch-and-compress path through the Stage-5 compiler (`qreuse_batch.py` lays B independent step circuits side by side with terminal measurements only, which the existing parser accepts today), the structural claim that compiled width is independent of memory horizon (contrast the windowed protocol’s measured physical = $T+1$), and the virtual-node fusion prototype behind an IR extension for measurement-terminated blocks.

Stage 8 implements Part XI Phases 8–12: the data plane with its leakage firewall, the serving core behind the `FeatureProvider` interface, the

calibrated decision layer, the product surfaces including the provenance-guarded agent, and the governance monitors whose promotion gates are the pre-registered criteria of Parts X–XI.

49 Promotion protocol

Work proceeds strictly up the ladder. To promote stage k : (i) its anchors and exit script pass with exit code 0; (ii) its numbers reproduce the regression targets of the documents that earned them (this handbook for Stages 1–3, the reuse report for Stages 4–5, Parts X–XI for Stages 6–8); (iii) any interface it exports downstream is frozen and versioned; (iv) a short stage report — configuration block, tables, honest failures — lands in `docs/`. Cross-stage experiments (Stage 7 onwards) may only import frozen interfaces. The repository is *finished* in the same sense this document is: a stranger rebuilds every figure and table with two commands, and every claim is scoped theory, simulation, or hardware, with surrogate data labelled wherever it appears.

Part X

Recurrence-Free QRC with Virtual-Node Readout: Theory and Implementation Plan

This part converts the project’s implementation plan for its next architectural step into the handbook. The plan is exactly that — a plan: no result below is claimed, every future number is simulation unless scoped otherwise, and the success criteria of Section 55 were fixed before any test data is touched. Notation: in this part the leak rate of the *classical* feature integrator is written ε (matching the RF-QRC literature) to avoid collision with the handbook’s classical-reservoir leak α ; V denotes the fixed random reservoir unitary.

50 The architectural improvement, precisely stated

The improvement composes three published components, chosen because they attack the two bottlenecks this project has itself measured: a memory-horizon-coupled circuit depth, and low effective feature rank.

(a) *Recurrence removal with leaky-integrated classical memory (RF-QRC)*. The windowed restart reservoir of the qubit-reuse experiment (Stage 5) encodes the last T inputs in one circuit, so the memory horizon *is* the circuit depth; the compile-time cost was measured directly as physical qubits = $T+1$. RF-QRC [52] replaces in-circuit memory with a classical leaky integrator over measured features (the input map Φ is uploaded twice):

$$|\psi_k\rangle = V \Phi(u_k) \Phi(u_k) |0\rangle^{\otimes n}, \quad (15)$$

$$r_k = (1 - \varepsilon) r_{k-1} + \varepsilon \phi(u_k), \quad (16)$$

$$\hat{y}_k = W_{\text{out}} r_k,$$

where $\phi(u_k)$ collects the measured features of $|\psi_k\rangle$, $\varepsilon \in (0, 1]$ is the classical leak rate, and W_{out} is the ridge read-out. The circuit now encodes one input (optionally a short window $m \leq 3$; Section 51), its depth is constant in the memory horizon, and the T per-step circuits are mutually independent — embarrassingly parallel, and batchable through the reuse compiler.

(b) *Virtual-node (temporally multiplexed) readout*. Features are collected at N_v intermediate evolution times $\{t_1, \dots, t_{N_v}\}$ inside the step circuit — equivalently, the Heisenberg-evolved observables

$\{O(t_i)\}$ are measured [32, 38]:

$$\begin{aligned} \phi(u_k) &= [p(t_1); \dots; p(t_{N_v})] \in \mathbb{R}^{N_v 2^n} \\ \text{or } [\langle Z_i(t_j) \rangle, \langle Z_i Z_l(t_j) \rangle] &\in \mathbb{R}^{N_v(n+\binom{n}{2})}. \end{aligned} \quad (17)$$

In exact simulation the snapshots are free (probabilities at layer barriers); in shots mode the default realisation is N_v truncated circuits, with a reused realisation as a Stage-7 extension.

(c) *Extreme-event evaluation battery and expanded controls*. Predictability horizon (PH), precision/recall/F-score versus prediction-time offset, and valid prediction time join NMSE; the baseline battery gains NVAR (next-generation reservoir computing; Gauthier *et al.*, Nat. Commun. **12**, 5564 (2021)), a QELM ablation ($\varepsilon = 1$), and this project’s own recurrent reservoirs as comparators, on top of the standing five (mean = 1.0 line, persistence, linear lags, size-matched ESN, Haar-random control).

51 Theory: why this should work, and where it can fail

51.1 The function class and its honest limitation

Unrolling Eq. (16) gives a Volterra-like series with an exponential kernel [11],

$$\hat{y}_k = \sum_{d \geq 0} \varepsilon (1 - \varepsilon)^d W_{\text{out}} \phi(u_{k-d}), \quad (18)$$

which is *additive across delays*: no cross-delay products $\phi(u_{k-d}) \otimes \phi(u_{k-d'})$ appear, although recurrent reservoirs and NVAR possess exactly those terms and NARMA- n targets contain them literally. This limitation is not stated in the RF-QRC literature. Two remedies are kept as switchable ablations: (i) a short in-circuit window $m \leq 3$, encoding $(u_k, \dots, u_{k-m+1})$ across qubit groups so the Born-rule quadratic and the entangler generate cross-delay products within the window ($m = 1$ recovers pure RF-QRC); (ii) a nonlinear read-out head (degree-2 polynomial or RBF-SVR), offered under a fairness rule to every classical baseline in the same table. Extreme events may nevertheless sit in the architecture’s sweet spot: turbulent bursts and solar ramps are threshold-crossing

phenomena whose precursors are plausibly low-order nonlinear functions of a *short* recent window — the high-nonlinearity/short-memory corner of the memory–nonlinearity trade-off [12] — whereas long-nonlinear-memory tasks should favour recurrence, as this project’s own ENSO comparison (recurrent NMSE ≈ 0.60 versus windowed ≈ 0.75 , Part VII) already witnesses.

51.2 Echo-state property by construction

Proposition 51.1 (Fading memory of the leaky recurrence-free map). *For any input sequence and any r_0, r'_0 , the states generated by Eq. (16) obey $\|r_k - r'_k\| \leq (1 - \varepsilon)^k \|r_0 - r'_0\|$.*

Proof. The map $r \mapsto (1 - \varepsilon)r + \varepsilon\phi(u)$ is affine in r with Lipschitz constant $(1 - \varepsilon) < 1$, uniformly in u ; iterate. $\square$

The echo-state property therefore holds for every $\varepsilon \in (0, 1]$, independent of whether the quantum dynamics is dissipative, unital, or chaotic — in contrast with recurrent QRC, whose contraction must be engineered and verified (PTM spectral radius, coherence influx; Part II), and with closed-unitary parameter-encoded reservoirs, which have no fading memory at all. Washout still discards N_w steps, but only to flush the classical r_0 . The kernel $(1 - \varepsilon)^d$ makes ε a pure memory dial, decoupled from the circuit — a hyperparameter separation the recurrent architecture does not offer [9].

51.3 Capacity and the virtual-node lever

Total information-processing capacity is bounded by the number of linearly independent read-out functionals [12]; single-time local- Z readout of n qubits caps them near n regardless of Hilbert dimension. Virtual nodes raise the cap: the read-out optimises over $\text{span}\{O(t_1), \dots, O(t_{N_v})\}$, since $\hat{y} = \sum_i w_i \text{Tr}[\rho_k O(t_i)] = \text{Tr}[\rho_k \sum_i w_i O(t_i)]$. The correlated-spin NMR experiment of arXiv:2508.12383 extracted 653 features from nine spins this way and attributes its one-to-two-order NARMA improvement and its daily-climate result largely to this lever. The caveat this project has already measured: nominal feature count is not capacity — the Stage-5 experiment recorded *effective rank* ≈ 2.2 on a nominally 2^{12} -dimensional feature vector under a scalar drive, the exponential-concentration problem in miniature [31, 45]. The participation ratio of the

feature covariance is therefore a mandatory reported diagnostic, and the mitigations are modest n (8–11), multichannel encoding, N_v virtual nodes, and operating below the scrambling regime.

51.4 Nonlinearity, re-uploading, and the encoding-wrap trap

The map’s nonlinearity stacks the Born-rule quadratic on a truncated Fourier series in the encoded input whose accessible frequency ladder grows with the number of data-encoding layers [33]; uploading Φ twice — the RF-QRC signature — doubles the ladder, consistent with the published ablation in which only the double-upload fully entangling map matched the recurrent architectures [52]. Encoding gain remains a swept parameter: this project’s own sweep found $\gamma = \pi$ wraps the RX encoding and destroys the linear read-out (NMSE ≈ 1.1) while $\pi/4$ was selected (≈ 0.03); the arcsin encoding is kept as a wrap-free alternative.

51.5 Operating point: the two edges

The inter-node evolution interval τ (equivalently $J\Delta t$) is swept log-spaced over at least two decades — the temporal edge, bounded by the Thouless time [20]; the published τ sensitivity in the NMR experiment is this edge in the wild. Where Hamiltonian-evolution entanglers are used (Stage-4 families), on-site disorder W is the parametric dial with the $\langle r \rangle$ anchors already unit-tested [47].

51.6 Shot noise

Without recurrence, finite-sampling noise is uncorrelated in time and cannot compound through a feedback loop — the mechanism by which sampling noise degrades recurrent QRC faster [41, 43]. The leak is itself a first-order low-pass filter; on top sit SVD low-rank truncation of the reservoir matrix and per-feature Savitzky–Golay filtering, which reduced training loss by roughly two orders of magnitude in the published hardware runs [43]. Standing rule: measure the noiseless analytic ceiling before any shot study, and report shots and wallclock next to accuracy.

51.7 Failure modes kept in view

(i) Effective-rank collapse despite N_v (the rank-versus- N_v curve is a primary figure whatever it shows). (ii) NVAR dominance at matched feature count — RF-QRC without the leak is a random nonlinear feature map of the current input,

so NVAR is its sharpest classical analogue and is absent from the published comparisons; if it wins, that is the headline. (iii) Exogenous stochastic extremes: solar ramps are cloud-advection driven, so the precursor information may be absent from a univariate series; an honest null on solar beside a positive MFE anchor is a coherent outcome. (iv) Long-memory tasks: an expected loss to recurrence on ENSO, declared in advance.

51.8 Fit to the qubit-reuse stack

The compiler’s equivalence contract preserves exactly the classical measurement distribution — all a QRC read-out consumes. RF-QRC step circuits have constant depth, so causal cones stay local and the Stage-5 width-scan result (constant physical count for logical $n = 8\text{--}40$) now holds at *any* memory horizon: the measured pathology physical $= T+1$ disappears because memory moved from a quantum resource to the classical ε . Batches of independent step circuits carry terminal measurements only and are parser-legal today; virtual-

node fusion — N_v truncated circuits joined by measure-and-reset boundaries on one register — shares its primitive with the reuse compiler and with reset-based streaming QRC [41], and waits behind an IR extension for measurement-terminated blocks.

52 Evidence base

53 Integration with the existing codebase

The compiler modules are untouched; `qrc_experiment.py`’s circuit builder is refactored into a shared step-circuit factory (the windowed protocol survives as the `recurrent-window` comparator); `counts_to_features` remains the single endianness point; the recurrent CZ-ring of `qrc_core.py` is imported as the long-memory comparator. New modules are those of repository Stages 6–7 (Listing 6). The configuration surface:

```

1 @dataclass
2 class RFQRCConfig:
3     n_qubits: int = 10
4     window_m: int = 1          # 1 = pure RF-QRC; 2-3 = short in-circuit window
5     n_uploads: int = 2         # RF-QRC signature; 1 = ablation
6     entangler: str = "brickwork_zz" # or "ham_trotter:<family>", "haar_control"
7     entangler_layers: int = 2
8     gamma: float = np.pi / 4  # swept {pi/4, pi/2, pi}; or "arcsin"
9     tau: float = 1.0           # J*dt; swept log over >= 2 decades
10    disorder_W: float = 0.0
11    n_virtual_nodes: int = 4    # snapshots at uniform layer boundaries
12    readout: str = "full_probs" # or "local_z", "local_zz"
13    leak_eps: float = 0.3       # swept log-grid (0.02 .. 1.0]
14    shots: Optional[int] = None # None = exact snapshots
15    denoise: str = "none"       # "svd:<k>", "avgol:<w>,<p>"
16    ridge_beta: tuple = (1e-6, 1e-9, 1e-12)
17    seed: int = 7

```

Listing 7: Configuration dataclass, printed at runtime (project convention).

54 Phased plan with acceptance criteria

Phase 0 — anchors. Environment restored and the untouched suites green; then the new anchors of Table 13, each with an exact expected value, appended to `run_tests.py`.

Phase 1 — core module. `rfqrc_reservoir.py`: shared step-circuit factory; exact path via statevector snapshots at layer

barriers; shots path via truncated circuits through the existing Aer runner; leak applied elementwise after multiplex concatenation; washout. Accept: anchors green; exact versus 10^6 -shot truncation agree within shot noise on a 4-qubit test.

Phase 2 — read-out and training. Ridge with chronological splits; λ by chronological validation block or GCV via SVD when the tail block is thin; denoisers between features and ridge in shots mode; optional heads behind the fairness rule. Ac-

Table 12: Published evidence for the adopted components. All are simulation except where noted; none runs a Haar-random or NVAR control — the gap this plan fills.

Source	Model	Task / baseline	Result and caveat
Ahmed et al. [52]	RF-QRC, double up-load, leak	MFE extreme events; tuned size-matched leaky ESN	median PH 8.09 Lyapunov times at 11 qubits vs classical plateau ≈ 5.5 ; higher F-score; Lorenz-63 loss reported. Noise-free statevector.
Ahmed et al. [43]	RF-QRC + SVD / Savitzky–Golay	shot-noise study; 10 qubits on <code>ibmq_kyoto</code>	recurrence-free degrades more slowly under sampling noise; denoising ~ 2 orders on training loss. Hardware metric is training loss, not PH.
arXiv:2508.12383	9-spin NMR, natural dissipation, time-multiplexed FID (653 features)	NARMA 2–20; Delhi daily climate vs ESN(500–10000)	NARMA error down 1–2 orders vs prior QRC experiments; temperature $\geq$ ESN(1000), with RBF-SVR head above ESN(10000). Ensemble NMR is backaction-free.
C-QRC Square (Research preprint, 2025)	gate-QRC features feeding LSTM/Transformer/XGBoost	solar generation, rolling forecast	gains concentrate in high-volatility and ramp regimes, not average accuracy. Preprint; cite only after audit.

Table 13: New validation anchors (Phase 0). “Exact” entries carry no shot noise.

Component	Anchor	Expected
leak	impulse response	$r_d = \varepsilon(1 - \varepsilon)^d$ exactly
leak	memory function, iid input	matches analytic form
virtual nodes	2-qubit hand-solved circuit	snapshot probabilities $= \text{Tr}[\rho(t_i)P]$
virtual nodes	snapshot vs truncation	identical distributions, TVD $< 10^{-12}$
endianness	known basis state	feature ordering matches statevector
PH ladder	synthetic ramp series	equals hand computation; ladder terminates
F-score	event-free window	degenerate cases handled
NVAR	known quadratic map	exact recovery to 10^{-10}
Haar control	two distinct inputs	distinct features; structure-free
NMSE	mean predictor	exactly 1.000

cept: ridge anchor; one evaluation harness applies any head to all models.

Phase 3 — metrics and battery. PH frozen to the Racca–Magri convention (criterion $|k_{\text{pred}} - k_{\text{true}}|/(k_e - \bar{k}) < 0.2$, ladder start 10 LT, decrement 0.5 LT); F-scores from $\geq 10^3$ starting points binned by prediction-time offset; feature-count matching rule — ESN nodes = NVAR features = quantum feature dimension $N_v F_{\text{step}}$ — so all read-outs train equally many weights; effective-rank column mandatory. Accept: battery end-to-end on a synthetic series, every number printed.

Phase 4 — MFE anchor study. Nine-mode MFE shear-flow model, $dt = 0.25$, $\Lambda = 0.0163$

(1 LT ≈ 61.3 time units), ensemble of 2000 series of 65 LT, laminarised series discarded ($k > k_l = 0.48$), training 25 series $\times$ 20 LT, test 500; extreme event $k \geq k_e = 0.1$. Sweeps: $n \in \{8, \dots, 11\}$, $N_v \in \{1, 2, 4, 8\}$, ε log-grid, τ two-decade log grid, γ set, $m \in \{1, 2, 3\}$, five seeds; comparators include the windowed protocol and the recurrent ring. Accept: the qualitative published finding reproduced or its failure reported; the Lorenz-63 check run and reported.

Phase 5 — solar ramp application. SURFRAD 1-min ground truth (aggregated to 5-min) primary, NSRDB 30-min secondary; registry vetting in full; clear-sky index k_t fit on the training span only; masking, never interpo-

lation. Ramp label $|k_t(t+h) - k_t(t)| \geq \rho$ with ρ the train-span 95th percentile, horizons 30 minutes to 4 hours; direct multi-horizon multitask read-outs; multichannel encoding across qubit groups; hybrid posture with quantum-only and classical-only ablation columns. Known hazard honoured: one-step point forecasting on solar is a four-way statistical tie with persistence, so event-level metrics are primary. Secondary honesty check: RF-QRC versus the recurrent ring on ENSO, expected loss declared. Accept: Section 55 evaluated once on the untouched test span.

Phase 6 — reuse integration. (a) Batch-and-compress through the Stage-5 pipeline, gated by the self-calibrated TVD rule; compression versus entangler topology including the all-to-all negative control; feature-interchangeability check repeated. (b) Virtual-node fusion prototype behind the IR extension, validated blockwise against unfused truncation. (c) Shot study, gated on the analytic-ceiling check, with the cost axis reported. Accept: as stated per item.

Phase 7 — documentation. This part and its results enter the handbook; citation audit before the PDF ships (the C-QRC preprint is cited only if it verifies); skills updated; handoff written.

55 Pre-registered success criteria and claim discipline

Fixed before test data is touched; evaluated once. (1) MFE: median PH of the best swept configuration at $n = 11$ exceeds the size-matched ESN plateau by $\geq 1.0\text{LT}$, exceeds the Haar control by $\geq 0.5\text{LT}$, and is $\geq \text{NVAR}$ at matched feature count; any clause failing, the corresponding null is the result. (2) Solar: at $h = 1$ and 2 hours, hybrid F_1 on ramp events exceeds the classical-only ablation by ≥ 0.03 absolute, quantum-only column reported; secondary NMSE on k_t with the mean line and persistence row present. (3) ENSO: reported, expected recurrence win. Claims permitted on success: event-level improvement at matched read-out capacity; width-efficiency (constant compiled width versus memory horizon); simulation-scoped. Claims banned regardless: quantum advantage or speed-up; hardware implications from Aer; any number not printed by a script.

56 Compute and risks

Exact simulation to $n = 11$ with N_v snapshots is cheap; the MFE ensemble and the sweep grid dominate (coarse-then-fine pruning). Feature dimensions up to $N_v 2^n \sim 8192$ against 10^3 – 10^4 training rows: ridge/GCV suffices, SVD truncation doubling as regularisation. Shots mode multiplies executions by N_v in the truncated realisation — exactly the cost the fusion prototype targets. Principal risks and responses: effective rank stagnant (report the curve; multichannel encoding is the lever); NVAR wins (report as headline; value then rests on width-efficiency and the hardware path); solar null (coherent beside a positive MFE anchor); IR extension complexity (deferred; batching delivers without it); preprint fails audit (drop the citation).

Part XI

From Pipeline to Product: Extreme-Climate Forecasting for Users

Part X delivers a validated forecast core. This part wraps it in the full system a user would touch — data plane, hybrid forecast core, decision layer, product surfaces, personas — adds implementation Phases 8–12, and integrates a survey of classical ML/DL and agentic-AI post-processing at the user boundary. The governance is inherited unchanged: pre-registered promotion gates, simulation-scoped claims, every number traceable to a run identifier, and the quantum contribution always reported as a delta against its own classical ablation.

57 Product definition

Two tracks, routed by measured memory requirements. Track A (operational extremes, minutes to hours): solar irradiance and PV-output ramp events driven by convective cloud fields, squalls, and haze — the extreme-climate signal at operational timescale, and the regime where the evidence of Part X locates QRC value; short-memory, high-nonlinearity targets, served by RF-QRC with virtual nodes. Flagship geography: tropical convective regimes (Singapore and South-East Asian rooftop and microgrid PV), where ramps are frequent, steep, and costly. Track B (tactical and seasonal extremes, weeks to months): ENSO phase transitions and anomalous-heat episodes; long-nonlinear-memory targets, served by the recurrent CZ-ring reservoir whose ENSO advantage this handbook measured (Part VII). The Part X “ENSO tension” thereby becomes an architecture-selection rule rather than an embarrassment.

Decisions fed and error costs, in writing. Track A feeds battery pre-charge and discharge, spinning-reserve commitment, curtailment, and intraday positions; a miss costs imbalance penalties, diesel spin-up, or voltage excursions, while a false alarm costs battery cycling and reserve over-procurement. Track B feeds seasonal hedging, maintenance windows, water and agricultural allocation, and parametric-insurance triggers.

Incumbents named. Persistence and smart-persistence (the honest one-step champion on solar — this project’s own calibration); NWP-derived

vendor forecasts; all-sky-imager nowcast systems, whose published ramp capture spans roughly 26–

92% with large false-alarm spreads — a genuinely weak incumbent frontier on ramps; and deep-learning weather models, which the product *consumes* as exogenous channels rather than competes with. The venture owns three things only: the event-focused hybrid feature layer, the calibrated decision layer that converts forecasts into cost-optimal alerts, and the benchmark-governance layer — the no-overclaiming discipline packaged as a feature. The quantum module sits behind an interface: if its pre-registered delta over the classical-only ablation fails at any promotion review, the product ships classical-only and the quantum track continues as research. Quantum-optional architecture is the de-risking posture.

Why RF-QRC specifically is the productisable QRC. Recurrent QRC carries quantum state across steps: inference is stateful, sequential, non-cacheable, and latency-bound to the quantum backend. RF-QRC step features depend only on the current quantised input window; memory is the classical leak. Features are therefore memoisable, per-step computation is stateless and parallel, and inference latency decouples from the backend. At product-relevant sizes ($n \leq 11$) the exact simulator *is* the production backend — fast, deterministic, and identical in distribution to ideal hardware — while the reuse compiler keeps the hardware path open for the regime where simulation fails, at compiled widths independent of memory horizon (Section 51.8). This is stated plainly to users: near-term inference is quantum-inspired simulation; hardware execution is a scaling option, not a marketing prop.

57.1 Personas and their contracts

Every persona contract names its metric before onboarding — the pre-registration discipline applied to customers.

58 End-to-end architecture

Table 14: Personas, decisions, and the metric each contract names.

Persona	Decision	Horizon	Surface	Metric that matters
microgrid / island-grid operator	battery and genset scheduling	15 min–4 h	alerts (chat/SM-S/webhook), dashboard	ramp F_1 at a fixed false-alarm budget; lead time
C&I rooftop energy manager	load shifting, demand-charge avoidance	30 min–6 h	dashboard, weekly digest	avoided cost per month
utility control room / VPP	reserve procurement, curtailment	1–24 h	API into EMS/S-CADA historian	pinball loss on quantiles; recall at capped false-alarm rate
energy trader	intraday position vs imbalance	1–12 h	API, scenario tool	cost-weighted CRPS; volatility-regime hit rate
climate-risk analyst / insurer	parametric triggers, seasonal exposure	weeks–months	reports, API	calibrated exceedance probabilities
researcher tier	benchmarking, method development	—	open harness, feature API	seed-exact reproducibility

```

1 L1 INGEST      L2 TRANSFORM      L3 FORECAST CORE      L4 DECISION      L5 SURFACES
2 SURFRAD/NSRDB QC (mask, never      classical features      calibration      API (REST/stream)
3 ERA5 / NOAA   interpolate)      + RF-QRC+VN features   (conformal,      dashboard
4 site SCADA -> clear-sky index -> via FeatureProvider -> isotonic)      -> alert engine
5 NWP + DL-      k_t (train-span    [sim|hw-batch|cache]    stacker (GBM/      agent assistant
6 weather feeds fit, frozen per    quantile+event heads   shallow DL)
7 (exogenous)    version); anomaly    shadow baseline        regime router
8                transforms;        battery (always on)     cost-sensitive
9                feature store      thresholds
10
11 L6 GOVERNANCE: drift + calibration monitors, effective-rank health,
12 champion-challenger, promotion gates = pre-registered criteria,
model cards, run-ID audit log

```

Listing 8: Six-layer architecture. L6 governance spans the whole pipeline.

W

```

1 class FeatureProvider(Protocol):
2     def features(self, window: np.ndarray, cfg: RFQRCConfig) -> np.ndarray: ...
3
4     # SimProvider      exact Aer snapshots (production default, n <= 11)
5     # HardwareProvider reuse-compiled batched jobs (greuse_batch -> compile_with_reuse),
6     #                  gated by the self-calibrated TVD equivalence check as CI
7     # CachedProvider   memo keyed by (config_hash, quantised window m); valid because
8     #                  RF-QRC step features are stateless; leak applied downstream

```

Listing 9: The forecast core’s only coupling to quantum execution.

W

```

1 { "site": "...", "event": "ramp_down", "horizon_min": 60,
2   "p_event": 0.83, "magnitude_q10_q50_q90": [-0.31, -0.22, -0.12],
3   "lead_time_min": 47, "regime": "convective_afternoon",
4   "conformal_coverage_target": 0.9, "run_id": "2026-07-11T04:20Z#a1b2",
5   "model_version": "hybrid-1.3.0",
6   "baseline_delta": {"vs_persistence_F1": +0.11} }

```

Listing 10: Alert object: honesty as payload.

L5, surfaces. REST and streaming API; a dashboard whose benchmark table is non-removable by design; alert channels; the agent assistant of Section 61.6.

L6, governance. Input-drift and rolling-coverage monitors; feature effective rank as a QRC-specific health metric (rank collapse is the leading indicator that quantum features have degenerated); alert-fatigue counters; retraining with walk-forward re-validation; promotion between model versions only through the pre-registered gates.

59 Implementation Phases 8–12

Phase 8 — data plane. Registry connectors and one private-feed adapter; QC service implementing the mask policy with gap reports; transform service with a frozen-statistics registry keyed by model version; a light feature store; a walk-forward replay harness driving everything through the same code path as live inference. Accept: bit-identical backtest reproduction from raw data with two commands; the leakage-audit script passes.

Phase 9 — serving core. `FeatureProvider` implementations (Sim, Cached; Hardware stub wired through the Stage-5 compiler as CI); quantile and event heads with per-horizon leak; adaptive conformal wrapper with a rolling-coverage monitor; shadow-battery service; latency budget of sub-second simulated features at $n \leq 11$ on a 5-minute cadence, alerts within 60 seconds of data arrival. Accept: rolling empirical coverage within three points of target on held-out streams; battery columns present in every stored prediction row; cache-hit accounting in logs.

Phase 10 — decision layer. Stacker ablation under the fairness rule (ridge versus gradient boosting versus shallow TCN/LSTM versus a small transformer, all offered to the classical-only ablation); regime router (clear-sky classification plus changepoint detection) with regime-conditional error reporting; a cost-sensitive threshold optimiser taking the persona cost matrix (expected-cost minimisation, not F_1 maximisation); alert engine with deduplication, escalation, and fatigue caps. Accept: decision-layer value reported in cost units alongside F_1 ; the stacker table includes the classical-only column, and if the quantum delta is zero the table says so.

Phase 11 — surfaces and agent. API with documentation and client snippets; dashboard; the assistant of Section 61.6 with its tool registry (`get_forecast`, `get_alert_rationale`, `run_whatif_backtest`, `get_benchmark_table`, `get_model_card`, `configure_site_draft`), retrieval index over model cards, run logs, and event post-mortems, the numeric-provenance guardrail, and the recommend-never-actuate boundary; report generator; onboarding copilot producing draft site configurations for human approval. Accept: a grounded-answer evaluation set passes with every numeric claim in sampled transcripts tracing to a tool call — zero fabricated numbers tolerated; time-to-first-alert for a new site under one day.

Phase 12 — pilot and promotion. One or two design-partner sites in shadow mode for at least six weeks against the operator’s incumbent, producing the honest benchmark table; pilot exit criteria written before the pilot starts (for example, ramp F_1 at least the incumbent’s plus 0.05 at the operator’s false-alarm budget, latency SLA met, zero provenance violations); activation on pass; quarterly promotion reviews re-running the pre-registered gates, including the contractual possibility of demoting the quantum feature block. Accept: the pilot report itself, whatever it says.

60 Economics and operational honesty

The cost axis is reported next to every accuracy number: simulated feature cost per prediction, cache-hit rate, and — for any hardware experiment — shots and wallclock per prediction; a model needing 10^9 shots to match a laptop has answered the business question. Public sources (SURFRAD, NSRDB, ERA5, NOAA) carry attribution requirements; site SCADA is private and stays on the customer boundary, with only derived features leaving if contracted. The degrade-graceful ladder — quantum features, then classical-only stacker, then smart-persistence — keeps the alert schema identical at every rung, so downgrades are invisible to integrations.

61 Survey: classical ML/DL and agentic AI at the post-processing end

61.1 Statistical post-processing

Raw regression outputs, reservoir or otherwise, are not calibrated decision inputs. The forecasting tradition supplies the fixes: distributional regression in the MOS/EMOS lineage, quantile regression on pinball loss, isotonic recalibration of event probabilities, CRPS as the proper score for the quantile fan. All are computationally trivial, all sit strictly downstream of the feature stream, and none care whether the features were quantum. Adopted: quantile heads plus isotonic recalibration.

61.2 Conformal prediction: guarantees instead of accuracy claims

Split and adaptive conformal methods wrap any point or quantile forecaster and deliver distribution-free finite-sample coverage; adaptive variants adjust the miscoverage rate online and so track regime shifts. The strategic fit is exact: the product can contractually promise “90-percent intervals that cover 90 percent, monitored weekly” — a guarantee — while accuracy claims remain comparative and scoped. Adopted: an adaptive conformal wrapper as the default interval layer, with the rolling-coverage monitor in L6.

61.3 Machine-learned stackers over hybrid features

Gradient-boosted trees remain the strongest tabular learners for stacking heterogeneous blocks: reservoir features, classical lags, calendar and geometry, NWP and DL-weather channels. The C-QRC preprint is the direct precedent — gate-QRC used as a feature extractor feeding LSTM, Transformer, and XGBoost heads, with the honest finding that gains concentrate in high-volatility and ramp regimes rather than average point accuracy. That is this project’s thesis stated by an independent group, and it fixes the stacker’s role: event-regime lift, not accuracy everywhere. Adopted: a gradient-boosted stacker as the default, under the fairness rule.

61.4 Deep heads and foundation models: two distinct roles

Sequence heads (TCN, LSTM, small transformers) are alternative stackers where horizon-coupled

multivariate structure matters, and are evaluated in the Phase-10 ablation rather than assumed. Time-series foundation models join the *baseline battery*: if a frozen pretrained forecaster matches the pipeline on event metrics, the honest table says so. Deep-learning weather models and classical NWP enter as *exogenous input providers* — their gridded forecasts become L2 channels — because the product’s edge is the event-focused last mile, not global weather modelling. This division of labour is the survey’s central architectural conclusion.

61.5 Regime-aware routing

Changepoint and hidden-Markov detectors with clear-sky classification gate per-regime read-outs and thresholds; regime-conditional error reporting doubles as the router’s evaluation. The Track A/B architecture routing of Section 57 is the same idea one level up: measured memory requirements select the reservoir.

61.6 Agentic AI at the user boundary

The 2024–2026 literature has converged on language-model agents as the interpretation and decision-support tier over energy forecasting systems, not as forecasters. A 2026 review of LLM-agent renewable forecasting (arXiv:2605.25141) proposes a six-layer taxonomy whose top layers are uncertainty estimation and natural-language reporting, and names hallucination control, drift, and EMS interoperability among twelve open challenges; prototype systems — GridMind for multi-agent power-system analysis (arXiv:2509.02494), Grid-Agent for grid control with stepwise rationales (arXiv:2508.05702), “Power Agent” concept papers, and forecasting-process automation frameworks — cluster into six patterns, of which this design adopts five and rejects one:

1. a conversational analyst over the forecast and metrics store (retrieval plus metric tools): questions about yesterday’s ramp are answered by tool calls whose numbers carry run identifiers — *adopted*;
2. an alert triage and explanation agent attaching rationale to each alert: regime label, leading feature attributions, retrieved analogue past events with outcomes — *adopted*, and the single highest-leverage adoption feature, because operators trust alerts they can interrogate;

3. a what-if and scenario agent orchestrating parameterised backtest calls (replay a monsoon week under a different alert policy and report the cost delta), the decision-theoretic direction of the review literature mapped onto the Phase-10 cost matrices — *adopted*;
4. a report and compliance generator drafting persona digests, post-event analyses, and re-validation reports from templates and tool-fetched numbers — *adopted*;
5. an onboarding copilot translating natural language into a draft site configuration for human approval — *adopted*, the easy-to-adopt lever, with time-to-first-alert measured in hours;
6. an autonomous control agent actuating switches or dispatch — *rejected*: the boundary is recommend-never-actuate with a human in the loop, and actuation belongs to the customer’s EMS and its safety case, not to a forecasting vendor’s language model.

Guardrails, non-negotiable and recognisably this project’s honesty discipline transposed: a numeric-provenance rule (the agent may state only numbers returned by tools, rendered with run identifiers, enforced by a verifier pass and audited with zero tolerance in Phase 11); uncertainty phrasing bound to the conformal coverage actually monitored; refusal templates for horizons and regimes outside validated scope; the benchmark table retrievable in one tool call, so “how much better than persistence?” is always answerable; and no advantage language — the agent describes quantum features as quantum-inspired simulated reservoir features with the measured ablation delta. Orchestration starts with a single tool-calling agent; a planner-verifier pair is added only if transcript audits show the need; tool interfaces follow an open tool-protocol convention so the assistant can later live inside the customer’s own chat tools.

61.7 Adoption design

An integration ladder in which each rung is a complete product: alerts by mail or chat with zero integration; webhook, CSV, and REST pull; EMS and SCADA integration through partner historians with adapters on the customer side; finally the assistant embedded in the operator’s chat tools. Trust artefacts shortcut sales: a six-week free shadow mode against the operator’s incumbent ending in the honest benchmark table — the

no-overclaiming discipline as the sales instrument; the open benchmark harness, so claims are independently re-runnable; and model cards carrying the pre-registered criteria and their outcomes, including nulls. Pricing is sketched per site, tiered by alert budget and horizon set, with a researcher tier at cost.

62 Risks specific to the product layer

63 Deliverables

Extending Part X’s D1–D6: D7, the data plane with its replay harness (Phase 8); D8, the serving core with its coverage monitor (Phase 9); D9, the decision layer with cost-unit evaluation (Phase 10); D10, the surfaces with a provenance-audited assistant (Phase 11); D11, the pilot report with pre-written exit criteria (Phase 12). Checkpoint hand-offs follow D8 and D10.

Table 15: Product-layer risks and responses.

Risk	Response
agent states a fabricated number	provenance verifier and zero-tolerance evaluation gate; numbers only via tools
alert fatigue kills adoption	cost-matrix thresholds, fatigue caps, weekly false-alarm budgets per persona
quantum delta ≈ 0 in production	contractual quantum-optional demotion; value rests on decision layer and governance either way
exogenous ramp drivers dominate	NWP and DL-weather channels in L2; sky-imager partnership on the roadmap, not the critical path
a foundation-model baseline matches the pipeline	it joins the battery and the table says so; the moat is then event-regime calibration and the decision layer, and the table proves whether that holds
data-licence creep on private feeds	derived-features-only export; raw stays on the customer boundary

References

- [1] Herbert Jaeger. The “echo state” approach to analysing and training recurrent neural networks. Technical Report GMD Report 148, German National Research Center for Information Technology, 2001.
- [2] Herbert Jaeger and Harald Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304:78–80, 2004.
- [3] Wolfgang Maass, Thomas Natschläger, and Henry Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14:2531–2560, 2002.
- [4] David Verstraeten, Benjamin Schrauwen, Michiel D’Haene, and Dirk Stroobandt. An experimental unification of reservoir computing methods. *Neural Networks*, 20:391–403, 2007.
- [5] Mantas Lukoševičius and Herbert Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3:127–149, 2009.
- [6] Jaideep Pathak, Brian Hunt, Michelle Girvan, Zhixin Lu, and Edward Ott. Model-free prediction of large spatiotemporally chaotic systems from data: A reservoir computing approach. *Physical Review Letters*, 120:024102, 2018.
- [7] Troy Arcomano, Istvan Szunyogh, Jaideep Pathak, Alexander Wikner, Brian R. Hunt, and Edward Ott. A machine learning-based global atmospheric forecast model. *Geophysical Research Letters*, 47:e2020GL087776, 2020.
- [8] Pantelis R. Vlachas, Jaideep Pathak, Brian R. Hunt, Themistoklis P. Sapsis, Michelle Girvan, Edward Ott, and Petros Koumoutsakos. Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics. *Neural Networks*, 126:191–217, 2020.
- [9] Herbert Jaeger, Mantas Lukoševičius, Dan Popovici, and Udo Siewert. Optimization and applications of echo state networks with leaky-integrator neurons. *Neural Networks*, 20:335–352, 2007.
- [10] Shumpei Kobayashi, Quoc Hoan Tran, and Kohei Nakajima. Extending echo state property for quantum reservoir computing. *Physical Review E*, 110:024207, 2024.
- [11] Stephen Boyd and Leon O. Chua. Fading memory and the problem of approximating nonlinear operators with Volterra series. *IEEE Transactions on Circuits and Systems*, 32:1150–1161, 1985.
- [12] Joni Dambre, David Verstraeten, Benjamin Schrauwen, and Serge Massar. Information processing capacity of dynamical systems. *Scientific Reports*, 2:514, 2012.
- [13] Lyudmila Grigoryeva and Juan-Pablo Ortega. Echo state networks are universal. *Neural Networks*, 108:495–508, 2018.
- [14] Jiayin Chen and Hendra I. Nurdin. Learning nonlinear input–output maps with dissipative quantum systems. *Quantum Information Processing*, 18:198, 2019.
- [15] Johannes Nokkala, Rodrigo Martínez-Peña, Gian Luca Giorgi, Valentina Parigi, Miguel C. Soriano, and Roberta Zambrini. Gaussian states of continuous-variable quantum systems provide universal and versatile reservoir computing. *Communications Physics*, 4:53, 2021.
- [16] Rodrigo Martínez-Peña and Juan-Pablo Ortega. Quantum reservoir computing in finite dimensions. *Physical Review E*, 107:035306, 2023.

- [17] Joschka Boedecker, Oliver Obst, Joseph T. Lizier, N. Michael Mayer, and Minoru Asada. Information processing in echo state networks at the edge of chaos. *Theory in Biosciences*, 131:205–213, 2012.
- [18] Ludvig Storm, Kristian Gustavsson, and Bernhard Mehlig. Constraints on parameter choices for successful time-series prediction with echo-state networks. *Machine Learning: Science and Technology*, 3:045021, 2022.
- [19] Joschka Herteux and Christoph R  th. Breaking symmetries of the reservoir equations in echo state networks. *Chaos*, 30:123142, 2020.
- [20] Kaito Kobayashi and Yukitoshi Motome. Edge of many-body quantum chaos in quantum reservoir computing. *Physical Review Letters*, 136:040602, 2026.
- [21] Michael C. Mackey and Leon Glass. Oscillation and chaos in physiological control systems. *Science*, 197:287–289, 1977.
- [22] Jaideep Pathak, Alexander Wikner, Rebecca Fussell, Sarthak Chandra, Brian R. Hunt, Michelle Girvan, and Edward Ott. Hybrid forecasting of chaotic processes: Using machine learning in conjunction with a knowledge-based model. *Chaos*, 28:041101, 2018.
- [23] Edward N. Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20:130–141, 1963.
- [24] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.
- [25] Heinz-Peter Breuer and Francesco Petruccione. *The Theory of Open Quantum Systems*. Oxford University Press, 2002.
- [26] Antonio Sannia, Rodrigo Mart  nez-Pe  a, Miguel C. Soriano, Gian Luca Giorgi, and Roberta Zambrini. Dissipation as a resource for quantum reservoir computing. *Quantum*, 8:1291, 2024.
- [27] Shumpei Kobayashi, Quoc Hoan Tran, and Kohei Nakajima. Coherence influx is indispensable for quantum reservoir computing. *arXiv:2409.12693*, 2024.
- [28] G  ran Lindblad. On the generators of quantum dynamical semigroups. *Communications in Mathematical Physics*, 48:119–130, 1976.
- [29] Vittorio Gorini, Andrzej Kossakowski, and E. C. George Sudarshan. Completely positive dynamical semigroups of N -level systems. *Journal of Mathematical Physics*, 17:821–825, 1976.
- [30] Laia Domingo, Gabriel Carlo, and Florentino Borondo. Optimal quantum reservoir computing for the noisy intermediate-scale quantum era. *Physical Review E*, 106:L043301, 2022.
- [31] Weijie Xiong, Zo   Holmes, Armando Angrisani, Yudai Suzuki, Thiparat Chotibut, and Supanut Thanasilp. Role of scrambling and noise in temporal information processing with quantum systems. *arXiv:2505.10080*, 2025.
- [32] Keisuke Fujii and Kohei Nakajima. Harnessing disordered-ensemble quantum dynamics for machine learning. *Physical Review Applied*, 8:024030, 2017.
- [33] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103:032430, 2021.
- [34] Kaito Kobayashi, Keisuke Fujii, and Naoki Yamamoto. Feedback-driven quantum reservoir computing for time-series analysis. *PRX Quantum*, 5:040325, 2024.
- [35] Giacomo Franceschetto, Marcin P  łodzie  n, Maciej Lewenstein, Antonio Ac  n, and Pere Mujal. Harnessing quantum back-action for time-series processing. *arXiv:2411.03979*, 2024.
- [36] Pere Mujal, Rodrigo Mart  nez-Pe  a, Johannes Nokkala, Jorge Garc  a-Ben  , Gian Luca Giorgi, Miguel C. Soriano, and Roberta Zambrini. Opportunities in quantum reservoir computing and extreme learning machines. *Advanced Quantum Technologies*, 4:2100027, 2021.
- [37] Jiayin Chen, Hendra I. Nurdin, and Naoki Yamamoto. Temporal information processing on noisy quantum computers. *Physical Review Applied*, 14:024065, 2020.

- [38] Kohei Nakajima, Keisuke Fujii, Makoto Negoro, Kosuke Mitarai, and Masahiro Kitagawa. Boosting computational power through spatial multiplexing in quantum reservoir computing. *Physical Review Applied*, 11:034021, 2019.
- [39] Pere Mujal, Rodrigo Martínez-Peña, Gian Luca Giorgi, Miguel C. Soriano, and Roberta Zambrini. Time-series quantum reservoir computing with weak and projective measurements. *npj Quantum Information*, 9:16, 2023.
- [40] Makoto Negoro, Kosuke Mitarai, Keisuke Fujii, Kohei Nakajima, and Masahiro Kitagawa. Machine learning with controllable quantum dynamics of a nuclear spin ensemble in a solid. *arXiv:1806.10910*, 2018.
- [41] Fangjun Hu, Saeed A. Khan, Nicholas T. Bronn, Gerasimos Angelatos, Graham E. Rowlands, Guilhem J. Ribeill, and Hakan E. Türeci. Overcoming the coherence time barrier in quantum machine learning on temporal data. *Nature Communications*, 15:7491, 2024.
- [42] Fangjun Hu, Gerasimos Angelatos, Saeed A. Khan, Marti Vives, Esin Türeci Dagli, Leon Bello, Graham E. Rowlands, Guilhem J. Ribeill, and Hakan E. Türeci. Tackling sampling noise in physical systems for machine learning applications: Fundamental limits and eigentasks. *Physical Review X*, 13:041020, 2023.
- [43] Osama Ahmed, Felix Tennie, and Luca Magri. Optimal training of finitely sampled quantum reservoir computers for forecasting of chaotic dynamics. *Quantum Machine Intelligence*, 7:31, 2025.
- [44] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018.
- [45] Antonio Sannia, Gian Luca Giorgi, and Roberta Zambrini. Exponential concentration and symmetries in quantum reservoir computing. *arXiv:2505.10062*, 2025.
- [46] Aki Kutvonen, Keisuke Fujii, and Takahiro Sagawa. Optimizing a quantum reservoir computer for time series prediction. *Scientific Reports*, 10:14687, 2020.
- [47] Rodrigo Martínez-Peña, Gian Luca Giorgi, Johannes Nokkala, Miguel C. Soriano, and Roberta Zambrini. Dynamical phase transitions in quantum reservoir computing. *Physical Review Letters*, 127:100502, 2021.
- [48] Toshiki Yasuda, Yudai Suzuki, Tomoyuki Kubota, Kohei Nakajima, Qi Gao, Wenlong Zhang, Satoshi Shimono, Hendra I. Nurdin, and Naoki Yamamoto. Quantum reservoir computing with repeated measurements on superconducting devices. *arXiv:2310.06706*, 2023.
- [49] Quoc Hoan Tran and Kohei Nakajima. Higher-order quantum reservoir computing. *arXiv:2006.08999*, 2020.
- [50] Saša Čindrak, Brecht Donvil, Kathy Lüdge, and Lina Jaurigue. Enhancing the performance of quantum reservoir computing and solving the time-complexity problem by artificial memory restriction. *Physical Review Research*, 6:013051, 2024.
- [51] Guillem Llodrà, Pere Mujal, Roberta Zambrini, and Gian Luca Giorgi. Quantum reservoir computing in atomic lattices. *Chaos, Solitons & Fractals*, 195:116289, 2025.
- [52] Osama Ahmed, Felix Tennie, and Luca Magri. Prediction of chaotic dynamics and extreme events: A recurrence-free quantum reservoir computing approach. *Physical Review Research*, 6:043082, 2024.
- [53] Ambrose Odo, Tirthak Pathak, Nouhaila Inan, and Muhammad Shafique. Quantum reservoir computing for short-term power load forecasting in resource-constrained energy systems. *arXiv:2606.12806*, 2026.
- [54] Gerasimos Angelatos, Saeed A. Khan, and Hakan E. Türeci. Reservoir computing approach to quantum state measurement. *Physical Review X*, 11:041062, 2021.
- [55] Yudai Suzuki, Qi Gao, Ken C. Pradel, Kenji Yasuoka, and Naoki Yamamoto. Natural quantum reservoir computing for temporal information processing. *Scientific Reports*, 12:1353, 2022.
- [56] Daniel Fry, Amol Deshmukh, Samuel Yen-Chi Chen, Vladimir Rastunkov, and Vanio Markov. Optimizing quantum noise-induced reservoir computing for nonlinear and chaotic time series prediction. *arXiv:2303.05488*, 2023.

- [57] Iris Paparelle, Johan Henaff, Jorge Garcia-Beni, Emilie Gillet, Gian Luca Giorgi, Miguel C. Soriano, Roberta Zambrini, and Valentina Parigi. Experimental memory control in continuous variable optical quantum reservoir computing. *arXiv:2506.07279*, 2025.
- [58] Jorge García-Beni, Gian Luca Giorgi, Miguel C. Soriano, and Roberta Zambrini. Scalable photonic platform for real-time quantum reservoir computing. *Physical Review Applied*, 20:014051, 2023.
- [59] Valeria Cimini, Mandar M. Sohoni, Federico Presutti, Benjamin K. Malia, Shi-Yuan Ma, Ryotatsu Yanagimoto, Tianyu Wang, Tatsuhiko Onodera, Logan G. Wright, and Peter L. McMahon. Large-scale quantum reservoir computing using a Gaussian boson sampler. *arXiv:2505.13695*, 2025.
- [60] Julien Dudas, Baptiste Carles, Erwan Plouet, Frank Alice Mizrahi, Julie Grollier, and Danijela Marković. Quantum reservoir computing implementation on coherently coupled quantum oscillators. *npj Quantum Information*, 9:64, 2023.
- [61] Niclas Götting et al. Connection between memory performance and optical absorption in quantum reservoir computing. *Physical Review Letters*, 135:240403, 2025.
- [62] Rodrigo Araiza Bravo, Khadijeh Najafi, Xun Gao, and Susanne F. Yelin. Quantum reservoir computing using arrays of Rydberg atoms. *PRX Quantum*, 3:030325, 2022.
- [63] Milan Kornjača, Hong-Ye Hu, Chen Zhao, Jonathan Wurtz, Phillip Weinberg, et al. Large-scale quantum reservoir learning with an analog quantum computer. *arXiv:2407.02553*, 2024.
- [64] Dolev Bluvstein, Simon J. Evered, Alexandra A. Geim, Sophie H. Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, et al. Logical quantum processor based on reconfigurable atom arrays. *Nature*, 626:58–65, 2024.
- [65] Chuanzhou Zhu, Peter J. Ehlers, Hendra I. Nurdin, and Daniel Soh. Practical few-atom quantum reservoir computing. *Physical Review Research*, 7:023290, 2025.
- [66] Alen Senanian, Sridhar Prabhu, Vladimir Kremenetski, Saswata Roy, Yingkang Cao, Jeremy Kroeze, Logan Chitambar, Logan G. Wright, and Peter L. McMahon. Microwave signal processing using an analog quantum reservoir computer. *Nature Communications*, 15:7490, 2024.
- [67] Steven A. Moses et al. A race-track trapped-ion quantum processor. *Physical Review X*, 13:041052, 2023.
- [68] Jacopo Settimo, L. Salatino, L. Mariani, M. Channab, L. Bozzolo, S. Vallisa, P. Barillà, A. Policicchio, N. Lo Gullo, A. Giordano, C. Mastroianni, and F. Plastina. Memory-augmented hybrid quantum reservoir computing. *Physical Review Applied*, 24:024019, 2025.
- [69] Philipp Pfeffer, Florian Heyder, and Jörg Schumacher. Hybrid quantum-classical reservoir computing of thermal convection flow. *Physical Review Research*, 4:033176, 2022.
- [70] Filip Wudarski, Jeffrey Marshall, Andre Petukhov, and Eleanor Rieffel. Hybrid quantum-classical reservoir computing for simulating chaotic systems. *arXiv:2311.14105*, 2023.
- [71] Adrian Parra-Rodriguez, Pavel Lougovski, Lucas Lamata, Enrique Solano, and Mikel Sanz. Digital-analog quantum computation. *Physical Review A*, 101:022305, 2020.
- [72] Niclas Götting, Frederik Lohof, and Christopher Gies. Exploring quantumness in quantum reservoir computing. *Physical Review A*, 108:052427, 2023.
- [73] Ana Palacios, Rodrigo Martínez-Peña, Miguel C. Soriano, Gian Luca Giorgi, and Roberta Zambrini. Role of coherence in many-body quantum reservoir computing. *Communications Physics*, 7:369, 2024.
- [74] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98:032309, 2018.
- [75] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.

- [76] J. Robert Johansson, Paul D. Nation, and Franco Nori. QuTiP: An open-source Python framework for the dynamics of open quantum systems. *Computer Physics Communications*, 183:1760–1772, 2012.
- [77] Yasunari Suzuki, Yoshiaki Kawase, Yuya Masumura, Yuria Hiraga, Masahiro Nakadai, et al. Qulacs: A fast and versatile quantum circuit simulator for research purpose. *Quantum*, 5:559, 2021.
- [78] Shi-Xin Zhang, Jonathan Allcock, Zhou-Quan Wan, Shuo Liu, Jiajie Sun, Hao Yu, et al. TensorCircuit: A quantum software framework for the NISQ era. *Quantum*, 7: 912, 2023.
- [79] Nathan Killoran, Josh Izaac, Nicolás Quesada, Ville Bergholm, Matthew Amy, and Christian Weedbrook. Strawberry Fields: A software platform for photonic quantum computing. *Quantum*, 3:129, 2019.
- [80] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter J. Karalekas, Andre A. Alves, et al. Mitiq: A software package for error mitigation on noisy quantum computers. *Quantum*, 6:774, 2022.
- [81] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. Evidence for the utility of quantum computing before fault tolerance. *Nature*, 618:500–505, 2023.
- [82] Lars S. Madsen, Fabian Laudenbach, Mohsen Falamarzi Askarani, Fabien Rortais, Trevor Vincent, Jacob F. F. Bulmer, Filippo M. Miatto, Leonhard Neuhaus, Lukas G. Helt, Matthew J. Collins, et al. Quantum computational advantage with a programmable photonic processor. *Nature*, 606:75–81, 2022.
- [83] Rich H. Inman, Hugo T. C. Pedro, and Carlos F. M. Coimbra. Solar forecasting methods for renewable energy integration. *Progress in Energy and Combustion Science*, 39:535–576, 2013.
- [84] Dazhi Yang, Jan Kleissl, Christian A. Gueymard, Hugo T. C. Pedro, and Carlos F. M. Coimbra. History and trends in solar irradiance and PV power forecasting: A preliminary assessment and review using text mining. *Solar Energy*, 168:60–101, 2018.
- [85] Tao Hong and Shu Fan. Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting*, 32:914–938, 2016.
- [86] Aoife M. Foley, Paul G. Leahy, Antonino Marvuglia, and Eamon J. McKeogh. Current methods and advances in forecasting of wind power generation. *Renewable Energy*, 37:1–8, 2012.
- [87] Stephan Rasp, Peter D. Dueben, Sebastian Scher, Jonathan A. Weyn, Soukayna Mouatadid, and Nils Thuerey. WeatherBench: A benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems*, 12: e2020MS002203, 2020.
- [88] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. Learning skillful medium-range global weather forecasting. *Science*, 382: 1416–1421, 2023.
- [89] Philipp Pfeffer, Florian Heyder, and Jörg Schumacher. Reduced-order modeling of two-dimensional turbulent Rayleigh–Bénard flow by hybrid quantum-classical reservoir computing. *Physical Review Research*, 5: 043242, 2023.
- [90] Batuhan Hangun et al. Quantum neural networks for wind energy forecasting: A comparative study of performance and scalability with classical models. *arXiv:2506.22845*, 2025.
- [91] Batuhan Hangun, Emine Akpınar, Oguz Altun, and Onder Eyecioglu. Comparative analysis of QNN architectures for wind power prediction: Feature maps and ansatz configurations. *arXiv:2506.14795*, 2025.
- [92] Felix Tennie and Tim N. Palmer. Quantum computers for weather and climate prediction: The good, the bad, and the noisy. *arXiv:2210.17460*, 2022.
- [93] Akshay Ajagekar and Fengqi You. Quantum computing for energy systems optimization: Challenges and opportunities. *Energy*, 179: 76–89, 2019.

- [94] Yuxuan Zhou, Zefan Tang, Nima Nikmehr, Pouya Babahajiani, Fei Feng, Tzu-Chieh Wei, Honghao Zheng, and Peng Zhang. Quantum computing in power systems. *iEnergy*, 1:170–187, 2022.
- [95] Manajit Sengupta, Yu Xie, Anthony Lopez, Aron Habte, Galen Maclaurin, and James Shelby. The National Solar Radiation Data Base (NSRDB). *Renewable and Sustainable Energy Reviews*, 89:51–60, 2018.
- [96] John A. Augustine, John J. DeLuisi, and Charles N. Long. SURFRAD—a national surface radiation budget network for atmospheric research. *Bulletin of the American Meteorological Society*, 81:2341–2357, 2000.
- [97] Caroline Draxl, Andrew Clifton, Bri-Mathias Hodge, and Jim McCaa. The Wind Integration National Dataset (WIND) Toolkit. *Applied Energy*, 151:355–366, 2015.
- [98] Tao Hong, Pierre Pinson, Shu Fan, Hamidreza Zareipour, Alberto Troccoli, and Rob J. Hyndman. Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond. *International Journal of Forecasting*, 32:896–913, 2016.
- [99] Artur Trindade. ElectricityLoadDiagrams20112014 data set. UCI Machine Learning Repository, 2015.
- [100] Lion Hirth, Jonathan Mühlenpfordt, and Marisa Bulkeley. The ENTSO-E Transparency Platform—a review of Europe’s most ambitious electricity data platform. *Applied Energy*, 225:1054–1067, 2018.
- [101] Frauke Wiese, Ingmar Schlecht, Wolf-Dieter Bunke, Clemens Gerbaulet, Lion Hirth, Martin Jahn, Friedrich Kunz, Casimir Lorenz, Jonathan Mühlenpfordt, Juliane Reimann, and Wolf-Peter Schill. Open Power System Data: Frictionless data for electricity system modelling. *Applied Energy*, 236:401–409, 2019.
- [102] Vadim Arzamasov, Klemens Böhm, and Patrick Jochem. Towards concise models of grid stability. In *2018 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2018.
- [103] Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Murathan Aykol, Patrick K. Herring, Dimitrios Fraggdakis, et al. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4:383–391, 2019.
- [104] Bhaskar Saha and Kai Goebel. Battery data set. NASA Ames Prognostics Data Repository, 2007.
- [105] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, Carole Peubey, Raluca Radu, Dinand Schepers, et al. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146:1999–2049, 2020.
- [106] Max Planck Institute for Biogeochemistry. Weather station data, Jena, Germany. <https://www.bgc-jena.mpg.de/wetter/>, 2026.
- [107] Veronika Eyering, Sandrine Bony, Gerald A. Meehl, Catherine A. Senior, Bjorn Stevens, Ronald J. Stouffer, and Karl E. Taylor. Overview of the Coupled Model Intercomparison Project Phase 6 (CMIP6) experimental design and organization. *Geoscientific Model Development*, 9:1937–1958, 2016.
- [108] Max J. Suarez and Paul S. Schopf. A delayed action oscillator for ENSO. *Journal of the Atmospheric Sciences*, 45:3283–3287, 1988.